Co-Simulation with Third-Party HDL Simulators

Contents

1. Preface
   1.1. About This Manual
   1.2. Documentation Conventions
   1.3. Terminology
   1.4. Customer Support
   1.5. Synopsys Statement on Inclusivity and Diversity
2. Chapter 1 Introduction
3. Chapter 2 HDL CoSimulation
   3.1. 2.1 Design Flow
   3.2. 2.2 Rules for SystemC-HDL Co-Simulation
   3.3. 2.3 Setting up the Environment for HDL Simulators
   3.4. 2.4 Mapping Ports
      3.4.1. 2.4.1 Connecting the Ports
      3.4.2. 2.4.2 Mapping Signal Types
      3.4.3. 2.4.3 Mapping Generic Values (VHDL) and Module Parameters (Verilog)
   3.5. 2.5 Instantiating HDL Models in a SystemC Design
      3.5.1. 2.5.1 Defining the SystemC Module
      3.5.2. 2.5.2 Specifying the HDL Module Name
      3.5.3. 2.5.3 Specifying the Library Name
      3.5.4. 2.5.4 Mapping Ports
      3.5.5. 2.5.5 Specifying the HDL Source Files
      3.5.6. 2.5.6 Specifying the Component List Package Name
      3.5.7. 2.5.7 Specifying the Architecture
      3.5.8. 2.5.8 Specifying the HDL Language
      3.5.9. 2.5.9 Checking the Source Type
   3.6. 2.6 Creating an HDL Proxy Module Automatically
      3.6.1. 2.6.1 Mapping HDL Ports to HDL Proxy Module Ports
      3.6.2. 2.6.2 Using VHDL Generic Values and Verilog Module Parameters
      3.6.3. Syntax
      3.6.4. Examples
      3.6.5. Syntax
      3.6.6. Examples
      3.6.7. Syntax
      3.6.8. Example
      3.6.9. Syntax
      3.6.10. Example
      3.6.11. Syntax
      3.6.12. Examples
      3.6.13. Syntax
      3.6.14. Example
      3.6.15. Syntax
      3.6.16. Example
      3.6.17. Syntax
      3.6.18. Example
      3.6.19. Syntax
      3.6.20. Examples
      3.6.21. Syntax
      3.6.22. Examples
      3.6.23. Syntax
      3.6.24. Example
      3.6.25. 2.6.3 Limitations

3.7. 2.7 Instantiating SystemC Models in an HDL Design
3.7.1. 2.7.1 Specifying SystemC Source File Names
3.7.2. 2.7.2 Specifying Module Name
3.7.3. 2.7.3 Specifying Header File Name
3.7.4. 2.7.4 Mapping Ports
3.7.5. 2.7.5 Referencing SystemC Modules
3.7.6. 2.7.6 Specifying Compile-Time Options
3.8. 2.8 Coding for Both SystemC-Only and Mixed-Mode HDL Simulation
3.9. 2.9 Creating a Wrapper for a Design Whose Top Level is an HDL Model
3.10. 2.10 Creating SystemC Proxy Modules from SystemC Designs
3.10.1. 2.10.1 Introducing the SystemC Proxy Generator
3.10.2. 2.10.2 Generating a SystemC Proxy Module
3.10.3. Syntax
3.10.4. Examples
3.10.5. Syntax
3.10.6. Examples
3.10.7. Syntax
3.10.8. Examples
3.10.9. Syntax
3.10.10. Examples
3.10.11. Syntax
3.10.12. Examples
3.10.13. Syntax
3.10.14. Examples
3.10.15. Syntax
3.10.16. Examples
3.10.17. Syntax
3.10.18. Examples
3.10.19. Syntax
3.10.20. Example
3.10.21. Syntax
3.10.22. Examples
3.10.23. 2.10.3 Co-Simulation Boundary Port Types
3.10.24. 2.10.4 Co-Simulation Boundary Port Data Types
3.10.25. 2.10.5 Examples of Creating SystemC Proxy Modules
3.10.26. 2.10.6 Limitations
3.11. 2.11 SystemC-HDL Co-Simulation API Reference
3.11.1. 2.11.1 Constructor
3.11.2. 2.11.2 Public Member Functions
3.11.3. 2.11.3 Member Function Documentation
3.12. 2.12 Coding Examples
3.12.1. 2.12.1 Coding Examples for HDL Proxy Modules in SystemC Designs
3.12.2. 2.12.2 Coding Examples for a SystemC Proxy Model in HDL Designs
3.13. 2.13 Building and Running a Simulation
3.13.1. 2.13.1 Specifying the Number of Language Switches
3.13.2. 2.13.2 Loading the Design
3.13.3. 2.13.3 Specifying Co-Simulation Time-Out
3.13.4. 2.13.4 Selecting the HDL Simulator
3.13.5. 2.13.5 Selecting the Simulator Mode
3.13.6. 2.13.6 Setting Simulation Options
3.13.7. 2.13.7 Setting Command-Line Arguments
3.13.8. 2.13.8 Selecting the Synchronization Mode
3.13.9. 2.13.9 Mapping Logical Library Names to Physical Directories
3.13.10. 2.13.10 ModelSim/Questa Support for HDL Simulator Simulation Control
3.13.11. 2.13.11 Managing Errors and Warnings
3.13.12. 2.13.12 Running the Simulation
3.14. 2.14 Debugging a SystemC-HDL Co-Simulation
3.14.1. 2.14.1 Using VP Explorer

3.14.2. 2.14.2 Using the SystemC Shell
3.15. 2.15 Exporting Structural HDL
3.15.1. 2.15.1 SystemC Shell Command to Export Structural HDL
3.15.2. 2.15.2 Possible Use-Model Scenarios
3.15.3. 2.15.3 Debugging and Simulation Control
3.16. 2.16 SystemC Shell Commands
3.16.1. 2.16.1 Overview
3.16.2. 2.16.2 cosim::map_hdl_library_name
3.16.3. Syntax
3.16.4. Arguments
3.16.5. Example
3.16.6. 2.16.3 cosim::set_cosim_timeout
3.16.7. Syntax
3.16.8. Arguments
3.16.9. Example
3.16.10. 2.16.4 cosim::set_delta_cycle_sync
3.16.11. Syntax
3.16.12. Arguments
3.16.13. 2.16.5 cosim::set_hdl_max_nesting
3.16.14. Syntax
3.16.15. Arguments
3.16.16. Example
3.16.17. 2.16.6 cosim::set_hdl_options
3.16.18. Syntax
3.16.19. Arguments
3.16.20. 2.16.7 cosim::set_hdl_sim_mode
3.16.21. Syntax
3.16.22. Arguments
3.16.23. 2.16.8 cosim::set_hdl_sim_type
3.16.24. Syntax
3.16.25. Arguments
3.16.26. Example
3.16.27. 2.16.9 cosim::set_verilog_timescale
3.16.28. Syntax
3.16.29. Arguments
3.16.30. Example
3.16.31. 2.16.10 cosim::show_hdl_options
3.16.32. Syntax
3.16.33. Arguments
3.16.34. 2.16.11 get_hdl_lib_map
3.16.35. Syntax
3.16.36. Arguments
3.16.37. Example
3.16.38. 2.16.12 hdl-options
3.16.39. Syntax
3.16.40. Arguments
3.16.41. 2.16.13 map_hdl_library_name
3.16.42. Syntax
3.16.43. Arguments
3.16.44. Example
3.16.45. 2.16.14 run-args
3.16.46. Syntax
3.16.47. Arguments
3.16.48. 2.16.15 set_cosim_timeout
3.16.49. Syntax
3.16.50. Arguments
3.16.51. Example
3.16.52. 2.16.16 set_delta_cycle_sync

- 3.16.53. Syntax
- 3.16.54. Arguments
- 3.16.55. 2.16.17 set_hdl_max_nesting
- 3.16.56. Syntax
- 3.16.57. Arguments
- 3.16.58. Example
- 3.16.59. 2.16.18 set_hdl_options
- 3.16.60. Syntax
- 3.16.61. Arguments
- 3.16.62. Examples
- 3.16.63. 2.16.19 set_hdl_sim_mode
- 3.16.64. Syntax
- 3.16.65. Arguments
- 3.16.66. 2.16.20 set_hdl_sim_type
- 3.16.67. Syntax
- 3.16.68. Arguments
- 3.16.69. Example
- 3.16.70. 2.16.21 set_verilog_timescale
- 3.16.71. Syntax
- 3.16.72. Arguments
- 3.16.73. Example
- 3.16.74. 2.16.22 show_hdl_options
- 3.16.75. Syntax
- 3.16.76. Arguments
- 3.16.77. 2.16.23 simargs
- 3.16.78. Syntax
- 3.16.79. Arguments
- 3.16.80. Example
- 3.16.81. 2.16.24 set_uvmc_enabled
- 3.16.82. Syntax
- 3.16.83. Arguments
- 3.17. 2.17 Profiling a SystemC-HDL Co-Simulation
- 3.18. 2.18 Limitations and Known Problems
  - 3.18.1. 2.18.1 Limitations
  - 3.18.2. 2.18.2 Known Problems
- 3.19. 2.19 Performance Optimization with VCS Co-Simulation
- 3.20. 2.20 Inout Network Resolution Mechanism
  - 3.20.1. 2.20.1 Usage
- 3.21. 2.21 Example VDK in Virtualizer Studio

4. Chapter 3 VCS-Centric HDL Co-Simulation
   - 4.1. 3.1 Overview of the VCS-Centric HDL Co-Simulation flow
   - 4.2. 3.2 Defining the Cross-Language Interface
     - 4.2.1. 3.2.1 Definition in Virtualizer Studio
     - 4.2.2. 3.2.2 Manual Definition
   - 4.3. 3.3 Building the Simulation
     - 4.3.1. 3.3.1 Building the VDK Subsystem
     - 4.3.2. 3.3.2 Building the HDL Design
   - 4.4. 3.4 Running the Simulation
     - 4.4.1. 3.4.1 Launching Using VP Config
     - 4.4.2. 3.4.2 Standalone launch
   - 4.5. 3.5 Example VDK in Virtualizer Studio
   - 4.6. 3.6 Limitations
5. Chapter 4 UVM-Connect
   - 5.1. 4.1 Creating a UVMC Connection in Virtualizer Studio
   - 5.2. 4.2 Creating a UVMC Connection in SystemVerilog
   - 5.3. 4.3 Enabling UVMC for a VCS-Centric Co-Simulation
   - 5.4. 4.4 Example VDK in Virtualizer Studio
   - 5.5. 4.5 Limitations

6. Chapter 5 Turbo Mode
6.1. 5.1 Configuration Parameters
6.1.1. 5.1.1 SystemC/HDL Speed Ratio and Throttle Window Length
6.1.2. 5.1.2 Synchronous Window Length
6.2. 5.2 Support for Clocked Designs
6.2.1. 5.2.1 Using Separate SystemC and HDL Clock Sources
6.3. 5.3 Enabling and Configuring the Turbo Mode for Co-Simulation
6.4. 5.4 Limitations
7. Index

1. Preface

The preface of the *Co-Simulation with Third-Party HDL Simulators* manual describes:

- About This Manual
- Documentation Conventions
- Terminology
- Customer Support
- Synopsys Statement on Inclusivity and Diversity

1.1. About This Manual

This manual describes the different types of co-simulation that are offered by the Platform Architect/Virtualizer product family.

DEPRECATED: The environment variable `COWARE_CXX_COMPILER` is deprecated. You are recommended to use the variable `SNPS_VP_COMPILER` instead. For more details, see *Platform Architect Release Notes*, *Virtualizer Release Notes* and *SystemC Shell Manual*.

This manual is organized as follows:

- Introduction provides an overview of the available co-simulations and how to use.
- HDL CoSimulation describes SystemC-HDL co-simulation, which allows you to co-simulate SystemC and HDL models. SystemC-HDL co-simulation is an extra-priced option.
- VCS-Centric HDL Co-Simulation describes the VCS-centric HDL co-simulation flow that allows the VDK engineer and RTL verification engineer to author independent design hierarchies, each with their own set of tools, and to facilitate co-simulation through an interface specification they agree upon.
- UVM-Connect describes an open source library developed by Mentor which enables transaction-level communication between SystemC models and SystemVerilog testbenches and provides a UVM Command API that allows controlling a UVM simulation from SystemC.
- Turbo Mode describes the purpose of the turbo mode to improve performance by decoupling the SystemC and HDL portions, letting SystemC run ahead in simulated time and skipping the costly simulation of the idling RTL.

1.2. Documentation Conventions

This section lists and explains the documentation conventions used throughout this manual.

Convention	Description and Examples
italic	Is used in running text for: • GUI elements. For example:

	The *Enumeration* field contains a space-separated list of values. • New terms. For example: A *protocol library* is a collection of protocol definitions. • Web sites. For example: For more information, see *www.eclipse.org*. • E-mail addresses. For example: Contact customer support via e-mail at *vp_support@synopsys.com*. • Manual names. For example: The preface of the *Analysis Manual* describes:

`courier`	Is used for:
	• Code text. For example:
	`list_library_configurations myConfig`
	In this example, `myConfig` is used.
	• System messages. For example:
	`JVM not found.`
	• Text you must type literally. For example:
	At the prompt, type `go`.
	• Names (of environment variables, commands, utilities, prompts, paths, macros, and so on). For example:
	The `build-options` command sets build parameters.

`courier italic`	Indicates variables. For example: *`scope`* specifies a module, a channel, or a refined port.
bold	Serves to draw your attention to the text in question. For example: `coreId = cwrSAGetCoreId("mycore");`

`[ ]`	Square brackets enclose optional items. For example:
	`clean [-pch]`
	If you must type a square bracket as part of the syntax, it is enclosed in single quotes. For example:
	`'['--use-vector']'`

`{ }`	Braces enclose a list from which you must choose one or more items. For example:
	`add {signalPattern | portPattern} ID`
	If you must type a brace as part of the syntax, it is enclosed in single quotes. For example:
	`DECLARE '{' Item1 Item1 '}'`

`|`	A vertical bar separates items in a list of choices. For example: `autoflush {on | off}`
>	A right angle bracket separates menu commands. For example: The *Project* > *Update System Library* menu command is available.

`...`	A horizontal ellipsis in syntax indicates that the preceding expression may have zero, one, or more occurrences. For example:
	`build-options -option optionArgs ...`
	A horizontal ellipsis in examples and system messages indicates material that has been omitted. For example:
	`::scsh> dtrace add top1.signal_* $t1`
	`::scsh> dtrace add top1.clk_* $t1`
	`...`
	`::scsh> dtrace flush *`

1.3. Terminology

API	Application Programmer's Interface
ASI	Accellera Systems Initiative
CLI	Command-Line Interface
DUT	Design Under Test in SystemC. In the context of Platform Architect-Specman Elite integration, the DUT is a SystemC design and therefore always uses the Synopsys SystemC simulator.
e code	A piece of source code written in the e language.
e file	A text file containing e code.
e design	Consists of a hierarchical set of connected e units. An e unit in Specman Elite is analogous to a module in SystemC. It is the basic building block. You can build hierarchical e units using instances of other e units. You connect e units using e ports.
e language	Is the verification language used by Specman Elite.
e port	Is a data port in e language to transfer data across units.
e unit	Is a design unit in the e language.
ESI	Is the External Simulation Interface defined by Specman Elite to connect Specman Elite to other simulators.
GCC	GNU Compiler Collection
GUI	Graphical User Interface
HDL	Hardware Description Language
HDL proxy module	Is a SystemC module that you define and instantiate in the SystemC design and that the system then replaces at elaboration time with the HDL model you want to instantiate.
IPC	InterProcess Communication
PLI	Programming Language Interface
SN_LIST	Is a C macro used by ESI to define a list of various types of elements.
SN_TYPE	Is a C macro used by ESI to declare various types of arguments to C or C++ functions.
SPM	Simulink Plant Model
SystemC adapter	Is the communication layer between Specman Elite and the SystemC simulator. This communication occurs through the Specman ESI.
SystemC proxy model	Is an HDL model that you define and instantiate in the HDL design and that the system then replaces at elaboration time with the SystemC module you want to instantiate.
TLC	Target Language Compiler
TLM	Transaction-Level Modeling
VDK	Virtualizer Development Kit
VPI	Verilog Procedural Interface
VSI	Virtualizer Systems Interface

1.4. Customer Support

For technical support (regarding license keys, IP downloads, Host ID, Project ID, documentation or general support), contact the Support Center with a description of your question and supplying the debug information, using one of the following methods:

- Go to *https://solvnetplus.synopsys.com* and sign-in with your Synopsys SolvNetPlus credentials. Select *Cases* from the menu bar, and select *Create a New Case*. Provide the requested information, including:
  - **Product L1:** Virtual Prototyping.
  - **Product L2:** Select the product type that closest matches yours.
  - **Case Type**: Select the case type from the drop-down menu.
  - **Case Severity:** Select the case severity from the drop-down menu.
  - **Subject:** Provide a brief summary of the issue or list the error message you have encountered.
  - **Description:** For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood.

  After creating the case, attach the debug files you have created, if any.
- Or, send an e-mail message to *vp_support@synopsys.com*. (your email will be queued and then, on a first-come, first-served basis, manually routed to the correct support engineer):
  - Include the Product name, Sub Product name, and Tool Version number in your email; so that it can be routed correctly.
  - For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood.
  - Attach any debug files you created in the previous step.
- Or, telephone your local support center:
  - North America:

    Call 1-800-245-8005 from 7:00 AM to 5:30 PM Pacific time, Monday through Friday.
  - All other countries:

    https://www.synopsys.com/support/global-support-centers.html

< > < >

1.5. Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

< > < >

2. Chapter 1 Introduction

Virtualizer and Platform Architect provides an integration path to several third-party simulators. Based on the type of the simulator and the simulation paradigm, different co-simulation mechanisms are necessary that are described in this manual.

The following are the available integrations:

1. Co-simulation with Synopsys SaberRD, Mathworks Simulink, and Vector CANoe.

   For the detailed information, see the *Multi-Node VDKs* documentation.
2. Co-simulation with Synopsys VCS, Cadence Incisive/Xcelium and Mentor Modelsim/Questa.

   These external simulators interface with Virtualizer/Platform Architect using the HDL co-simulation interface. For Cadence Incisive/Xcelium and Mentor Modelsim/Questa, Verilog and VHDL are supported. For Synopsys VCS, Verilog is supported. For

more information, see "HDL CoSimulation" on page 15.

3. Chapter 2
HDL CoSimulation

For the list of supported HDL simulator and corresponding versions, see "Third-Party Tools" of the *Synopsys Virtual Prototyping Product Installation Guide*.

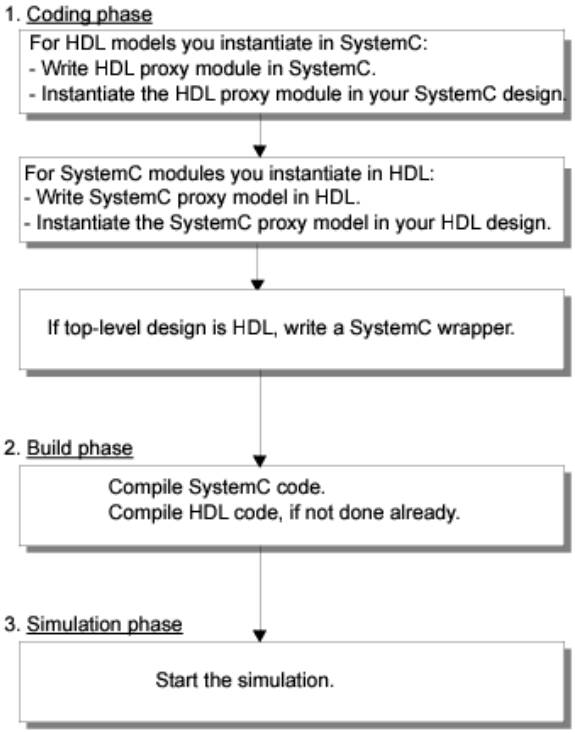
This chapter describes:

- Design Flow
- Rules for SystemC-HDL Co-Simulation
- Setting up the Environment for HDL Simulators
- Mapping Ports
- Instantiating HDL Models in a SystemC Design
- Creating an HDL Proxy Module Automatically
- Instantiating SystemC Models in an HDL Design
- Coding for Both SystemC-Only and Mixed-Mode HDL Simulation
- Creating a Wrapper for a Design Whose Top Level is an HDL Model
- Creating SystemC Proxy Modules from SystemC Designs
- SystemC-HDL Co-Simulation API Reference
- Coding Examples
- Building and Running a Simulation
- Debugging a SystemC-HDL Co-Simulation
- Exporting Structural HDL
- SystemC Shell Commands
- Profiling a SystemC-HDL Co-Simulation
- Limitations and Known Problems
- Performance Optimization with VCS Co-Simulation
- Inout Network Resolution Mechanism
- Example VDK in Virtualizer Studio

3.1. 2.1 Design Flow

To perform SystemC-HDL co-simulation, execute the steps listed in the following figure.

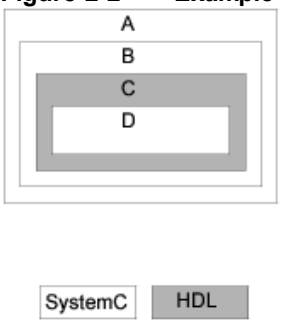
Figure 2-1 The Coding Phase



The following illustrates an example of a design hierarchy you want to simulate:

- `A`, `B`, and `D` are SystemC modules.
- `C` is an HDL model.

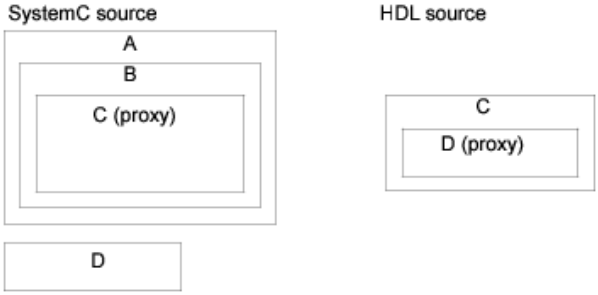
Figure 2-2 Example of a Design Hierarchy



For the design hierarchy illustrated above, the coding phase will include:

- A SystemC module for `A`.
- A SystemC module for `B`.
- An HDL model for `C`.
- A SystemC module for `D`.
- For the HDL model, `C`, you must create a proxy module in SystemC that you instantiate in the SystemC module `B`.
- For the SystemC module, `D`, you must create a proxy model in HDL that you instantiate in the HDL model `C`, as illustrated in the following figure.

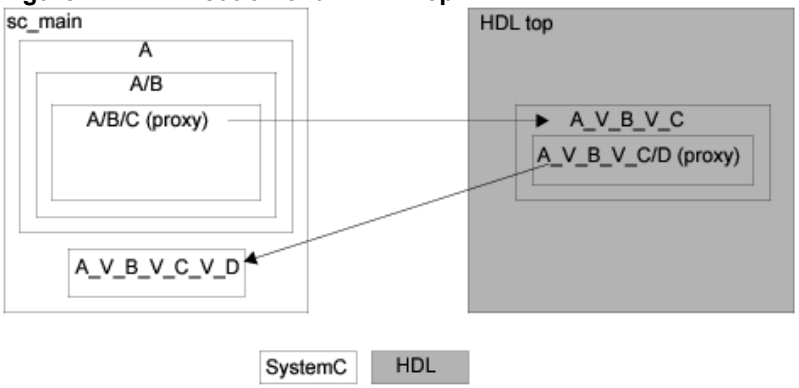
Figure 2-3 Creating a Proxy Model in HDL



When performing co-simulation on this design, the design is loaded as follows in both simulators:

- `sc_main` contains the top-level design (`A` and its hierarchy) and all SystemC modules that are embedded in HDL (`D`). These embedded modules are named by their full hierarchical name, using `_V_` as delimiter.
- An HDL top is automatically created containing all HDL models embedded in SystemC (`C`), as illustrated in the following figure.

Figure 2-4 Creation of an HDL Top

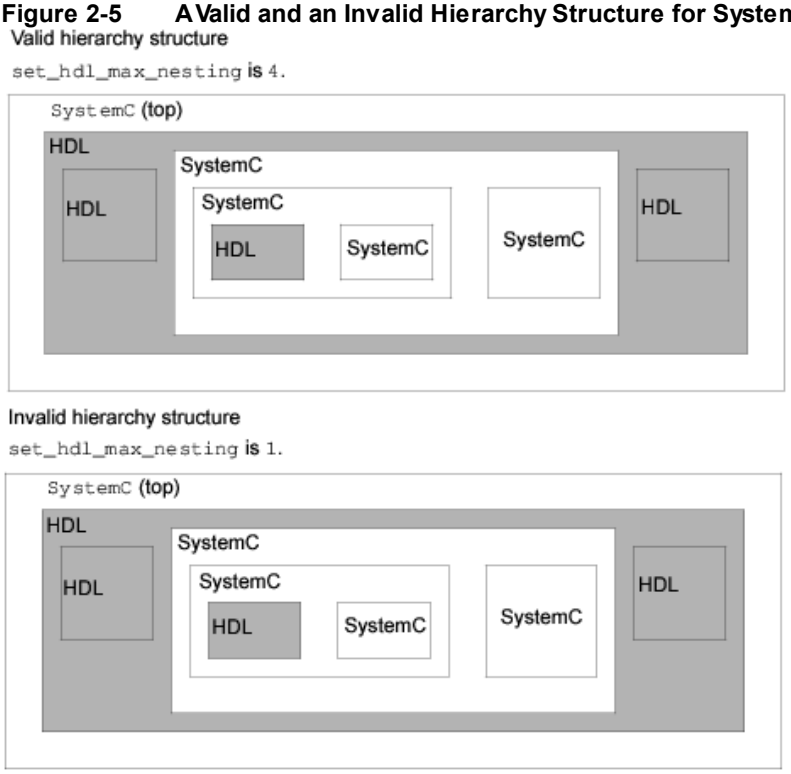


3.2. 2.2 Rules for SystemC-HDL Co-Simulation

You can perform SystemC-HDL co-simulation only on designs to which the following rules apply:

- All HDL models in a SystemC-HDL design are simulated with `one` HDL simulator you select. For more information, see "Selecting the HDL Simulator" on page 81. If your HDL models contain both Verilog and VHDL, you must use one HDL simulator that supports both.
- The top-level of a design hierarchy is SystemC.
- The entry point for SystemC-HDL co-simulation is SystemC Shell, not an HDL simulator.
- The level below the SystemC top level can have recursive mixed language nesting, that is, SystemC can contain HDL, which can contain SystemC, and so on. However, you must specify the maximum number of language switches allowed before loading the simulator. For more information, see "Specifying the Number of Language Switches" on page 80. The following figure illustrates a valid and an invalid hierarchy structure for SystemC-HDL co-simulation.

Figure 2-5 A Valid and an Invalid Hierarchy Structure for SystemC-HDL Co-Simulation



3.3. 2.3 Setting up the Environment for HDL Simulators

In order to use the HDL Simulators for co-simulation, SystemC Shell needs to know its installation path. It uses the following environment variables for this:

- `vcs: VCS_INSTALLED_AT`

  `VCS_INSTALLED_AT` has to be set to the same value as `VCS_HOME`. Internally, co-simulation flow will set `VCS_HOME` so that the VCS tools can use it.

- `vcsi:_INSTALLED_AT`

  `VCSI_INSTALLED_AT` has to be set to the same value as `VCS_HOME`. Internally, co-simulation flow will set `VCS_HOME` so that the VCS tools can use it.

- `nscim: NC_INSTALLED_AT`

  `NC_INSTALLED_AT` needs to be set to the platform specific top-level directory of the Cadence Incisive/Xcelium NC-Sim installation. For example,

  `/global/apps/ius_8.20/linux`

  Internally, the co-simulation flow will setup the environment for Incisive/Xcelium, so that the correct tools are used.

- `mti: MTI_INSTALLED_AT`

  `MTI_INSTALLED_AT` has to be set to the top-level Mentor Graphics ModelSim/Questa installation. For example,

  `/global/apps/mti_6.2f/modeltech`

  Internally, the co-simulation flow uses this variable to setup `LD_LIBRARY_PATH` and `PATH` variables for using the MTI tools.

A convenient way to set these environment variables is to modify the HDL setup script in `$SNPS_VP_HOME/setup`. For example, for C-Shell users, `vcs.csh` file allows you to set the required variables, as shown below:

```
# To activate Synopsys VCS co-simulation support, uncomment the next two
# lines and replace the ... with the pathname to the directory where VCS is
# installed.
#setenv VCS_INSTALLED_AT ...
#setenv VCSI_INSTALLED_AT ...
```

There are setup scripts for Bash and for C-Shell users for each of the supported simulators (`vcs`, `ncsim`, `mti`). These scripts are included by the general setup script of Platform Architect/Virtualizer. By using these setup scripts, you can ensure that all users have the required environment variables set once they source the top-level setup script.

3.4. 2.4 Mapping Ports

When mapping ports across simulator boundaries, the following three issues are important:

1. Which port in the proxy connects to which port in the actual HDL module?
2. How are the port types of the ports mapped?
3. How are generic values in VHDL and module parameters in Verilog mapped?

This section describes:

- Connecting the Ports
- Mapping Signal Types
- Mapping Generic Values (VHDL) and Module Parameters (Verilog)

2.4.1 Connecting the Ports

Ports between SystemC proxy models and HDL models are mapped in one of the following ways:

1. By position. This is the default mode.
2. By name.

You override the default mapping method by defining a public member function in the HDL proxy module, as described in "Mapping Ports" on page 26.

2.4.2 Mapping Signal Types

You can use the following signal types for ports that cross the co-simulation boundary. The table lists the valid SystemC signal types and their corresponding HDL types in VHDL and Verilog. Any other signal types used at the co-simulation boundary results in an error.

Table 2-1 Valid Signal Types

Type Description	sc_in, sc_out, sc_inout Ports in SystemC	VHDL	Verilog
Boolean type; `TRUE` or `FALSE`	`bool`	`std_logic`	`scalar`
2-state logic; `0` or `1`	`sc_bit`	`bit`	`scalar`
4-state logic; Value mapping to smaller type defaults to `0` or `FALSE`	`sc_logic`	`std_logic`	`scalar`
2-state bit vector;	`sc_bv<W>`	`bit_vector` (`W-1` downto `0`), or Unconstrained formal	`wire[W-1:0]`
4-state logic vector	`sc_lv<W>`	`std_logic_vector` (`W-1` downto `0`)	`wire[W-1:0]`
Signed integer for width `W` <= 64 bits	`sc_int<W>`	`std_logic_vector` (`W-1` downto `0`)	`wire[W-1:0]`
Unsigned integer for width `W` <= 64 bits	`sc_uint<W>`	`std_logic_vector` (`W-1` downto `0`)	`wire[W-1:0]`
Signed integer for width `W` > 64 bits	`sc_bigint<W>`	`std_logic_vector` (`W-1` downto `0`)	`wire[W-1:0]`
Unsigned integer for width `W` > 64 bits	`sc_biguint<W>`	`std_logic_vector` (`W-1` downto `0`)	`wire[W-1:0]`
8-bit signed or unsigned integer	`char`, `unsigned char`	`std_logic_vector` (`7` downto `0`)	`wire[7:0]`
16-bit signed or unsigned integer	`short`, `unsigned short`	`std_logic_vector` (`15` downto `0`)	`wire[15:0]`
32-bit signed integer	`int`, `long`	`std_logic_vector` (`31` downto `0`)	`wire[31:0]`
32-bit unsigned integer	`unsigned`, `unsigned long`	`std_logic_vector` (`31` downto `0`)	`wire[31:0]`
64-bit signed or unsigned integer	`long long`, `unsigned long long`	`std_logic_vector` (`63` downto `0`)	`wire[63:0]`
Floating point	`float`, `double`	`real`	`wire[63:0]`

2.4.3 Mapping Generic Values (VHDL) and Module Parameters (Verilog)

The SystemC-HDL co-simulation environment provides support for passing generic values in VHDL and module parameters in Verilog across the co-simulation interface.

The three data types supported for generic values and module parameters are `int`, `double`, and `char*`.

`Char*` supports the following escape sequences `\a`, `\b`, `\f`, `\n`, `\r`, `\t`, `\v`, and `\\`.

Note `\"` is not supported.

The way you set the HDL generic values or module parameters in the SystemC code depends on whether or not you use them to specify a port size.

2.4.3.1 When Used to Specify Port Size

Setting the generic values or module parameters related to port size is a two-step process:

1. Templatize the `sc_hdl_module` on port size.
2. Call `ncwr_set_hdl_param()` in the constructor of `SC_HDL_MODULE` to set the default value of the parameter. For more information on `ncwr_set_hdl_param()`, see "When Not Used to Specify Port Size" on page 21.

2.4.3.2 When Not Used to Specify Port Size

You set generic values or module parameters that are not used to specify port size by calling any of the following public member functions of `sc_hdl_module`.

```
void ncwr_set_hdl_param(const char* param_name, int val);
void ncwr_set_hdl_param(const char* param_name, double val);
void ncwr_set_hdl_param(const char* param_name, const char* val);
```

where *`param_name`* must be the same generic value or module parameter name used in the HDL code.

You can only call these functions in the constructor of the `hdl_module`.

Note Only name mapping is supported. Positional mapping is not supported. Also, multiple calls to set the value of the same generic value or module parameter in HDL is not allowed.

2.4.3.3 Example of Using a Verilog Module Parameter to Define the Port Size

The following example specifies a module parameter to set the port size (`WIDTH`). The `SC_HDL_MODULE` is templatized with `WIDTH`, whose value is set in the constructor by `ncwr_set_hdl_param ()`.

```
//Corresponding SystemC Proxy Module file
  template <int W1=2>
  SC_HDL_MODULE(mxor) {
    sc_in<sc_lv<W1> > dut_inp1;
    sc_in<sc_lv<W1> > dut_inp2;
    sc_out<sc_lv<W1> > dut_out;

    std::string mylib;

    std::string mName;

    bool isPositionalPortMapping() { return false; }
    std::string libraryName() { return mylib; }
    virtual void getVerilogSourceFiles(std::vector<std::string>& file_list)
                                        { file_list.push_back("mxor.v"); }
    std::string moduleName() { return mName; }

    mxor(char *name)
      :sc_hdl_module(mod_name),
      mylib("lib1"),mName("mxor"),dut_inp1("I1"),
      dut_inp2("I2"),dut_out("out")   {
      ncwr_set_hdl_param("WIDTH", W1);
    }
  };


//Verilog file - mxor.v

  module mxor (I1, I2, O);
    parameter WIDTH = 2;
    input [WIDTH - 1:0] I1;
    input [WIDTH - 1:0] I2;
    output [WIDTH - 1:0] O;
  endmodule
```

The proxy module is templatized with the C++ parameter `W1`.

`W1` defines the port size in the proxy module.

`W1` is the name of the template parameter in the proxy module.

`WIDTH` is the name of the module parameter in Verilog.

`WIDTH` defines the port size.

2.4.3.4 Example of Using a Verilog Module Parameter that Is Not Used to Specify the Port Size

The following example sets a parameter `FLAG`. `FLAG` is set in the constructor of `SC_HDL_MODULE`. In the Verilog code, `FLAG` is not used to define the port width.

```
//Corresponding SystemC Proxy Module file
  SC_HDL_MODULE(mxor) {
    sc_in<sc_lv > dut_inp1;
    sc_in<sc_lv > dut_inp2;
    sc_out<sc_lv > dut_out;

    std::string mylib;
    std::string mName;

    std::string libraryName() { return mylib; }
    virtual void getVerilogSourceFiles(std::vector<std::string>& file_list)
                                        { file_list.push_back("mxor.v"); }

    std::string moduleName() { return mName; }

    mxor(char *name, int flag = 1)
      :sc_hdl_module(mod_name),
      mylib("lib1"),mName("mxor"),dut_inp1("I1"),
      dut_inp2("I2"),dut_out("out")  {
      ncwr_set_hdl_param("FLAG", flag);
      }
    };


//Verilog file - mxor.v
  module mxor (I1, I2, O);
    parameter FLAG = 1;
    input I1;
    input I2;
    output O;
    assign O = I1 ^ I2 ^ FLAG;
  endmodule
```

`flag` is the C++ variable used to pass a value to the Verilog module parameter.

Name of the Verilog module parameter.

Note	User-defined struct types and port arrays from SystemC to VHDL and Verilog cannot be passed across the co-simulation boundaries.

< > < >

3.5. 2.5 Instantiating HDL Models in a SystemC Design

When you want to instantiate an HDL model in a SystemC design, you must first create an HDL proxy module for the model. An HDL proxy module is a SystemC module that you define and instantiate in the SystemC design and that the system then replaces at elaboration time with the HDL model you want to instantiate.

To define the SystemC module, use the `sc_hdl_module` class and provide the following information:

- HDL module name
- Library name
- Port mapping
- HDL source files
- Component list package name, if the HDL model is in VHDL
- Architecture name, if multiple architectures are defined for the same VHDL entity
- Optionally, the HDL language for which the HDL proxy is created

Using this information, SystemC Shell calls the HDL compiler, if the modules are not compiled yet, generates an HDL top level, and invokes the HDL simulator.

This section describes:

- Defining the SystemC Module
- Specifying the HDL Module Name
- Specifying the Library Name
- Mapping Ports
- Specifying the HDL Source Files
- Specifying the Component List Package Name

- Specifying the Architecture
- Specifying the HDL Language
- Checking the Source Type

2.5.1 Defining the SystemC Module

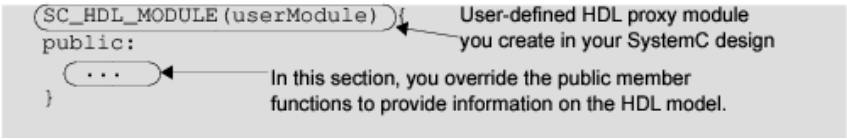
You define the SystemC module by deriving its class from the following class:

```
class sc_hdl_module : public sc_foreign_module {
  public:
    typedef enum {
      cwr_Not_specify = 0,
      cwr_verilog = 1,
      cwr_vhdl = 2
  } cwr_hdlLangType;

  public:
    void ncwr_set_hdl_param(const char* param_name, int val);

    void ncwr_set_hdl_param(const char* param_name, double val);

    void ncwr_set_hdl_param(const char* param_name, const char* val);

    virtual bool isPositionalPortMapping() { return true; }

    virtual void getVerilogSourceFiles(std::vector<std::string>& file_list);

    virtual void getVhdlSourceFiles(std::vector<std::string>& file_list);

    virtual std::string moduleName() = 0;

    virtual std::string libraryName() = 0;

    virtual std::string vhdlComponentListPackageName() { return NULL; }

    virtual std::string vhdlArchitectureName() { return std::string(); }

    virtual cwr_hdlLangType hdl_language_type() { return cwr_Not_specify; }

}
```

In the same way you use the `SC_MODULE` macro in SystemC, you can use the `SC_HDL_MODULE` macro which is defined as follows:

```
#define SC_HDL_MODULE(user_module_name) \
   class user_module_name : public sc_hdl_module
```

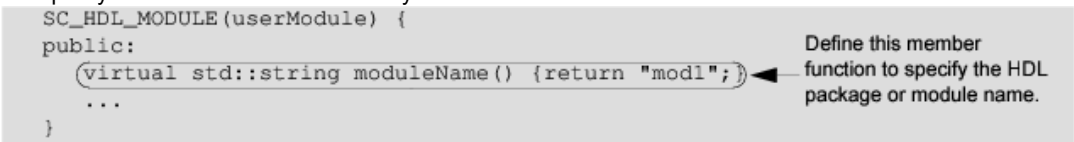
Hence, you define an HDL proxy module as follows:



2.5.2 Specifying the HDL Module Name

At elaboration time, the simulator must know the HDL module you want to instantiate.

You specify the name of the HDL module you want to instantiate as follows:



If the compiled model you want to instantiate is a mixed VHDL/Verilog model, the HDL simulator that supports mixed languages will recognize the mixed mode.

2.5.3 Specifying the Library Name

At elaboration time, the simulator must know the name of the HDL logical library, containing the compiled HDL model you want to instantiate. If your model is not compiled yet, the library name is also required because the compiler stores the compiled model in the specified library name.

You specify the name of the library as follows:

```
SC_HDL_MODULE(userModule) {
public:
  virtual std::string libraryName() {return "work";}
  ...
}
```

Define this member function to specify the logical library name.

2.5.4 Mapping Ports

By default, port mapping occurs by position; that is, the ports listed in the HDL proxy module are matched with the ports listed in the HDL module you want to instantiate in the order of their listing.

Alternatively, you can map ports on the basis of their names. In this case, the port names must be defined in the HDL proxy module.

Assume the HDL model with ports `A`, `B`, and `F`. The ports need to be mapped to a SystemC module with ports `X`, `Y`, and `Z`. This can either be done by position or by name.

2.5.4.1 By Position

This is the default mapping method. In the HDL model, you must define the ports in the exact same order as in the SystemC module. The following illustrates an example:

SystemC module

```
SC_HDL_MODULE(MyHdlModule)
{

sc_in<bool>          X;
sc_out<sc_uint<24>   Y;
sc_in<int>           Z;
...
};
```

HDL model

```
entity HdlModule
port{
   A : in std logic;
   B : out std_logic(23 downto 0);
   F : in std_logic(31 downto 0);
};
```

2.5.4.2 By Name

You must override the default mapping method (which is by position) and include the port mapping in the constructor. The order in which the ports are defined in the HDL model and the SystemC module is irrelevant.

You specify port mapping by name as follows:

```
SC_HDL_MODULE(userModule) {
  sc_in<int> X;
  sc_in<bool> Y;
  sc_out<int> Z;
  ...
public:
  virtual bool isPositionalPortMapping() {return false;}
  ...

SC_CTOR(userModule) :   X("A"), Y("B"), Z("F"){...}
}
```

Port declaration

Override default if you want port mapping by name.

Port mapping

C++ identifier name

SystemC object name which must map to the HDL name

2.5.5 Specifying the HDL Source Files

If the HDL models have not been pre-compiled when you perform co-simulation, you must include the source code files for the HDL models. The default behavior assumes that the HDL model is already compiled and stored in a library; hence, no source code needs to be provided and you do not have to include the methods described below.

You specify the HDL source code files to include as follows:

```
SC_HDL_MODULE(userModule) {

public:
virtual void getVhdlSourceFiles(std::vector<std::string>& file_list)
       { file_list.push_back("mod_p.vhd"); }

virtual void getVerilogSourceFiles(std::vector<std::string>& file_list)
       { file_list.push_back("dff_p.vl");  }

...

   SC_CTOR(userModule) :
   mylib("worklib"),mName("dff"),clk("clk"),
      set("set"), D("D"), Q("Q") {
   }
}
```

Override this member function to specify a VHDL source list.

Override this member function to specify a Verilog source list.

You can specify both the VHDL and Verilog source files. However, the following rules apply:

- If the selected HDL simulator allows mixed language simulation, then both member functions can return nonempty lists.
- If the selected HDL simulator only allows VHDL simulation, `sc_hdl_module::getVerilogSourceFiles` must return an empty list.
- If the selected HDL simulator only allows Verilog simulation, `sc_hdl_module::getVhdlSourceFiles` must return an empty list.

DEPRECATED: The member functions `verilogSourceFileList()` and `vhdlSourceFileList()` are deprecated but are still available for backward compatibility. They are no longer used to specify the HDL source files because they use `sc_pvector` as a return value; and use of `sc_pvector` is deprecated in SystemC 2.2.

2.5.6 Specifying the Component List Package Name

If you want to co-simulate a SystemC-VHDL design, you must specify a package name where the component declaration of the VHDL design you want to instantiate is defined. The package must be defined in the library whose name you specified using `sc_hdl_module::libraryName` (see "Specifying the Library Name" on page 25).

You specify the name of the package that defines the component declaration as follows:

```
SC_HDL_MODULE(userModule) {
public:
   virtual std::string vhdlComponentListPackageName()
                          {return "componentList";}
   ...
```

Define this member function to specify a valid package name.

This member function must return an empty string if the model is a Verilog module.

If source files are provided for compilation, one of the source files must include the package definition, or the package definition must have been compiled into the library before.

If the HDL proxy module does not have any port declarations, `sc_hdl_module::vhdlComponentListPackageName` is not required and a component declaration is generated by SystemC Shell.

2.5.7 Specifying the Architecture

In VHDL, if you have multiple architectures defined for the same VHDL entity, you must specify the architecture to which you want to bind.

You specify the architecture as follows:

```
SC_HDL_MODULE(userModule) {
public:
   virtual std::string vhdlArchitectureName()
                              { return std::string(); }
   ...
}
```

Define this member function to specify a architecture for the VHDL entity.

If you have only one design architecture, you do not need to implement this member function.

2.5.8 Specifying the HDL Language

Optionally, you can explicitly specify the language for which the HDL proxy is created. If you do not specify this information, SystemC-HDL co-simulation will retrieve this information.

It is highly recommended to specify the language if:

- The design is a mixed Verilog/VHDL design, in which case you specify the boundary language.
- The HDL proxy module has no port declarations; hence, the language cannot be derived from the component list package information (see "Specifying the HDL Language" on page 28).
- You have a precompiled HDL library; hence, the language cannot be derived from the source files (see "Specifying the HDL Source Files" on page 26).

You specify the language as follows:

```
SC_HDL_MODULE(userModule) {
public:

    typedef enum {
      cwr_Not_specify = 0,
      cwr_verilog = 1,
      cwr_vhdl = 2
  } cwr_hdlLangType;
...
virtual cwr_hdlLangType hdl_language_type()
                              { return cwr_Not_specify; }
```

Define this member function to specify for which language the HDL proxy is created.

2.5.9 Checking the Source Type

You can check all SystemC modules to determine whether the module is a native SystemC module or an HDL proxy module using the following member function:

```
class sc_hdl_module : public sc_foreign_module {
public:
   virtual bool isHDLModule() {return true;}
   ...
}
```

This member function returns `TRUE` for HDL proxy modules and `FALSE` for SystemC native modules. You cannot override this function.

< > < >

3.6. 2.6 Creating an HDL Proxy Module Automatically

When simulating a SystemC design that instantiates HDL models, you must create an HDL proxy module in the SystemC design. An HDL proxy module is a SystemC module that you define and instantiate in the SystemC design and that the system then replaces at elaboration time with the HDL model you want to instantiate.

"Instantiating HDL Models in a SystemC Design" on page 23 describes how to manually create an HDL proxy module. Use `hdl_proxy_gen` to automatically generate an HDL proxy module.

This section describes:

- Mapping HDL Ports to HDL Proxy Module Ports
- Using VHDL Generic Values and Verilog Module Parameters
- Limitations

2.6.1 Mapping HDL Ports to HDL Proxy Module Ports

By default, Verilog and VHDL ports are mapped to HDL proxy module ports as described in "Mapping Ports" on page 26. However, you can override the default mappings.

- For all vector ports, that is, Verilog vector ports, VHDL `std_logic_vector` ports, and VHDL `bit_vector` ports, you can override the default mapping and instruct to map them to any of the built-in supported SystemC types, such as `sc_lv`, `sc_bv`, `sc_uint`, or `sc_int`, but not to C/C++ data types like `int` or `long`.

  For example, if you specify that the vector ports need to be mapped to `sc_uint`, then all the Verilog vector ports, VHDL `std_logic_vector` ports, and VHDL `bit_vector` ports will be mapped to SystemC type `sc_uint`.

- For all scalar ports, that is, Verilog scalar ports, VHDL `std_logic` ports, and VHDL bit ports, you can override the default mapping and instruct to map them to any of the built-in supported SystemC types.

  For example, if you specify scalar ports to be mapped to `sc_logic`, then all the VHDL bit ports will be mapped to SystemC type `sc_logic` instead of the default `sc_bit`.

You cannot change the default mapping for VHDL real ports to SystemC floats.

2.6.2 Using VHDL Generic Values and Verilog Module Parameters

`hdl_proxy_gen` supports VHDL generic values and Verilog module parameters. If the parameter or generic is used to specify the size of the interface ports, then the corresponding HDL proxy module is defined as a template based on that parameter or generic value.

If the module parameter or generic value is not used to specify the size of the interface ports, then a constant is declared in the constructor of the HDL proxy module.

For more information, see "Mapping Generic Values (VHDL) and Module Parameters (Verilog)" on page 21.

To invoke hdl_proxy_gen:

At the `scsh` prompt, execute:

```
hdl_proxy_gen [--import-method compiledModule | sourceFile][--hdl-language verilog | vhdl]
              [--simulator vcs | modelsim | incisive] [--sim-ws-file workspaceFile]
              [--library logicalLib] [-m | --module moduleName][--view viewName]
              [--compile-src] [--gen-pkg] [--named] [--map options] [--imap identifier]
              [-o | --output fileName] [--pkg packageName] [-sc-mod moduleName]
              [--scalar-map SystemCDataTypeName] [--src listOfFiles]
              [--top-gen fileName] [--vector-map SystemCDataTypeName]
              [--vhdl listOfFiles] [--vlog listOfFiles] [-v2k]
              [-a | --analyzer options]
```

To obtain an overview of the options you can use, execute:

```
hdl_proxy_gen -h | --help
```

The following explains the options of the `hdl_proxy_gen` command. Each option is described in detail in the subsequent sections.

`--import-method compiledModule | sourceFile`	Sets it to `sourceFile` if the HDL is in source form, or `compiledModule` if the HDL is in compiled form in a library. Default is `sourceFile`, if HDL files are provided as input.
`--hdl-language verilog | vhdl`	Specifies the HDL language. Can be set to `verilog` or `vhdl`. Default option is set to `verilog` if `--vlog` is provided as input or `vhdl` if `--vhdl` is provided as input source file(s).
`--simulator vcs | modelsim | incisive`	Specifies HDL simulator name. Can be set to `vcs`, `modelsim` or `incisive`. The default simulator is `vcs`.
`--sim-ws-file` *`workspaceFile`*	Specifies the workspace file for HDL simulator: • `synopsys_sim.setup` for VCS • `modelsim.ini` for ModelSim • `cds.lib` for Incisive This is needed for `compiledModule` import.
`--library` *`logicalLib`*	Specifies the name of the logical library containing the compiled HDL design.
`-m | --module` *`moduleName`*	Specifies the name of the HDL module for which to generate an HDL proxy module.

`--view viewName`	Specifies the view to be imported. For VHDL, specify the architecture name and for Verilog, specify the module name.
`--compile-src`	Specifies that the source files are compiled by the HDL simulator. Thus, the `getVerilogSourceFiles()` or `getVhdlSourceFiles()` method will be implemented.
`--gen-pkg`	Generates a file with the VHDL component declarations for the HDL proxy module.
`--named`	Specifies that the port mapping is named. The default value is positional.
`--map options`	Maps a port name to a SystemC data type.
`--imap identifier`	Maps a port name to an identifier.
`-o | --output fileName`	Specifies the name of the output file containing the HDL proxy module.
`--pkg packageName`	Specifies the name of the file with the VHDL component declarations for the HDL proxy module.
`--sc-mod moduleName`	Specifies the name of the SystemC model to generate.
`--scalar-map SystemCDataTypeName`	Specifies the mapping for the scalar ports in the HDL module.
`--src listOfFiles`	Specifies that the source file names following this option do not have to be parsed by the HDL parser, but must be compiled by the HDL simulator.
`--top-gen fileName`	Specifies the name of the file in which to instantiate the generated HDL proxy module.
`--vector-map SystemCDataTypeName`	Specifies the mapping for the vector ports in the HDL module.
`--vhdl listOfFiles`	Marks the beginning of arguments for VHDL source files and commands. The end is recognized when another supported option is specified.
`--vlog listOfFiles`	Marks the beginning of arguments for Verilog source files and commands. The end is recognized when another supported option is specified.
`-v2k`	Specifies the Verilog 2001 mode.
`-a | --analyzer options`	Passes comma-separated options on to the Verilog/VHDL analyzer.

2.6.2.1 Specifying the HDL Source Files for the HDL Proxy Module

You must specify the source files of the Verilog and VHDL models for which to create an HDL proxy module:

- Use the `--vlog` option to specify Verilog files.
- Use the `--vhdl` option to specify VHDL files.

Although not necessary, you can also provide the complete source description of the design. For mixed VHDL/Verilog designs, you can use both options together, as described below.

Syntax

The syntax of the options is as follows:

```
--vlog file [file]...
--vhdl file [file]...
```

where *file* is the name of the Verilog or VHDL description. `hdl_proxy_gen` considers all arguments following the option as input files until it recognizes another valid `hdl_proxy_gen` option. You can use both options on the same command line, and you can specify them more than once.

Examples

- The following example generates a Verilog proxy module for `mxor`. The module is described in the Verilog file `mxor.v`.

  ```
  hdl_proxy_gen -m mxor --vlog mxor.v
  ```

- The following example generates a VHDL proxy module for `mxor`. The entity is described in the VHDL file `mxor.vhdl`.

```
hdl_proxy_gen --vhdl mxor.vhdl -m mxor
```

2.6.2.2 Specifying Analyzer and Parser Options

Use the `-a` or `--analyze` option to specify VHDL or Verilog analyzer and parser options. Options (that are not `hdl_proxy_gen` options but analyzer and parser options) are separated by commas (`,`) and must not contain blank spaces.

Syntax

The syntax of the options is as follows:

```
{-a | --analyze},option[,option]...
```

where *`option`* is a Verilog or VHDL analyzer or parser option, such as `-v`.

Examples

The following examples are identical and generate a Verilog proxy module for `mxor`. The module is described in the Verilog file `mxor.v`. The `-v` option followed by the Verilog library, `my_lib.v`, is analyzer/parser specific.

```
hdl_proxy_gen -m mxor --vlog mxor.v -a,-v,my_lib.v

hdl_proxy_gen --vlog mxor.v -a,-v,mylib.v -m mxor

hdl_proxy_gen -m mxor --vlog mxor.v --analyze,-v,mylib.v
```

2.6.2.3 Specifying the Name of the HDL Model

You must provide the name of the HDL model for which to generate an HDL proxy module. Use the `-m` or `--module` option to specify the name.

By default, the name of the generated SystemC module is the HDL model name.

For VHDL modules, you specify the name of the entity which you want to instantiate. Likewise, for Verilog modules, you specify the name of the module.

In one invocation of `hdl_proxy_gen`, you can only generate **one** HDL proxy module. If multiple proxy modules are required, run `hdl_proxy_gen` for each HDL model.

The generated `sc_hdl_module::moduleName()` member function will return the name of that HDL model.

You must provide one or more HDL source files describing that module or entity; otherwise an error appears. For more information, see "Specifying the HDL Source Files for the HDL Proxy Module" on page 32.

Syntax

The syntax of the option is as follows:

```
{-m | --module} moduleName
```

where *`moduleName`* is the name of the VHDL entity or Verilog module for which to generate an HDL proxy module.

Example

The following example generates a Verilog proxy module for `mxor`. The Verilog module is described in the Verilog file `mxor.v`.

```
hdl_proxy_gen -m mxor --vlog mxor.v
```

2.6.2.4 Specifying the SystemC Module Class

Optionally, you can specify the name of the SystemC module class you want `hdl_proxy_gen` to generate. By default, the name of the class is the name of the HDL module, specified using the `-m` option. For more information, see "Specifying the Name of the HDL Model" on page 33.

Use the `--sc_mod` option if you want the name of the generated class to be different from the HDL module.

Syntax

The syntax of the option is as follows:

```
-sc_mod moduleClassName
```

where *`moduleClassName`* is the name of the SystemC module class.

Example

The following example generates a HDL proxy module `my_proxy` for the Verilog module `mxor`. The module is described in the Verilog file `mxor.v`.

```
hdl_proxy_gen -m mxor --vlog mxor.v --sc-mod my_proxy
```

The HDL proxy module is shown below.

```
my_proxy
 template <int WIDTH=2>
 SC_HDL_MODULE(my_proxy) {
        sc_in<sc_lv<WIDTH> > dut_inp1;
        sc_in<sc_lv<WIDTH> > dut_inp2;
        sc_out<sc_lv<WIDTH> > dut_out;

        std::string mylib;
        std::string mName;
        bool isPositionalPortMapping() { return false; }
        std::string libraryName() { return mylib; }
        void getVerilogSourceFiles(std::vector<std::string>& file_list)
                                   { file_list.push_back("mxor.v"); }
        std::string moduleName() { return mName; }


       my_proxy(char* name) : sc_hdl_module(name),
              mylib("lib1"),mName("mxor"),dut_inp1("I1"),
              dut_inp2("I2"),dut_out("out") {
           nccwr_set_param("WIDTH", WIDTH);
        }
 };
```

2.6.2.5 Specifying the File Name of the HDL Proxy Module to Generate

Optionally, you can specify the name of the output file `hdl_proxy_gen` generates. By default, the name of the SystemC file containing the proxy is *moduleName*`_proxy.h`.

Use the `-o` or `--output` option to specify a name for the SystemC file.

Syntax

The syntax of the option is as follows:

```
{-o | --output} proxyFileName
```

where *proxyFileName* is the name of the SystemC file that `hdl_proxy_gen` generates.

Examples

- The following example generates a SystemC file, `mxor_proxy.h`, containing the Verilog proxy module for Verilog module `mxor`. The module is described in the Verilog file `mxor.v`.

  ```
  hdl_proxy_gen -m mxor --vlog mxor.v
  ```

- The following example generates a SystemC file, `my_proxy.h`, for Verilog module `mxor`. The module is described in the Verilog file `mxor.v`.

  ```
  hdl_proxy_gen -o my_proxy.h -m mxor --vlog mxor.v
  ```

2.6.2.6 Specifying the Port Mapping Style

By default, `hdl_proxy_gen` generates an HDL proxy module whose ports are mapped by position to the ports in the HDL model. However, you can generate an HDL proxy module whose ports are mapped by name.

Use the `--named` option to request port mapping by name. In this case, the `sc_hdl_module::isPositionalPortMapping()` member function is defined in the HDL proxy module.

Syntax

The syntax of the option is as follows:

```
--named
```

Example

The following example generates a VHDL proxy module for the Verilog module `mxor`. Port mapping will occur by name. The module is described in the Verilog file `mxor.v`.

```
hdl_proxy_gen --named -m mxor --vlog mxor.v
```

2.6.2.7 Specifying the Name of the Logical Library

You must provide the name of the target library that is used to compile the source files.

Use the `--library` option to specify the name of the target library. The `sc_hdl_module::libraryName()` member function will return the library name.

Syntax

The syntax of the option is as follows:

`--library` *`libraryName`*

where *`libraryName`* is the name of the target library used to compile the source files.

Example

The following example generates a Verilog proxy module for the Verilog module `mxor`. The module is described in the Verilog file `mxor.v`. The source file will be compiled with the `lib1` library.

```
hdl_proxy_gen -m mxor --library lib1 --vlog mxor.v
```

2.6.2.8 Specifying Verilog and VHDL Source Files for Compilation at Simulation Build Time

The `--compile-src` option determines whether the simulator must compile all HDL source files, or whether you provide a precompiled library of the HDL model.

- If not specified, the `getVhdlSourceFiles()` and `getVerilogSourceFiles()` member functions will be implemented such that they always specify an empty list. Hence, the simulator does not compile any HDL source files, and expects you to provide the library with the precompiled HDL model. This is the default behavior.
- If specified, the `getVhdlSourceFiles()` and `getVerilogSourceFiles()` member functions will be implemented such that they return all the HDL source files. Hence, the simulator compiles the HDL source files at compile time.

  In addition, if you provide the precompile library and the HDL model is a large hierarchical design, then it is more efficient for `hdl_proxy_gen` to only parse the file(s) that contain the Verilog module or VHDL entity for which to create the HDL proxy module. For this reason, the options `--vlog` and `--vhdl` have a `--src` option.

Syntax

The syntax of the options is as follows:

`--compile-src {--vlog | --vhdl} [--src` *`file`* `[`*`file`*`]...]`

where *`file`* is the name of Verilog or VHDL source files, depending on the option preceding `--src`.

Note	`--src` must always follow either `--vlog` or `--vhdl`; The list of files will be considered Verilog or VHDL, respectively. Also, the `--src` option is only meaningful if the `--compile-src` option is specified.

Example

Assume three files `a.v`, `b.v`, and `c.vhdl` that describe the complete HDL design for which to create an HDL proxy module. File `a.v` has a module (module name is `a`) for which a Verilog proxy module must be generated. Following are the ways you can use the `--compile-src`, `--src`, `--vlog`, and `--vhdl` options.

- The following example specifies only the source file(s) for the HDL proxy module and precompile all sources separately.

  ```
  hdl_proxy_gen -m a --library lib1 --vlog a.v
  ```

  Because the `-compile-src` option is not specified, you only need to provide the Verilog source file needed by `hdl_proxy_gen`. The generated HDL proxy module will implement `getVerilogSourceFiles()` and `getVhdlSourceFiles()` member functions to specify no source files.
- The following example specifies all the source files and the option `--compile-src` so that the simulator compiles the HDL model at build time. `hdl_proxy_gen` implements the `getVhdlSourceFiles()` and `getVerilogSourceFiles()` member functions to return the list of source files.

  ```
  hdl_proxy_gen -m a --library lib1 --vlog a.v b.v --vhdl c.vhdl --compile-src
  ```

  `hdl_proxy_gen` parses the source files specified by the `--vlog` and `--vhdl` options, and searches for the module/entity `a`.

- The following examples specify the `-compile-src` option, and add the `--src` option to the `--vlog` or `--vhdl` options for source files that you want the HDL simulator to compile, but that are not required to generate the HDL proxy module. The two examples are equivalent.

  ```
  hdl_proxy_gen -m a --library lib1 --vlog a.v --src b.v
  --vhdl --src c.vhdl --compile-src

  hdl_proxy_gen -m a --library lib1 --vlog a.v --vhdl
  --src c.vhdl --compile-src --vlog --src b.v
  ```

  The HDL simulator will compile all the source files but `hdl_proxy_gen` only parses `a.v`. The generated member function `getVhdlSourceFiles()` will return the file name `c.vhdl`. The generated member function `getVerilogSourceFiles()` will return two file names, `a.v` and `b.v`.

2.6.2.9 Specifying the Component Package Name

If you want to generate a VHDL proxy module, you must declare the components for the module in a package. `hdl_proxy_gen` implements a member function `vhdlComponentListPackageName()` to return this package name.

Use the `--pkg` option to specify the name of the package. You can generate the package manually, or specify the `--gen-pkg` option to generate this package in *`packageName`*`_ppkg.vhdl`.

Syntax

The syntax of the options is as follows:

`--pkg` *`packageName`* `[--gen-pkg]`

where *`packageName`* is the name of the VHDL package declaring the components.

Examples

- The following example creates the package named `a_comp_pkg_ppkg.vhdl`.

  ```
  hdl_proxy_gen -m a --library lib1 --vhdl a.vhdl --pkg a_comp_pkg
  ```

- The following example automatically creates a package named `a_comp_pkg_ppkg.vhdl`.

  ```
  hdl_proxy_gen -m a --library lib1 --vhdl a.vhdl --pkg a_comp_pkg- --gen-pkg
  ```

- The following example generates a file named, `a_pcomp_pkg_ppkg.vhl`, and automatically creates the package named, `a_pcomp_pkg`, whose components correspond to the entity for which you create a VHDL proxy module.

  ```
  hdl_proxy_gen -m a --library lib1 --vhdl a.vhdl --gen-pkg
  ```

2.6.2.10 Changing the Default Port Mappings

You can override the default port mappings described in "Mapping Ports" on page 26.

Use the `--vector-map` option to change the data type mappings of vector ports; Use the `--scalar-map` option to change the data type mappings of scalar ports; Use the `--map` option to force the port type mapping for a specific port.

Syntax

- The syntax of the `--vector-map` and `-scalar-map` options is as follows:

  `{--vector-map | --scalar-map}` *`typeName`*

  where *`typeName`* is a built-in SystemC type name such as `sc_lv`, `sc_bv`, `sc_uint`, or `sc_int`, but not a C/C++ data type like `int` or `long`. For more information, see "Mapping Ports" on page 26.

- The syntax of the `--map` options is as follows:

  `--map` *`portName=scDTypeName`*

  where *`portName`* is the name of the port and *`scDTypeName`* is the SystemC data type that is assigned to the port name.

  This option can be specified multiple times on the command line, once for each port name whose SystemC data type you want to override.

  This option has the highest priority; it overrides the default SystemC data type as well as the SystemC data type set by `--vector_map` or `--scalar_map`.

Examples

- Assume the VHDL module has two vector ports of type `std_logic_vector` (`3` down to `0`) and `bit_vector` (`2` down to `1`). The following example generates a VHDL proxy module with corresponding ports of data types `sc_uint<4>` and `sc_uint<2>`, respectively.

```
hdl_proxy_gen -m a --vlog a.vhdl --vector-map sc_uint
```

- Assume the Verilog module has a scalar port (which by default will be mapped to `sc_logic`). The following example generates a Verilog proxy module with corresponding ports of type `sc_bit`.

  ```
  hdl_proxy_gen -m a --vlog a.vhdl --scalar-map sc_bit
  ```

- The following example assigns the `sc_int` SystemC data type to the `my_port` port:

  ```
  --map my_port=sc_int
  ```

2.6.2.11 Generating a SystemC Instance for the HDL Top Module

If the HDL model for which you generate an HDL proxy module is the design top module, then you must create a simple SystemC wrapper to instantiate the design in the SystemC-HDL co-simulation environment.

Use the `--top-gen` option to generate this wrapper; that is, a file in which the generated HDL proxy module is instantiated.

Note	You can only use this option to create a wrapper if the module does not have interface ports. A warning appears and no wrapper is created if the module has interface ports.

Syntax

The syntax of the option is as follows:

`--top-gen` *`fileName`*

where *`fileName`* is the name of the wrapper to create for the HDL proxy module.

Note	Alternatively, you can use `top_sc_gen` to create the wrapper, as described in "Creating a Wrapper for a Design Whose Top Level is an HDL Model" on page 46.

Example

The following example generates a SystemC module named `scTop` corresponding to HDL module `hdlTop`. The example assumes that the top-level HDL module does not have any interface ports.

```
hdl_proxy_gen -o scTop_proxy.h -m hdlTop --vlog a.v --sc-mod scTop --top-gen my_main.cpp
```

Also, the following file, named `my_main.cpp`, is generated.

my_main.cpp

```
#include "systemc.h"

#include "scTop_proxy.h"

int sc_main(int argc, char** argv) {
      scTop sc_inst("sc_inst");
      sc_start();
      return 0;
}
```

2.6.3 Limitations

- In the case of a Verilog proxy module, an error appears if the ports do not have names and name mapping is requested. You can map the ports using positional mapping. For example:

  ```
  module(out, {in[1], in[2], in[0]});
  output out;
  input [2:0]in;
  endmodule
  ```

  In the example, you cannot specify named mapping (using the option `--named`) because of the unnamed ports `{in[1], in[2], in[0]}`.

- In the case of a Verilog proxy module, an error appears if a `NULL` port is specified in the module and name positional mapping is requested. You can map the ports using named mapping. For example:

  ```
  module(out, ,in);
  output out;
  input [2:0]in;
  endmodule
  ```

  In the example, you must specify the `--named` option; otherwise an error appears.

- Vector ports data types (both for Verilog and VHDL) can be mapped to SystemC built-in data types only, such as `sc_lv`, `sc_bv`, `sc_uint`, or `sc_int`, but not to C/C++ data types like `int` or `long`.

- Because the SystemC-HDL co-simulation interface supports only `std_logic`, `std_logic_vector`, `bit`, `bit_vector`, and real VHDL data types for the interface design unit, `hdl_proxy_gen` reports an error if it encounters a data type other than these in the interface port specifications for that design unit.
- `hdl_proxy_gen` supports VHDL generic parameters of type integer, string, and real only. No other data types for the interface generics are supported.
- Unconstrained `std_logic_vector` or `bit_vector` ports in VHDL proxy modules are not supported.

< > < >

3.7. 2.7 Instantiating SystemC Models in an HDL Design

When you want to instantiate a SystemC module in an HDL design, you must first create a SystemC proxy model for the module. A SystemC proxy model is an HDL model that you define and instantiate in the HDL design and that the system then replaces at elaboration time with the SystemC module you want to instantiate.

To define the HDL module, provide the following information:

- SystemC source file names
- Module name
- Header file name
- Port mapping
- Compile-time options

For VHDL, you provide this information using `string` constants; For Verilog, you use `specparam` statements.

In addition, you must reference the SystemC module in your HDL model. For VHDL, you use the foreign language interface; For Verilog, you use the initial thread.

Once the HDL model is defined, SystemC Shell compiles the SystemC module and loads it with the elaborated SystemC design.

This section describes:

- Specifying SystemC Source File Names
- Specifying Module Name
- Specifying Header File Name
- Mapping Ports
- Referencing SystemC Modules
- Specifying Compile-Time Options

2.7.1 Specifying SystemC Source File Names

Use `CWR_SYSC_FILES_LIST` to specify the list of SystemC files you want the system to compile. You can specify the list in one of the following ways:

- Specify the file names with the suffixes as recognized by `load` in SystemC Shell. For more information about this command, see "load" in the *SystemC Shell Manual*.
- Specify the name of an ASCII `.fof` file, listing the SystemC files.

This string constant must take the same format as recognized by the SystemC Shell command line. For more information, see the *SystemC Shell Manual*.

2.7.1.1 In VHDL

```
constant CWR_SYSC_FILES_LIST : string := "file.cpp [file.cpp]...";

constant CWR_SYSC_FILES_LIST : string := "file.fof";
```

2.7.1.2 In Verilog

```
specparam CWR_SYSC_FILES_LIST = "file.cpp [file.cpp]...";

specparam CWR_SYSC_FILES_LIST = "file.fof";
```

where:

file.cpp	Specifies the name of the SystemC file you want to compile.

file.fof	Specifies the name of the ASCI file listing the SystemC files you want to compile.

2.7.1.3 VHDL Examples

- The following VHDL statement specifies a list of files for compilation.

```
constant CWR_SYSC_FILES_LIST : string :=
                            "stage1.cpp stage2.cpp stage11.cpp stage12.cpp";
```

- The following VHDL command specifies a file listing a list of files for compilation.

```
constant CWR_SYSC_FILES_LIST : string := "mySourceFiles.fof";
```

2.7.1.4 Verilog Examples

- The following Verilog statement specifies a list of files for compilation.

```
specparam CWR_SYSC_FILES_LIST = "stage1.cpp stage2.cpp stage11.cpp
                                                    stage12.cpp";
```

- The following Verilog command specifies a file listing a list of files for compilation.

```
specparam CWR_SYSC_FILES_LIST = "mySourceFiles.fof";
```

2.7.2 Specifying Module Name

Use `CWR_SYSC_MODULE_NAME` to specify the SystemC module name.

2.7.2.1 In VHDL

```
constant CWR_SYSC_MODULE_NAME : string := "moduleName";
```

2.7.2.2 In Verilog

```
specparam CWR_SYSC_MODULE_NAME = "moduleName";
```

where *moduleName* is the name of the SystemC module you want to compile.

2.7.2.3 VHDL Example

The following VHDL statement specifies the SystemC module name.

```
constant CWR_SYSC_MODULE_NAME: string := "stage2";
```

2.7.2.4 Verilog Example

The following Verilog statement specifies the SystemC module name.

```
specparam CWR_SYSC_MODULE_NAME = "stage2";
```

2.7.3 Specifying Header File Name

Use `CWR_SYSC_MODULE_HDR_FILE_NAME` to specify the header file name in which the SystemC module is declared.

2.7.3.1 In VHDL

```
constant CWR_SYSC_MODULE_HDR_FILE_NAME : string := "headerFile";
```

2.7.3.2 In Verilog

```
specparam CWR_SYSC_MODULE_HDR_FILE_NAME = "headerFile";
```

2.7.3.3 VHDL Example

The following VHDL statement specifies the name of the SystemC header file.

```
constant CWR_SYSC_MODULE_HDR_FILE_NAME : string :=
                                        "/user4/home/test/cosim/stage2.h";
```

2.7.3.4 Verilog Example

The following Verilog statement specifies the name of the SystemC header file.

```
specparam CWR_SYSC_MODULE_HDR_FILE_NAME = "/user4/home/test/cosim/stage2.h";
```

2.7.4 Mapping Ports

Use `CWR_SYSC_NAMED_MAPPING_REQD` to specify that you want ports mapped by name. By default, ports are mapped by position.

2.7.4.1 In VHDL

```
constant CWR_SYSC_NAMED_MAPPING_REQD : boolean := (true | false);
```

2.7.4.2 In Verilog

```
specparam CWR_SYSC_NAMED_MAPPING_REQD = (0 | 1);
```

2.7.4.3 VHDL Example

The following VHDL statement specifies to map ports by name.

```
constant CWR_SYSC_NAMED_MAPPING_REQD : boolean := true;
```

2.7.4.4 Verilog Example

The following Verilog statement specifies to map ports by name.

```
specparam CWR_SYSC_NAMED_MAPPING_REQD = 1;
```

2.7.5 Referencing SystemC Modules

To reference SystemC modules, you must do the following:

- For VHDL:
  - You must use the foreign language interface to reference the SystemC module. This will be different for different HDL simulator vendors. For more information, see the vendor's documentation.
  - You must specify the port type and direction match with the SystemC module port type and direction, as shown in the following example.

    ```
    entity pipeline is
      port (
        clk : in std_logic;
        po  : out std_logic_vector( 2 downto 0);
        pi  : in std_logic_vector( 2 downto 0 );
        pio : inout std_logic_vector( 2 downto 0 );
      );
    end;
    architecture behave of pipeline is
    attribute foreign of behave : architecture is :
    "cwrCosim:cwr_cosim_init";
    begin
    end behave;
    ```
- For Verilog:
  - You must use the initial thread to reference the SystemC module.
  - You must specify the port type and direction match with the SystemC module port type and direction.
  - For `out` ports, you need to declare a `reg`.
  - For `inout` port, you need to declare a dummy driver to prevent the optimizer from optimizing out the connection, as shown in the following example.

    ```
    module pipeline (
       clk,
       po,
       pi,
       pio);

      input  clk;
    ```

```
  output [2:0] po;
  input  [2:0] pi;
  inout  [2:0] pio;

  reg  [2:0] po;
  reg  [2:0] pio_cwr;
  wire [2:0] pio;

  assign pio = pio_cwr

initial
begin
$cwr_cosim_init;
end

endmodule
```

The corresponding SystemC module is:

```
class pipeline : public sc_module {
   ...
Public:
   sc_in<bool> clk;
   sc_out< sc_lv<3> > po;
   sc_in< sc_lv<3> > pi;
   sc_inout< sc_lv<3> > pio;
...
};
```

2.7.6 Specifying Compile-Time Options

You can specify compile-time options including compiler options, linker options, and preprocessor options. In addition, you can limit the number of errors to report and suppress warning display during parsing.

Use `CWR_SYSC_COMPILE_OPTIONS` to specify the compile-time options. These options override the options you specify on the SystemC Shell command line or in a start-up file using `cwr_set_simbld_opts`. For more information about this command, see "cwr_set_simbld_opts" in the *SystemC Shell Manual*.

Note	If the top-level SystemC design (above the HDL hierarchy) is not compiled in optimized mode, the embedded model files will not compile in optimized mode either. Similarly, if the top-level SystemC design (above the HDL hierarchy) is compiled in optimized mode, the embedded model files will also compile in optimized mode. This can significantly slow down compilation time.

2.7.6.1 In VHDL

```
constant CWR_SYSC_COMPILE_OPTIONS : string := "group value[; group value]...";
```

2.7.6.2 In Verilog

```
specparam CWR_SYSC_COMPILE_OPTIONS = "group value[; group value]...";
```

where:

group	Is one of the following: • `compiler` sets compiler options. • `errorLimit` limits the number of error messages during compilation. The default value is `100`. It can be any positive integer. • `linker` sets linker options. • `noWarnings` disables parser warnings. • `preprocessor` sets options for the preprocessor.
value	Is any valid value for the specified *group*.

For information on `compiler`, `linker`, and `preprocessor` options, see your GCC documentation.

2.7.6.3 VHDL Example

The following VHDL statement sets compile-time options.

```
constant CWR_SYSC_COMPILE_OPTIONS: string :=
          "preprocessor -I/user4/test; compiler -g;
          linker -L../libs -lmylib";
```

2.7.6.4 Verilog Example

The following Verilog statement sets compile-time options.

```
specparam CWR_SYSC_COMPILE_OPTIONS
           = " preprocessor -I/user4/test; compiler -g;
           linker -L../libs -lmylib";
```

< > < >

3.8. 2.8 Coding for Both SystemC-Only and Mixed-Mode HDL Simulation

You can code one module for both SystemC-only and mixed-mode simulation.

Depending on the class from which the module is derived, the following occurs:

- If derived from the `sc_hdl_module` class, the module is interpreted as a proxy and the information is passed through proxy member functions (see "Instantiating HDL Models in a SystemC Design" on page 23). Hence, a mixed-mode simulation starts.

  In this case, the SystemC hierarchy in the module is ignored.
- If derived from the `sc_module` class, the module is interpreted as native SystemC. Hence, no mixed-mode simulation occurs.

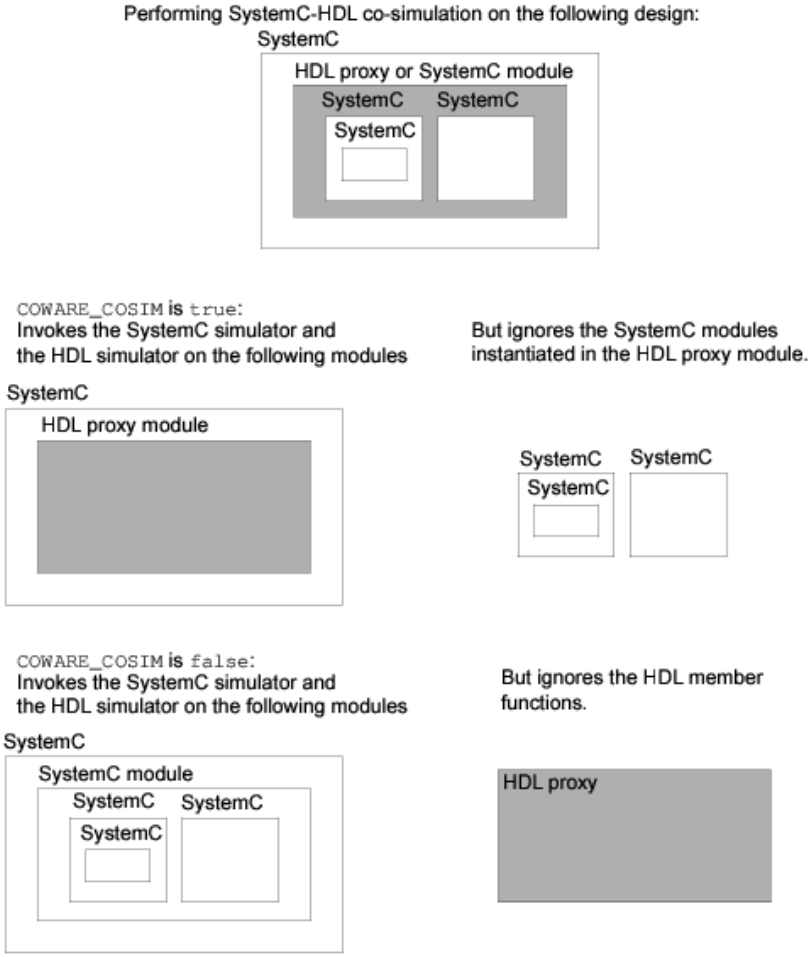
  In this case, the proxy member functions are ignored.

Using a macro, you can decide at compile time whether you want pure SystemC or mixed-mode simulation for that module. The following example illustrates how the macro `COWARE_SIM` controls the type of simulation.

```
#ifdef COWARE_COSIM
                                  Use as HDL proxy class.

#define HDL_PROXY_CLASS_DEF(aName) SC_HDL_MODULE(aName)
#else                             Use as regular SystemC class.

#define HDL_PROXY_CLASS_DEF(aName) SC_MODULE(aName)
#endif

HDL_PROXY_CLASS_DEF(myClass) {
...


...
}
...
```

The following figure illustrates the two possible values of `COWARE_COSIM`.

Figure 2-6 Possible Values of COWARE_COSIM



3.9. 2.9 Creating a Wrapper for a Design Whose Top Level is an HDL Model

The top level of a design for co-simulation must always be in SystemC. If you have a design whose top level is in HDL, you must create a SystemC wrapper and the HDL model must be compiled into a package before performing co-simulation.

If the HDL top level does not have any interface ports, and therefore, does not require port binding, you can use `top_sc_gen` to automatically generate the wrapper. If the HDL top level does have any interface ports, you must use `hdl_proxy_gen` to create the wrapper. For more information, see "Generating a SystemC Instance for the HDL Top Module" on page 38.

To generate a SystemC wrapper:

For VHDL, from the command line, execute:

```
top_sc_gen -c moduleClass -m hdlModule -l hdlLibrary -p vhdlPackage
```

For Verilog, from the command line, execute:

```
top_sc_gen -c moduleClass -m hdlModule -l hdlLibrary
```

where:

moduleClass	Specifies the name of the HDL proxy module that is instantiated from the generated `sc_main`.
hdlModule	Specifies the name of the HDL module that is instantiated at the top level.
hdlLibrary	Specifies the name of the HDL library in which the top-level HDL module is compiled.
vhdlPackage	Specifies the VHDL package name that includes a component declaration for the VHDL module. This argument is irrelevant for Verilog models.

The following illustrates an example of how to generate a SystemC wrapper for an HDL top-level design that does not have any interface ports, and hence, does not require port binding.

```
top_sc_gen -c top -m top_hdl -l hdl_lib -p hdl_comps
```

The generated code for this example appears in `top.cpp` and is shown below. The wrapper is a SystemC wrapper for a VHDL model.

top.cpp

```
#include "systemc.h"

class top : public sc_hdl_module {
public:
   std::string libraryName() {return "hdl_lib";}
   std::string moduleName() {return "top_hdl";}
   std::string vhdlComponentListPackageName()

                                {return "hdl_comps";}
   top(sc_module_name) {}
};

int sc_main(int argc, char**) {
   top top_i("top_i");
```

< > < >

3.10. 2.10 Creating SystemC Proxy Modules from SystemC Designs

This section describes the SystemC Proxy Generator utility `sc_proxy_gen`, allowing you to create a SystemC proxy module from a SystemC design.

- Introducing the SystemC Proxy Generator
- Generating a SystemC Proxy Module
- Co-Simulation Boundary Port Types
- Co-Simulation Boundary Port Data Types
- Examples of Creating SystemC Proxy Modules
- Limitations

2.10.1 Introducing the SystemC Proxy Generator

The input for the SystemC Proxy Generator is a SystemC header file containing one or more subclasses of SystemC `sc_module`. The output is an HDL language wrapper (either Verilog or VHDL) which can be used as a SystemC proxy module in the HDL co-simulation flow.

The SystemC proxy module is an HDL foreign module. In Platform Architect HDL co-simulation, you have to put an HDL foreign module in the HDL design hierarchy to indicate that it is being connected to a SystemC design subhierarchy. The HDL foreign module will provide information so that the Synopsys SystemC kernel can generate code to instantiate the intended instance of the SystemC module, to compile user SystemC code, and/or to load SystemC libraries. In previous releases of Platform Architect, you had to create this HDL foreign module manually. The SystemC Proxy Generator utility creates such a HDL foreign module automatically.

2.10.2 Generating a SystemC Proxy Module

The SystemC Proxy Generator is a stand-alone utility which you can invoke manually to generate the HDL foreign module. You can incorporate the generated HDL foreign module into your HDL design.

To create a SystemC proxy module:

From the command line, execute:

```
sc_proxy_gen --lang language [--sim simulator]
             [-m | --module foreignModuleName]
             --sc-mod SystemCModuleName [--named] [--compiler compilerOptions]
             [--errorLimit integer] [--linker linkerOptions] [--noWarnings]
             [--pp preprocessorOptions] [--src fileList]
             -o | --output outputName [--log logName] C++FileName
```

To obtain an overview of the options you can use, execute:

```
sc_proxy_gen -h | --help
```

The following explains the options of the `sc_proxy_gen` command.

`--lang` *`language`*	Specifies the HDL language. It can be `verilog` or `vhdl`.
`--sim` *`simulator`*	Specifies the HDL simulator. Possible values are: `ncsim`, `vcs`, `mti`.
`-m | --module` *`foreignModuleName`*	Specifies the name of the generated HDL foreign module if different from the SystemC module name.
`--sc-mod` *`SystemCModuleName`*	Specifies the name of the SystemC module to be generated.
`--named`	Specifies that the port mapping is named. By default, the port mapping is positional.
`--compiler` *`compilerOptions`*	Specifies the compiler parameters.
`--errorLimit` *`integer`*	Specifies an error limit.
`--linker` *`linkerOptions`*	Specifies the linker parameters.
`--noWarning`	Specifies that no warning are shown.
`--pp` *`preprocessorOptions`*	Specifies the preprocessor options. It is possible to specify more than one preprocessor option after `--pp` and to use `--pp` more than once on the command line.
`--src` *`fileList`*	Specifies additional SystemC source files for compilation.
`-o | --output` *`outputName`*	Specifies the name of the output file.
`--log` *`logName`*	Specifies the log file name. It defaults to `stderr`.
`C++FileName`	Specifies the SystemC header file.

DEPRECATED: The `--preprocessor` option, used to specify preprocessor compiler options, is deprecated but is still available for backward compatibility. You should use the `--pp` option instead. The `--preprocessor` option splits its parameter into several parameters for the compiler when it contains spaces. This can lead to problems if, for example, include paths contain spaces. The `--pp` option puts its parameters in single quotes when invoking the compiler.

2.10.2.1 Specifying the Targeted HDL Language

Specifying the targeted HDL language for the HDL foreign module is mandatory. It can be either Verilog or VHDL. Specifying this option multiple times gives an error.

Syntax

```
--lang [verilog | vhdl]
```

Examples

- The following example generates a Verilog foreign module.

  ```
  sc_proxy_gen --lang verilog --output mxor.v mxor.h
  ```

- The following example generates a VHDL foreign module.

  ```
  sc_proxy_gen --lang vhdl --sim ncsim --output mxor.vhd mxor.h
  ```

2.10.2.2 Specifying the Targeted HDL Simulator

Specifying the targeted HDL simulator is optional. In VHDL, each VHDL simulator vendor has their own format for the VHDL foreign module. This option is required if the `--lang` option is `vhdl`. If the `--lang` option is `verilog`, this option is ignored. Currently, only two VHDL simulators are supported:

- NC-Sim
- ModelSim/Questa VHDL or Verilog Simulator

The default simulator is NC-Sim.

Syntax

```
--sim [ncsim | mti]
```

Examples

- The following example tries to generate a Verilog foreign module for NC-Sim. Since the targeted HDL language is Verilog, the simulator option is ignored.

  ```
  sc_proxy_gen --lang verilog --sim ncsim -o mxor.v mxor.h
  ```

- The following example generates a VHDL foreign module for NC-Sim.

  ```
  sc_proxy_gen --lang vhdl --sim ncsim -o mxor.vhd mxor.h
  ```

2.10.2.3 Specifying the Module Name for the Generated HDL Foreign Module

Specifying the module name for the generated HDL foreign module is optional. It is only required if you want to specify a module name different from the SystemC module name.

Syntax

`-m` *`foreignModuleName`*

or:

`--module` *`foreignModuleName`*

Examples

- The following example:

  ```
  sc_proxy_gen --lang verilog --module cwr_xor -o mxor.v mxor.h
  ```

  generates a Verilog module with the name `cwr_xor`:

  ```
  module cwr_xor
  ...
  end module
  ```

- The following example:

  ```
  sc_proxy_gen --lang vhdl --sim ncsim --module cwr_xor -o mxor.vhd mxor.h
  ```

  generates a VHDL module with the name `cwr_xor`:

  ```
  entity cwr_xor is
  ...
  end;

  architecture behave of cwr_xor is
  ...
  end behave;
  ```

2.10.2.4 Specifying the Template-Generic SystemC Module Name

Specifying the template-generic SystemC module name is mandatory. You can use this option to provide the full name when a template parameter is required for instantiating the SystemC module. This sets the parameter `CWR_SYSC_MODULE_NAME` in the HDL foreign module to the specified string. Only the template parameters of the integral data type (`short`, `int`, `long`, their `unsigned` version, and `bool`) are supported.

Syntax

`--sc-mod` *`SystemCmoduleName`*

Examples

- Consider the following nontemplatized SystemC module:

  ```
  class mxor : public sc_module {
  ...
  };
  ```

  Specifying the following command:

  ```
  sc_proxy_gen --lang verilog --sc-mod mxor -o mxor.v mxor.h
  ```

  renders:

```
module mxor
...
specify
  ...
  specparam CWR_SYSC_MODULE_NAME = "mxor";
  ...
endspecify
...
end module
```

Specifying the following command:

```
sc_proxy_gen --lang vhdl --sim ncsim --sc-mod mxor -o mxor.vhd mxor.h
```

renders:

```
entity mxor is
generic (
  ...
  constant CWR_SYSC_MODULE_NAME : string := "mxor";
  ...
      );
...
end;
```

- Consider the following SystemC module:

```
template <int N1=1, N2=5>
class mxor : public sc_module {
...
};
```

Specifying the following command:

```
sc_proxy_gen --lang verilog --sc-mod mxor -o mxor.v mxor.h
```

renders:

```
module mxor
   parameter N1 = 1;
   parameter N2 = 5;
...
specify
   ...
   specparam CWR_SYSC_MODULE_NAME = "mxor<N1, N2>";
   ...
endspecify
...
end module
```

Specifying the following command:

```
sc_proxy_gen --lang vhdl --sim ncsim --sc-mod mxor -o mxor.vhd mxor.h
```

renders:

```
entity mxor is
generic (
  constant N1 : integer = 1;
  constant N2 : integer = 2;
  ...
  constant CWR_SYSC_MODULE_NAME : string := "mxor<N1, N2>";
  ...
      );
...
end;
```

2.10.2.5 Specifying the Port Mapping Type

Specifying the port mapping type is optional. The SystemC Proxy Generator generates ports in the HDL foreign module according to the order of the ports in the SystemC module and specifies a positional mapping during the elaboration time. You can use this option to enable the named port mapping during the elaboration time.

Syntax

```
--named
```

Examples

- The following example:

  ```
  sc_proxy_gen --lang verilog --named -o mxor.v mxor.h
  ```

  generates a Verilog module with named mapping.

  ```
  module mxor
  ...
  specify
    ...
    specparam CWR_SYSC_NAMED_MAPPING_REQD =1;
        ...
  endspecify
  ...

  end module
  ```

- The following example:

  ```
  sc_proxy_gen --lang vhdl --sim ncsim --named -o mxor.vhd mxor.h
  ```

  generates a VHDL module with named mapping.

  ```
  entity mxor is
  generic (
    ...
    constant CWR_SYSC_NAMED_MAPPING_REQD : boolean = true;
     ...
        );
  ...
  end;
  ```

2.10.2.6 Specifying the C++ Compile-Time Options

Specifying C++ compile-time options is optional. You can specify the C++ compile-time options for compiling and linking of the SystemC code. These options all default to the "unset" state.

Syntax

```
--compiler compilerOptions
--errorLimit integer
--linker linkerOptions
--noWarnings
--pp preprocessorOptions
```

> **DEPRECATED:** The `--preprocessor` option, used to specify preprocessor compiler options, is deprecated but is still available for backward compatibility. You should use the `--pp` option instead. The `--preprocessor` option splits its parameter into several parameters for the compiler when it contains spaces. This can lead to problems if, for example, include paths contain spaces. The `--pp` option puts its parameters in single quotes when invoking the compiler.

Examples

- The following example:

  ```
  sc_proxy_gen --lang verilog --compiler "-O2" --errorLimit 10 --linker
               "-L../libs -lmylib" --noWarnings --pp -I/user/test
               -DMYTEST -o mxor.v mxor.h
  ```

  renders:

  ```
  module mxor
  ...
  specify
  ...
  specparam CWR_SYSC_COMPILE_OPTIONS = "compiler -O2; errorLimit 10; linker
  -L../libs -lmylib; -noWarnings; preprocessor '-I/user/test' '-DMYTEST'";
  ...
  endspecify
  ...

  end module
  ```

- The following example:

```
sc_proxy_gen --lang vhdl --sim ncsim -module --compiler "-O2"
             --errorLimit 10 --linker "-L../libs -lmylib" --noWarnings
             --pp -I/user/test --pp -DMYTEST -o mxor.vhd mxor.h
```

renders:

```
entity mxor is
generic (
  ...
  constant CWR_SYSC_COMPILE_OPTIONS: string = "compiler -O2; errorLimit 10;
           linker -L../libs -lmylib; -noWarnings; preprocessor
          `-I/user/test' '-DMYTEST'";
  ...
      ...);
...
end;
```

2.10.2.7 Specifying Additional SystemC Source Files for Compilation

Specifying additional SystemC source files for compilation is optional. You can have other SystemC source files (not header files) for the design subhierarchy precompiled and then specified in the linker option or you can specify additional source files which will be compiled during the simulation build. These additional SystemC source files are not parsed by the SystemC Proxy Generator.

Syntax

`--src` *`fileList`*

Examples

- The following command:

  ```
  sc_proxy_gen --lang verilog --src mxor.cpp -o mxor.v mxor.h
  ```

  renders:

  ```
  module mxor
  ...
  specify
     ...
    specparam specparam CWR_SYSC_FILES_LIST = "mxor.cpp";
        ...
  endspecify
  ...

  end module
  ```

- The following command:

  ```
  sc_proxy_gen -vhdl -sim ncsim -src mxor.cpp -o mxor.vhd mxor.h
  ```

  renders:

  ```
  entity mxor is
  generic (
    ...
    constant CWR_SYSC_FILES_LIST: string = "mxor.cpp";
     ...
        );
  ...
  end;
  ```

2.10.2.8 Specifying the Name of the Output File for the HDL Foreign Module

Specifying the name of the output file for the HDL foreign module is mandatory. You can provide the name of the output file. The suffix of the file name depends on the HDL language: `.v` for Verilog and `.vhd` for VHDL. You have to specify this.

Syntax

`-o` *`outputName`*

or:

`--output` *`outputName`*

Examples

- The following example generates a Verilog foreign module named `cwr_xor.v`.

  ```
  sc_proxy_gen --lang verilog --output cwr_xor mxor.h
  ```

- The following example generates a VHDL foreign module named `cwr_xor.vhd`.

  ```
  sc_proxy_gen --lang vhdl --sim ncsim --output cwr_xor mxor.h
  ```

2.10.2.9 Specifying the Log File

Specifying the log file is optional. You can provide the name of a log file for all the messages. It defaults to `stderr`.

Syntax

`--log` *`logName`*

Example

The following example generates a log file named `mylog`.

```
sc_proxy_gen --lang verilog --log mylog mxor.h
```

2.10.2.10 Specifying the Targeted SystemC Header File

Specifying the targeted SystemC header file is mandatory. You need to specify the C++ file which contains the class definition of the targeted SystemC module.

Syntax

`C++FileName`

Examples

- The following command:

  ```
  sc_proxy_gen --lang verilog -o mxor.v mxor.h
  ```

  renders:

  ```
  module mxor
  ...
  specify
    ..
    specparam specparam CWR_SYSC_HEADER_FILE = "mxor.h";
        ...
  endspecify
  ...

  end module
  ```

- The following command:

  ```
  sc_proxy_gen --lang vhdl --sim ncsim -o mxor.vhd mxor.h
  ```

  renders:

  ```
  entity mxor is
  generic (
    ...
    constant CWR_SYSC_HEADER_FILE: string = "mxor.h";
    ...
        ...);
  ...
  end;
  ```

2.10.3 Co-Simulation Boundary Port Types

The following table outlines the SystemC port types supported by the SystemC Proxy Generator utility.

Table 2-2 Supported Port Types

SystemC	VHDL	Verilog
`sc_in`	`in`	`input`

sc_out	out	output
sc_inout	inout	inout

2.10.4 Co-Simulation Boundary Port Data Types

For an overview of signal types for ports that cross the co-simulation boundary, see "Mapping Signal Types" on page 20.

2.10.5 Examples of Creating SystemC Proxy Modules

This section provides examples of creating SystemC proxy modules from SystemC designs.

2.10.5.1 Nontemplatized Subclass of a SystemC Module

This example shows a simple SystemC module and its corresponding Verilog foreign module and VHDL foreign module. This is particularly interesting as it highlights a slightly different mapping of `inout` ports in Verilog.

Consider the following SystemC module:

```
SC_MODULE(block) {
   sc_in<bool> pin;
   sc_out<bool> pout;
   ...
};
```

To generate the Verilog foreign module, execute:

```
sc_proxy_gen --lang verilog -m block -o block.v block.h
```

This command renders:

```
module block (
   pin,
   pout);
   input pin;
   output pout;
   reg pout;
specify
   specparam CWR_SYSC_MODULE_NAME = "block"
   specparam CWR_SYSC_MODULE_HDR_FILE_NAME = "block.h";
   specparam CWR_SYSC_FILES_LIST = "";
   specparam CWR_SYSC_NAMED_MAPPING_REQD = 0;
endspecify
initial
begin
   $cwr_cosim_init;
end
endmodule
```

To generate the VHDL foreign module, execute:

```
sc_proxy_gen --lang vhdl --sim ncsim -m block -o block.vhd block.h
```

This command renders:

```
entity block is
   generic (
      constant CWR_SYSC_FILES_LIST : string := "";
      constant CWR_SYSC_MODULE_NAME : string := "block";
      constant CWR_SYSC_MODULE_HDR_FILE_NAME : string := "block.h";
      constant CWR_SYSC_NAMED_MAPPING_REQD : boolean := false
   );
   port (
      pin : in std_logic;
      pout: out std_logic
   );
end;
architecture behave of block is
attribute foreign of behave:architecture is "cwrCosim:cwr_cosim_init";
begin -- architecture
end behave;
```

2.10.5.2 Nontemplatized Subclass of a SystemC Module with an inout Port

This example is similar to the previous example. The only difference is the handling of `inout` port in Verilog and VHDL.

Consider the following SystemC module:

```
   "SystemC module

SC_MODULE(block) {
   sc_in<bool> pin;
   sc_out<bool> pout;
   sc_inout<bool> pinout;
   ...
};
```

To generate the Verilog foreign module, execute:

```
sc_proxy_gen --lang verilog -m block -o block.v block.h
```

This command renders:

```
module block (
   pin,
   pout
   pinout);
   input pin;
   output pout;
   inout pinout;
   reg pout;
   reg pinout_cwr;
   wire pinout;
   assign pinout = pinout_cwr;
specify
   specparam CWR_SYSC_MODULE_NAME = "block"
   specparam CWR_SYSC_MODULE_HDR_FILE_NAME = "block.h";
   specparam CWR_SYSC_FILES_LIST = "";
   specparam CWR_SYSC_NAMED_MAPPING_REQD = 0;
endspecify
initial
begin
   $cwr_cosim_init;
end
endmodule
```

To generate the VHDL foreign module, execute:

```
sc_proxy_gen --lang vhdl -sim ncsim -m block -o block.vhd block.h
```

This command renders:

```
entity block is
   generic (
      constant CWR_SYSC_FILES_LIST : string := "";
      constant CWR_SYSC_MODULE_NAME : string := "block";
      constant CWR_SYSC_MODULE_HDR_FILE_NAME : string := "block.h";
      constant CWR_SYSC_NAMED_MAPPING_REQD : boolean := false
   );
   port (
      pin : in std_logic;
      pout: out std_logic;
      pinout: inout std_logic
   )
end;
architecture behave of block is
attribute foreign of behave:architecture is "cwrCosim:cwr_cosim_init";
begin -- architecture
end behave;
```

2.10.5.3 Templatized Subclass of a SystemC Module

Template support is limited to `short`, `int`, `long`, `unsigned short`, `unsigned int`, `unsigned long`.

Consider the following SystemC module:

```
   "SystemC module

template <int N1=5, N2=10, N3=32>
SC_MODULE(block) {
```

```
   sc_in< sc_int<N1> > pin;
   sc_out< sc_int<N2> > pout;
   sc_inout< sc_int<N3> > pinout;
   ...
};
```

To generate the Verilog foreign module, execute:

```
sc_proxy_gen --lang verilog -m block -o block.v block.h
```

This command renders:

```
module block (
   pin,
   pout,
   pinout);
   parameter N1 = 5;
   parameter N2 = 10;
   parameter N3 = 32;
   input [N1-1 : 0] pin;
   output [N2-1 : 0] pout;
   inout [N3-1 : 0] pinout;
   reg [N2-1 : 0] pout;
   reg [N3-1 : 0] pinout_cwr;
   wire [N3-1 : 0] pinout;
   assign pinout = pinout_cwr;
specify
   specparam CWR_SYSC_MODULE_NAME = "block<N1, N2, N3>"
   specparam CWR_SYSC_MODULE_HDR_FILE_NAME = "block.h";
   specparam CWR_SYSC_FILES_LIST = "";
   specparam CWR_SYSC_NAMED_MAPPING_REQD = 0;
endspecify
initial
begin
   $cwr_cosim_init;
end
endmodule
```

To generate the VHDL foreign module, execute:

```
sc_proxy_gen --lang vhdl --sim ncsim -m block -o block.vhd block.h
```

This command renders:

```
entity block is
   generic (
      constant N1 : integer = 5;
      constant N2 : integer = 10;
      constant N3 : integer = 32;
      constant CWR_SYSC_FILES_LIST : string := "";
      constant CWR_SYSC_MODULE_NAME : string := "block<N1, N2, N3>";
      constant CWR_SYSC_MODULE_HDR_FILE_NAME : string := "block.h";
      constant CWR_SYSC_NAMED_MAPPING_REQD : boolean := false
   );
   port (
      pin : in std_logic_vector( N1-1 downto 0);
      pout: out std_logic_vector( N2-1 downto 0);
      pinout: inout std_logic_vector( N3-1 downto 0)
   );
end;
architecture behave of block is
attribute foreign of behave:architecture is "cwrCosim:cwr_cosim_init";
begin -- architecture
end behave;
```

2.10.6 Limitations

This section describes limitations related to template SystemC modules, SystemC modules with multiple constructors, and module name specifications, respectively.

2.10.6.1 Template SystemC Module Restrictions

The following template SystemC module restrictions apply:

- Templates with `class`/`typename`/`template` parameters are not supported.

Templates with `class`/`typename`/`template` parameters are not supported as the class `T` cannot be mapped generically to HDL. It may be anything and it is not known what it could be mapped to just looking at the template.

```
template<class T>
class mod1 : public sc_module {
   ...
};
```

- Templates with user or partial specialization are not supported.

  If you have specified a template generic as well as a user specialization or partial specialization then the code for the user-specialized class is not used. Only the template generic's code is mapped to a parametrized HDL model. For example:

```
// Will be generated as generic
template<int I>
class mod2 : public sc_module {
   sc_in< sc_bv<I> > pin; // This uses "sc_bv"
   ...
};
// Will not be used to map to HDL
template<>
class mod2<5> : public sc_module {
   sc_in < sc_lv<5> > pin;
   // Contrived but highlights that the user may have
   // a completely different type "sc_lv" wrt the generic
};
```

- Template instances are not supported.

  For SystemC module definition, instances of the module must not be specified on the command line. For example:

```
template <int N>
class mod3 : public sc_module {
   ...
};
// End SystemC code
--sc-mod mod3 : ok
--sc-mod "mod3<5>": unsupported
```

2.10.6.2 SystemC Module with Multiple Constructors

Only the constructor which takes a module name as argument is supported.

2.10.6.3 Module Name Specification Restrictions

The full module name needs to be supplied on the command line. For example, for module `mod1` in the following example, you need to specify `-sc-mod allmods::mod1` on the command line.

```
namespace allmods {
   template <int N>
   class mod1 : public sc_module {..}
}
using namespace allmods;
// end SystemC code
```

The following command-line specification is inadequate:

```
--sc-mod mod1
```

The following command-line specification is correct:

```
--sc-mod allmods::mod1
```

The generated name will by default be based on the `sc-mod` name.

< > < >

3.11. 2.1 SystemC-HDL Co-Simulation API Reference

The following lists the full SystemC-HDL Co-Simulation API, which contains one class, `sc_hdl_module`.

- Constructor
- Public Member Functions
- Member Function Documentation

2.11.1 Constructor

- SC_CTOR(*moduleName*):*initializerList* {...};

2.11.2 Public Member Functions

- virtual std::string *moduleName*() = 0;
- virtual std::string *libraryName*() = 0;
- virtual bool isPositionalPortMapping() { return true; }
- virtual std::string vhdlComponentListPackageName() { return std::string(); }
- virtual void getVhdlSourceFiles(std::vector<std::string>& file_list) {}
- virtual void getVerilogSourceFiles(std::vector<std::string>& file_list) {}
- virtual bool isHDLModule() { return true; }

2.11.3 Member Function Documentation

`SC_CTOR(moduleName):initializerList {...};`

Use this macro to call the constructor. *initializerList* is the list of initializations for the data members of the class *moduleName*. This list must include the port mappings if port mapping by name is selected. For more information, see "Mapping Ports" on page 26.

`virtual std::string moduleName() = 0;`

Specifies the HDL package or module name. For more information, see "Specifying the HDL Module Name" on page 25.

`virtual std::string libraryName() = 0;`

Specifies the library name. For more information, see "Specifying the Library Name" on page 25.

`virtual bool isPositionalPortMapping() { return true; }`

Overrides this member function if you want port mapping by name. For more information, see "Mapping Ports" on page 26.

`virtual std::string vhdlComponentListPackageName() { return std::string(); }`

For SystemC-VHDL designs, specifies the name of the package where the component declaration of the VHDL design you want to instantiate is defined. For more information, see "Specifying the Component List Package Name" on page 27.

`virtual void getVhdlSourceFiles(std::vector<std::string>& file_list) {}`

Specifies a VHDL source list. For more information, see "Specifying the HDL Source Files" on page 26.

`virtual void getVerilogSourceFiles(std::vector<std::string>& file_list) {}`

Specifies a Verilog source list. For more information, see "Specifying the HDL Source Files" on page 26.

`virtual bool isHDLModule() { return true; }`

Used to check all SystemC modules to determine whether the module is a native SystemC module or an HDL proxy module. This member function returns `TRUE` for HDL proxy modules and `FALSE` for SystemC native modules. You cannot override this function. For more information, see "Checking the Source Type" on page 29.

< > < >

3.12. 2.12 Coding Examples

This section describes:

- Coding Examples for HDL Proxy Modules in SystemC Designs
- Coding Examples for a SystemC Proxy Model in HDL Designs

2.12.1 Coding Examples for HDL Proxy Modules in SystemC Designs

This section describes coding examples for HDL proxy modules in SystemC designs.

2.12.1.1 Coding a VHDL Proxy Module in SystemC

This section describes how to convert a SystemC module into an HDL proxy module for a VHDL model.

Assume the following SystemC example with a hierarchy structure shown below.

Figure 2-7 The Hierarchy Structure of a SystemC Module

In the example, we want to replace the SystemC description of `pipeline` with a VHDL description of `pipeline` to create the following hierarchy.

Figure 2-8 The Hierarchy Structure of a SystemC Module with the Replaced Pipeline

To convert the SystemC module into an HDL proxy module perform the following steps:

1. Coding VHDL Model for the Pipeline
2. Modifying the SystemC Module Pipeline to a VHDL Proxy Module

The source code for the SystemC module `pipeline` looks as follows:

pipeline.h

```
#ifndef PIPELINE_H
#define PIPELINE_H

#include "systemc.h"
#include "stage1.h"

#include "stage2.h"

SC_MODULE (pipeline) {

    sc_in<int>     in1;
    sc_in<int>     in2;
    sc_in<bool>    clk;
    sc_out<int>    pow;

    sc_signal<int> prod;
    sc_signal<int> quot;

    stage1*       dStage1;
    stage2*       dStage2;

    //Constructor
    SC_CTOR( pipeline ) {
        dStage1 = new stage1("stage1");
        (*dStage1)(in1, in2, prod, quot, clk);
```

2.12.1.1.1 Coding VHDL Model for the Pipeline

The VHDL model definition for `pipeline` is listed below. Port mapping between the HDL proxy ports and the actual HDL model is name based.

The `pipeline_p.vhd` package file defines the package for `pipeline`.

pipeline_p.vhd

```
LIBRARY ieee;
use ieee.std_logic_1164.all;

package componentList is
   component pipeline
      port(
            clock   : in std_logic;
            input1  : in std_logic_vector;
            powerof : out std_logic_vector;
            input2  : in std_logic_vector;
      );
   end component;

end componentList;
```

The `pipeline.vhd` file defines the behavior of `pipeline`.

pipeline.v

```
LIBRARY ieee;
use ieee.std_logic_1164.all;

entity pipeline is
   port(
         clock   : in std_logic;
         input1  : in std_logic_vector;

         powerof : out std_logic_vector;
         input2  : in std_logic_vector;
   );
end pipeline;
architecture behave of pipeline is
begin -- architecture
...
end behave;
```

2.12.1.1.2 Modifying the SystemC Module Pipeline to a VHDL Proxy Module

The `pipeline.cpp` file describing the module definition for `pipeline` looks as follows:

pipeline.c

```
SC_HDL_MODULE (pipeline) {

    sc_in<int>      in1;
    sc_in<int>      in2;
    sc_in<bool>     clk;
    sc_out<int>     pow;

  sc_signal<int> prod;
  sc_signal<int> quot;

  stage1*         dStage1;
  stage2*         dStage2;

  SC_CTOR( pipeline ) : in1("input1"), in2("input2"),
        clk("clock"), pow("powerof") {
    }

  virtual std::string libraryName() { return "work"; }
  virtual std::string moduleName() { return "pipeline"; }
  virtual std::string vhdlComponentListPackageName()
                                  { return "componentList"; }
  virtual void getVhdlSourceFiles(std::vector<std::string>& file_list)
  { file_list.push_back("pipeline.vhd");
  file_list.push_back("pipeline_p.vhd"); }
}

  virtual bool isPositionalPortMapping() { return false; }
```

Use the macro to derive a class from `sc_hdl_module`.

Interface signals to the HDL.

The following signals are not required and should be removed. The simulator may return an error or may ignore these signals.

The module handles are no longer needed.

In the constructor, you map the VHDL port names to SystemC port names.

Return module name, library name, and component list package name.

Return VHDL source file list.

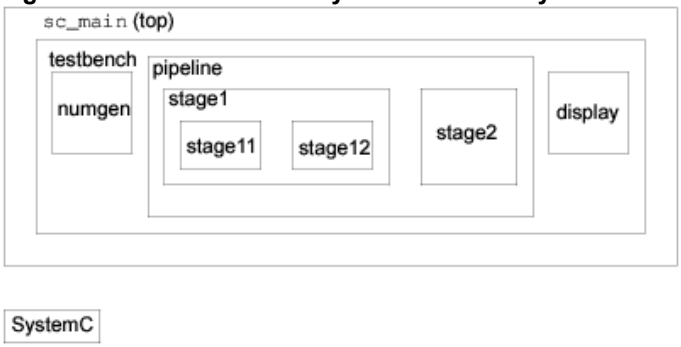
Port mapping is name based.

2.12.1.2 Coding a Verilog Proxy Module in SystemC

This section describes how to convert a SystemC module into an HDL proxy module for a Verilog model.

Assume the following SystemC example with a hierarchy structure shown below.

Figure 2-9 The Hierarchy Structure of a SystemC Module



In the example, we want to replace the SystemC description of `pipeline` with a Verilog description of `pipeline` to create the following hierarchy.

Figure 2-10 The Hierarchy Structure of a SystemC Module with the Replaced Verilog Description of the Pipeline

To convert the SystemC module into an HDL proxy module perform the following steps:

1. Coding Verilog Model for the Pipeline
2. Modifying the SystemC Module Pipeline to a Verilog Proxy Module

The source code for the `pipeline` SystemC module looks as follows:

pipeline.h

```
#ifndef PIPELINE_H
#define PIPELINE_H

#include "systemc.h"
#include "stage1.h"
#include "stage2.h"

SC_MODULE (pipeline) {

     sc_in<int>     in1;
     sc_in<int>     in2;
     sc_in<bool>    clk;
     sc_out<int>    pow;

     sc_signal<int> prod;
     sc_signal<int> quot;

     stage1*       dStage1;
     stage2*       dStage2;

     //Constructor
     SC_CTOR( pipeline ) {
          dStage1 = new stage1("stage1");
          (*dStage1)(in1, in2, prod, quot, clk);
          dStage2 = new stage2("stage2");
          (*dStage2)(prod, quot, pow, clk);
     }
};
```

2.12.1.2.1 Coding Verilog Model for the Pipeline

The Verilog model definition for `pipeline` is listed below. Port mapping between the HDL proxy ports and the actual HDL model is name based.

The `pipeline.v` file defines the Verilog model for `pipeline`.

pipeline.v

```
`timescale 1 ps / 1 ps

module pipeline (
   clock,
   input1,
   powerof,

   input2
);

   input              clock;
   input     [31:0]   input1;
   output    [31:0]   powerof;
   input     [31:0]   input2;
   reg       [31:0]   powerof;
...

endmodule
```

2.12.1.2.2 Modifying the SystemC Module Pipeline to a Verilog Proxy Module

The `pipeline.cpp` file describing the module definition for `pipeline` looks as follows:

pipeline.cpp

```
SC_HDL_MODULE (pipeline) {

      sc_in<int>    in1;
      sc_in<int>    in2;
      sc_in<bool>   clk;
      sc_out<int>   pow;

   sc_signal<int> prod;
   sc_signal<int> quot;

   stage1*     dStage1;
   stage2*     dStage2;

   SC_CTOR( pipeline ) : in1("input1"), in2("input2"),
      clk("clock"), pow("powerof") {
       }

   virtual std::string libraryName() { return "work"; }
   virtual std::string moduleName() { return "pipeline"; }

   virtual void getVerilogSourceFiles(std::vector<std::string>& file_list)
   { file_list.push_back("pipeline.v"); }

   virtual bool isPositionalPortMapping() { return false; }
```

Use the macro to derive a class from `sc_hdl_module`.

Interface signals to the HDL.

The following signals are not required and should be removed.The simulator may return an error ormay ignore these signals.

The module handles are no longer needed.

In the constructor, you map the Verilog port names to SystemC port names.

Return module name and library name.

Return Verilog source file list.

Port mapping is name based.

2.12.2 Coding Examples for a SystemC Proxy Model in HDL Designs

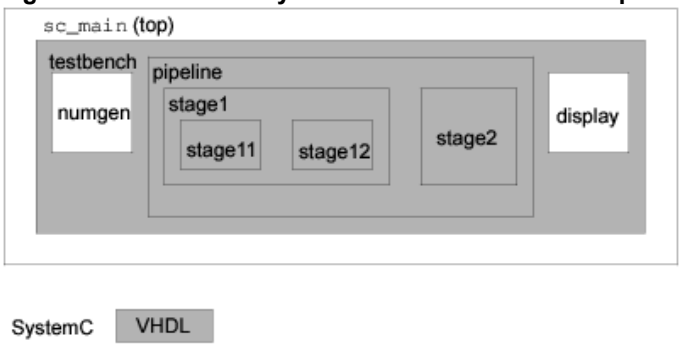
This section provides coding examples for a SystemC proxy model in HDL designs.

2.12.2.1 Coding a SystemC Proxy Model in VHDL

This section describes how to code a SystemC proxy model in VHDL.

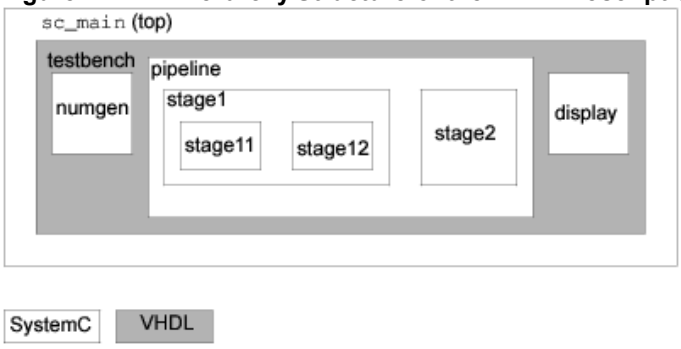
Assume the following VHDL example with the hierarchy structure shown below.

Figure 2-11 Hierarchy Structure of a VHDL Description



We want to replace the VHDL description of `pipeline` with a SystemC description to create the following hierarchy.

Figure 2-12 Hierarchy Structure of the VHDL Description with the Pipeline Replaced



To code the SystemC proxy model in VHDL perform the following steps:

1. Coding the SystemC Module for the Pipeline
2. Coding the SystemC Proxy Model in VHDL for the Pipeline

2.12.2.1.1 Coding the SystemC Module for the Pipeline

The `pipeline.h` file defines the SystemC module for `pipeline`.

pipeline.h

```
#ifndef PIPELINE_H
#define PIPELINE_H

#include "systemc.h"
#include "stage1.h"
#include "stage2.h"

SC_MODULE (pipeline) {

     sc_in<int>     in1;
     sc_in<int>     in2;
     sc_in<bool>    clk;
     sc_out<int>    pow;

     sc_signal<int> prod;
     sc_signal<int> quot;

     stage1*       dStage1;
     stage2*       dStage2;

     //Constructor
     SC_CTOR( pipeline ) : in1("input1"), in2("input2"),
                        clk("clock"), pow("powerof")
      {
          dStage1 = new stage1("stage1");
          (*dStage1)(in1, in2, prod, quot, clk);
          dStage2 = new stage2("stage2");
          (*dStage2)(prod, quot, pow, clk);
      }
};

#endif
```

2.12.2.1.2 Coding the SystemC Proxy Model in VHDL for the Pipeline

The `pipeline.vhd` file defines the SystemC proxy model for `pipeline`.

pipeline.v

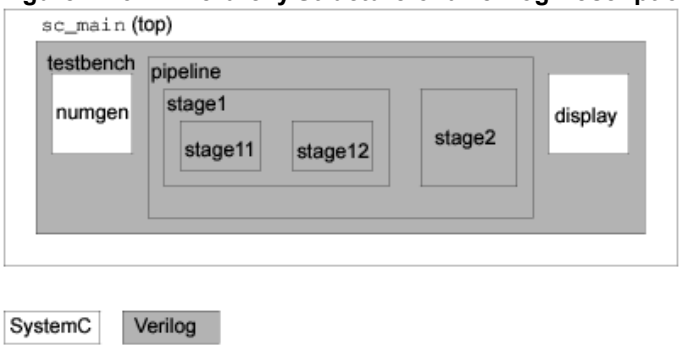
```
LIBRARY ieee;
use ieee.std_logic_1164.all;

entity pipeline is
   generic(
      constant CWR_SYSC_FILES_LIST : string :=
            "my list of source files or fof";
      constant CWR_SYSC_MODULE_NAME : string := "pipeline";
      constant CWR_SYSC_MODULE_HDR_FILE_NAME : string :=
            "../inc/pipeline.h";
      constant CWR_SYSC_COMPILE_OPTIONS : string :=
            "preprocessor I/user4/home/test/cosim/; compiler -g";
      constant CWR_SYSC_NAMED_MAPPING_REQD : boolean := true;
   );

   port (
      clock    : in std_logic;
      input1   : in std_logic_vector;
      powerof  : in std_logic_vector;
      input2   : out std_logic_vector;
   );
end pipeline;
architecture behave of pipeline is
attribute foreign of behave : architecture is :
   "cwrCosim:cwr_cosim_init";
begin
end behave;
```

2.12.2.2 Coding a SystemC Proxy Model in Verilog

This section describes how to code a SystemC proxy model in Verilog.

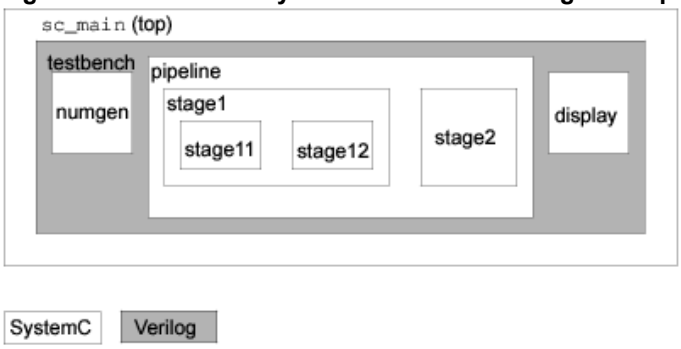
Assume the following Verilog example with the hierarchy structure shown below.

Figure 2-13 Hierarchy Structure of a Verilog Description



We want to replace the Verilog description of `pipeline` with a SystemC description to create the following hierarchy.

Figure 2-14 Hierarchy Structure of the Verilog Description with the Pipeline Replaced



To code the SystemC proxy model in Verilog perform the following steps:

1. Coding the SystemC Module for the Pipeline
2. Coding the SystemC Proxy Model in Verilog for the Pipeline

2.12.2.2.1 Coding the SystemC Module for the Pipeline

The `pipeline.h` file defines the SystemC module for `pipeline`.

pipeline.h

```
#ifndef PIPELINE_H
#define PIPELINE_H

#include "systemc.h"
#include "stage1.h"
#include "stage2.h"


SC_MODULE (pipeline) {

     sc_in<int>     in1;
     sc_in<int>     in2;
     sc_in<bool>    clk;
     sc_out<int>    pow;

     sc_signal<int> prod;
     sc_signal<int> quot;

     stage1*       dStage1;
     stage2*       dStage2;

     //Constructor
     SC_CTOR( pipeline ) {
          dStage1 = new stage1("stage1");
          (*dStage1)(in1, in2, prod, quot, clk);
          dStage2 = new stage2("stage2");
          (*dStage2)(prod, quot, pow, clk);
    }
};

#endif
```

2.12.2.2.2 Coding the SystemC Proxy Model in Verilog for the Pipeline

The `pipeline.v` file defines the SystemC proxy model for `pipeline`.

pipeline.v

```
module pipeline (input1, input2, clock, powerof);

      input  [31:0]  input1;
      input  [31:0]  input2;
      input          clock;
      output [31:0]  power;
      reg    [31:0]  power;

  specify
     specparam CWR_SYSC_FILES_LIST = "../modules/src/pipeline.cpp";
     specparam CWR_SYSC_MODULE_NAME = "pipeline";
     specparam CWR_SYSC_MODULE_HDR_FILE_NAME =
                                     "../modules/inc/pipeline.h";     specparam
CWR_SYSC_COMPILE_OPTIONS =
             "preprocessor -I/user4/home/test/cosim/; compiler -g";
     specparam CWR_SYSC_NAMED_MAPPING_REQD = 0;
  endspecify

  /*interface initialization function name cosim_shell init
  is fixed by the system and should be the same for all foreign
  models */

  initial
  begin
    $cwr_cosim_init;
  end
endmodule
```

2.12.2.3 Coding a SystemC Proxy Model in Verilog with inout Ports

This section describes how to code a SystemC proxy model in Verilog with `inout` port.

Assume the Verilog example of figures 2-13 and 2-14.

To code the SystemC proxy model in Verilog with `inout` ports, perform the following steps:

1. Coding the SystemC Module for the Pipeline
2. Coding the SystemC Proxy Model in Verilog for the Pipeline

2.12.2.3.1 Coding the SystemC Module for the Pipeline

The `pipeline.h` file defines the SystemC module for `pipeline`.

pipeline.h

```
#ifndef PIPELINE_H
#define PIPELINE_H

#include "systemc.h"
#include "stage1.h"
#include "stage2.h"

SC_MODULE (pipeline) {

     sc_in<int>     in1;
     sc_in<int>     in2;
     sc_in<bool>    clk;
     sc_inout<int>    pow;

     sc_signal<int> prod;
     sc_signal<int> quot;

     stage1*       dStage1;
     stage2*       dStage2;

     //Constructor
     SC_CTOR( pipeline ) {
         dStage1 = new stage1("stage1");
         (*dStage1)(in1, in2, prod, quot, clk);
          dStage2 = new stage2("stage2");
          (*dStage2)(prod, quot, pow, clk);
    }
};

#endif
```

2.12.2.3.2 Coding the SystemC Proxy Model in Verilog for the Pipeline

The `pipeline.v` file defines the SystemC proxy model for `pipeline`.

pipeline.v

```
module pipeline (input1, input2, clock, powerof);

      input  [31:0]  input1;
      input  [31:0]  input2;
      input          clock;
      inout [31:0]   power;
      reg   [3:0]    power_cwr;
      assign power = power_cwr;

  specify
     specparam CWR_SYSC_FILES_LIST = "../modules/src/pipeline.cpp";
     specparam CWR_SYSC_MODULE_NAME = "pipeline";
     specparam CWR_SYSC_MODULE_HDR_FILE_NAME =
                                    "../modules/inc/pipeline.h";
     specparam CWR_SYSC_COMPILE_OPTIONS =
             "preprocessor -I/user4/home/test/cosim/; compiler -g";
     specparam CWR_SYSC_NAMED_MAPPING_REQD = 0;
  endspecify

  /*interface initialization function name cosim_shell init
  is fixed by the system and should be the same for all foreign
  models */

  initial
  begin
    $cwr_cosim_init;
  end
endmodule
```

< > < >

3.13. 2.1 Building and Running a Simulation

This section describes how to build and run a SystemC-HDL co-simulation, including:

- Specifying the Number of Language Switches
- Loading the Design
- Specifying Co-Simulation Time-Out
- Selecting the HDL Simulator
- Selecting the Simulator Mode
- Setting Simulation Options
- Setting Command-Line Arguments
- Selecting the Synchronization Mode
- Mapping Logical Library Names to Physical Directories
- ModelSim/Questa Support for HDL Simulator Simulation Control
- Managing Errors and Warnings
- Running the Simulation

2.13.1 Specifying the Number of Language Switches

Before you load the simulator, you must set the number of language switches allowed for recursive mixed language nesting using the SystemC Shell command "cosim::set_hdl_max_nesting" on page 94.

For example, the top level SystemC can contain HDL, which can contain SystemC, and so on. In this case, you must specify the maximum number of language switches allowed. The top (always SystemC) is considered `0`; For more information, see "cosim::set_hdl_max_nesting" on page 94.

To set the number of language switches allowed:

Type the following:

`scsh > cosim::set_hdl_max_nesting` *`value`*

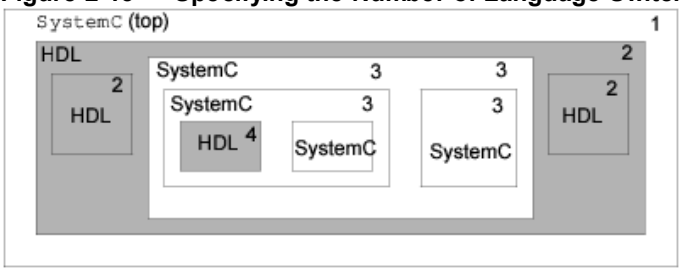
where *`value`* is the number of language switches allowed for mixed-language nesting.

The following example sets the maximum number of levels allowed for mixed-language nesting to `4`.

```
cosim::set_hdl_max_nesting 4
```

In the illustration, the number indicates the number of language switches from the top to every module. The maximum number is 4.

Figure 2-15 Specifying the Number of Language Switches



2.13.2 Loading the Design

You load the design for SystemC Shell. For the procedure, see the *SystemC Shell Manual*.

2.13.3 Specifying Co-Simulation Time-Out

In a SystemC-HDL co-simulation process, you must specify the maximum time in seconds SystemC Shell waits for the HDL simulator to report back. When the specified time is exceeded, SystemC Shell reports an error. By default, SystemC Shell waits 120s (2 minutes) when you run the simulation in batch mode, 1800s (30 minutes) when you run the simulation in interactive mode. For more information on running SystemC Shell in batch or interactive mode, see "Selecting the Simulator Mode" on page 81.

Note	The time you specify should exceed the time the HDL simulator needs to perform its own simulation step.

To specify co-simulation time-out:

Type the following:

```
scsh > cosim::set_cosim_timeout time
```

where *`time`* is the maximum time (in seconds) SystemC Shell waits for the HDL simulator to report back.

2.13.4 Selecting the HDL Simulator

SystemC-HDL co-simulation supports the following HDL simulators:

- Synopsys VCS and VCSi or Verilog simulation
- Cadence Incisive NC-VHDL for VHDL simulation
- Cadence Incisive NC-Verilog for Verilog simulation
- Cadence Incisive NC-Sim for mixed-Verilog/VHDL simulation
- Mentor Graphics Modelsim/Questa VHDL or Verilog Simulator for mixed-Verilog/VHDL simulation

For the supported versions of these HDL simulators, see "Third-Party Tools" in the *Synopsys Virtual Prototyping Product Installation Guide*.

To select the HDL simulator:

Type the following:

```
scsh > cosim::set_hdl_sim_type {ncsim | vcs | vcsi | mti}
```

where:

`ncsim`	Specifies to invoke NC-Verilog, NC-VHDL, or NC-Sim, depending on your installation.
`vcs`	Specifies to invoke the VCS simulator.

`vcsi`	Specifies to invoke the VCSi simulator. For designs with SystemC instantiated in Verilog, `vcsi` generates the following message, which you can ignore:
	`VPI Error:vpi_handle_by_name: Object CWR_HDL_SHELL not found`
	`At time 0 in file shell2.v line 30`

`mti`	Specifies to invoke the ModelSim/Questa simulator.

The following example invokes the VCS simulator:

```
scsh > cosim::set_hdl_sim_type vcs
```

2.13.5 Selecting the Simulator Mode

You can run the HDL simulation from the GUI or from the command line. If you do not specify a mode, the simulator starts in command-line mode.

To select the simulator mode:

Type the following:

```
scsh > cosim::set_hdl_sim_mode {batch | interactive}
```

where:

`batch`	Specifies that you want to run the simulator from the command line.
`interactive`	Specifies that you want to run the simulator from the GUI.

The following example invokes the simulator in GUI mode:

```
scsh > cosim::set_hdl_sim_mode interactive
```

2.13.6 Setting Simulation Options

Each HDL simulator has a set of options that are specific to the simulator you select. You can set options for the different phases of the simulation process: compilation, elaboration, and runtime.

You can also retrieve information on the options that are set.

To set simulation options:

Type the following:

```
scsh > cosim::set_hdl_options {ncsim | vcs | vcsi | mti }
       {vlog_compile | vhdl_compile | elaborate | runtime}
       value [value]...
```

where *`value`* is simulator specific. For more information on the options, see the documentation of your HDL simulator.

The following examples illustrate this.

- The following command sets compilation options for a Verilog design simulated in VCS:

  ```
  scsh > cosim::set_hdl_options vcs vlog_compile -notice
         -l myVerilog.log
  ```

- The following command sets compilation options for a VHDL design simulated in NC-Sim:

  ```
  scsh > cosim::set_hdl_options ncsim vhdl_compile -mess -smartorder -status
         -list -logfile myVhdl.log
  ```

- The following command sets elaboration options for a design simulated in VCS:

  ```
  scsh> cosim::set_hdl_options vcs elaborate -notice
  ```

- The following command sets runtime options for a design simulated in VCS:

  ```
  scsh > cosim::set_hdl_options vcs runtime -notice -l sim.log
  ```

- The following command sets a time resolution to invoke Modelsim/Questa. The default time resolution used in ModelSim/Questa is 1 ps.

  ```
  scsh> [::scsh::cosim::]set_hdl_options mti runtime -t 1fs
  ```

To retrieve information on the simulation options specified:

Type the following:

```
scsh > cosim::show_hdl_options [ncsim | vcs | vcsi | mti]
```

If you do not specify a simulator, all specified options for all simulators are listed. "Performance Optimization with VCS Co-Simulation" on page 107 describes some optimization flags that can be used with VCS to speed up co-simulation.

2.13.7 Setting Command-Line Arguments

You can set command-line arguments for the SystemC process invoked by the HDL simulator.

To specify the command-line arguments:

Type the following:

```
scsh > simargs argument [argument]...
```

where *argument* is the argument you want to pass to the SystemC simulator.

Specifying this command causes the SystemC simulator to be invoked from the HDL simulator as if the following was specified:

```
char* av[N]={program, arg0, arg1, ..., argM};
return sc_main(N, av);
```

As is UNIX convention, *program* is treated as the first argument.

2.13.8 Selecting the Synchronization Mode

The two simulators, SystemC Shell and the HDL simulator must be synchronized so that the models running on the two simulators can communicate and the value updates result in correct simulation results. The simulators can be synchronized at different levels:

- At the delta-cycle level, where the internal simulation loops are synchronized. The two simulators must run at the same time resolution.

  Delta cycles are simulation cycles for which the simulation time does not advance. Delta cycles are used for more deterministic simulation results by avoiding race conditions. Synchronization at delta cycle means that both simulators execute their process evaluation phases before the signal update phase is executed.

  Because the simulators synchronize at each delta cycle, the overall simulation speed is slower compared to event-based synchronization.

- At event-based level, where the simulators synchronize if an event occurs across the boundary or at a place where an event could potentially occur. The two simulators can run at different time resolutions.

  SystemC Shell can run at a coarser resolution than the HDL simulator. If so, the simulation time resolution of SystemC Shell must be a whole number multiple of the time resolution of the HDL simulator. This guarantees that, any time SystemC Shell breaks while debugging, the simulation times of both the simulators are the same.

  Because the simulators do not have to synchronize whenever the HDL simulator with finer resolution sends an event to the SystemC simulator, the overall simulation speed is faster compared to delta-cycle synchronization.

Note	Using the event-based synchronization mode may result in incorrect timing results if the model depends on true signal/clock behavior.

By default, when modeling across simulation boundaries, the synchronization between SystemC and HDL is event based.

To select the synchronization mode:

Type the following:

```
scsh > cosim::set_delta_cycle_sync {true | false}
```

The following example specifies that you want to run the simulator in the delta-cycle-synchronization mode:

```
scsh > cosim::set_delta_cycle_sync true
```

2.13.9 Mapping Logical Library Names to Physical Directories

You must map an HDL logical library name to a directory path. Because you map the logical library name to the physical directory path in SystemC Shell (and not in your HDL model), you can run different simulations with different HDL libraries without changing the source code of the model.

Within a SystemC Shell session, you can map several logical library names to their physical library paths.

The library name that the member function `sc_hdl_module::library` returns (see "Specifying the Library Name" on page 25) is the logical library name. The HDL simulator assumes that the mapping to a physical directory exists.

To map a HDL library name to a physical directory:

Type the following:

```
scsh > cosim::map_hdl_library_name name path
```

where:

name	Specifies the name of the HDL library.
path	Specifies the path name to the directory where the library resides.

The following example maps an HDL library to the directory where the library resides:

```
scsh > cosim::map_hdl_library_name myLib /usr1/home/joe/work
```

2.13.10 ModelSim/Questa Support for HDL Simulator Simulation Control

ModelSim/Questa DO files are scripts that contain ModelSim/Questa commands and Tcl commands. You can specify a DO file in the batch mode simulation. If you do not specify a DO file, the simulation simply runs and quits.

To specify a DO file, use `runtime` of the `set_hdl_options` command as the first argument in the list. For example:

```
scsh> cosim::set_hdl_options mti runtime -do macro_file -t 1fs
```

Caution	Make sure that you specify `-do` as the first option, otherwise it will not be taken into account.

2.13.11 Managing Errors and Warnings

You can control the errors and warnings displayed by the parser, as described in "Managing SystemC Parser Errors and Warnings" of the *SystemC Shell Manual* for SystemC Shell.

2.13.12 Running the Simulation

You run the simulator, as described in "Running the Simulation" of the *SystemC Shell Manual* for SystemC Shell.

< > < >

3.14. 2.14 Debugging a SystemC-HDL Co-Simulation

This section describes the issues related to debugging a SystemC-HDL co-simulation. They include:

- Using VP Explorer
- Using the SystemC Shell

2.14.1 Using VP Explorer

Using and debugging a co-simulation from VP Explorer works exactly like for normal simulations. Note that instead of a simulation executable in `Debug/sim` there will be a symbolic link `Debug/sim` that points to a script `Debug/run_cosim.sh`. This script is generated when the co-simulation is created with SystemC Shell. It allows to run the co-simulation like a normal simulation. The symbolic link has the same name as a "normal" pure SystemC simulation so that it can be used in the standard debugging flow. For example, starting the simulation from Platform Creator in VP Explorer works exactly like for standard SystemC simulations.

2.14.2 Using the SystemC Shell

This section explains:

- Debugging Source Code
- Navigating the Design Hierarchy
- Setting Breakpoints

2.14.2.1 Debugging Source Code

The SystemC module instances that area embedded in the HDL hierarchy are debugged at source level; that is, in the same way as any other SystemC module.

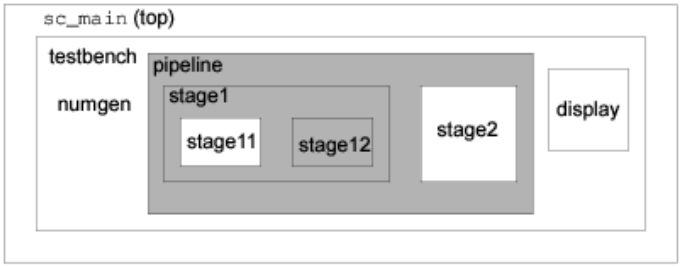
2.14.2.2 Navigating the Design Hierarchy

You navigate the design hierarchy as described in "Navigating the Design Hierarchy" in the *SystemC Shell Manual* for SystemC Shell. You can only navigate the SystemC hierarchy. You cannot traverse the HDL hierarchy in SystemC Shell. Instead, use an HDL browsing tool.

When navigating the HDL design hierarchy and the hierarchy contains SystemC module instances, these SystemC modules are listed at the top level of the SystemC hierarchy, as shown in the example below.

Assume the following design hierarchy.

Figure 2-16 A Design Hierarchy for Navigation



As shown in "Design Flow" on page 15, `sc_main` contains, in addition to `testbench`, the SystemC modules that are embedded in HDL (`stage11` and `stage2`). The names of the embedded modules in `sc_main` are the full hierarchical path names separated by `_V_`.

The following example illustrates the output after issuing navigation commands in SystemC Shell:

```
scsh > cd /
scsh > ls

testbench (kind: sc_module)
/testbench_V_pipeline_V_stage1_V_stage11 (kind: sc_module)
/testbench_V_pipeline_V_stage2 (kind: sc_module)

scsh > cd testbench
scsh > ls

numgen (kind: sc_module)
display (kind: sc_module)
pipeline (kind: hdl_module)
```

The names of the SystemC modules include the path through the co-simulation boundary. The hierarchy delimiter is `_V_`.

2.14.2.3 Setting Breakpoints

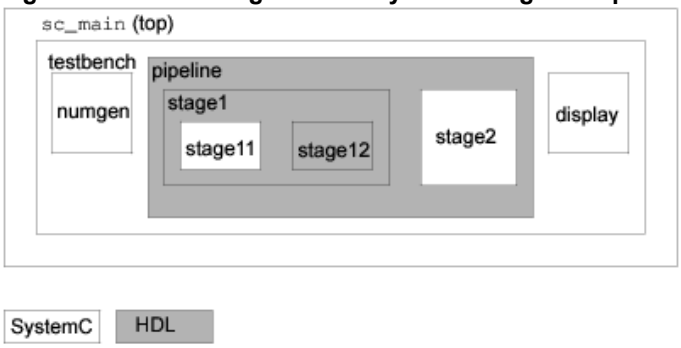
You specify breakpoints as described for SystemC Shell in "Setting Unconditional Breakpoints" and "Setting Conditional Breakpoints" of the *SystemC Shell Manual*.

You can only specify breakpoints in the SystemC modules, not in HDL proxy modules and in HDL models.

If you want to set a breakpoint in a SystemC module instance that is part of the HDL design hierarchy, you must specify the full path to define the port, as shown in the following example.

Assume the following design hierarchy.

Figure 2-17 A Design Hierarchy for Setting Breakpoints



The following command sets a threshold-event breakpoint on `sum`, which is a port in module instance `stage11`. The instance is simulated in SystemC and is instantiated from `pipeline`, which is an HDL model:

```
scsh > teb /testbench_V_pipeline_V_stage1_V_stage11/sum >= 21
```

The hierarchy delimiter is `_V_`.

3.15. 2.15 Exporting Structural HDL

This section describes how to automatically export structural HDL hierarchy from SystemC hierarchy into HDL so that the exported hierarchy can be simulated in an HDL simulator. By moving the structural netlist from SystemC to HDL, the connection ports across the co-simulation boundary are reduced. As a result, this will improve the co-simulation runtime performance.

- SystemC Shell Command to Export Structural HDL
- Possible Use-Model Scenarios
- Debugging and Simulation Control

2.15.1 SystemC Shell Command to Export Structural HDL

By default, exporting structural HDL is enabled. To disable it, the `set_hdl_structural_optimization` command is available.

It syntax looks as follows:

```
[::scsh::cosim::] set_hdl_structural_optimization [ on | off ]
```

For example:

```
scsh> ::scsh::cosim:: set_hdl_structural_optimization off
```

2.15.2 Possible Use-Model Scenarios

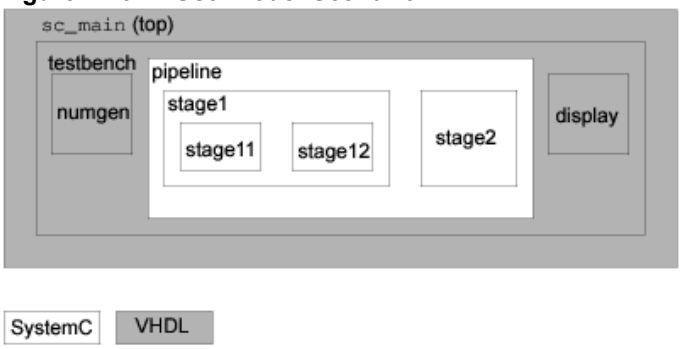
The following figure shows a pipeline example. This section describes the possible use-model scenarios

Figure 2-18 Example for Use-Model Scenarios

2.15.2.1 Scenario 1

The following figure shows scenario 1.

Figure 2-19 Use-Model Scenario 1



For this scenario, the elaborator can only export the `sc_hdl_module` into HDL because it cannot identify any possible optimization.

The hierarchy is the same with and without optimization.

- At the SystemC side, the hierarchy looks as follows:

```
sc _main/
      |__ testbench [sc_hdl_module]
      |__ testbench_v_pipeline [sc_module]
          |__ stage1 [sc_module]
              |__ stage11 [sc_module]
              |__ stage12 [sc_module]
          |__ stage2 [sc_module]
```

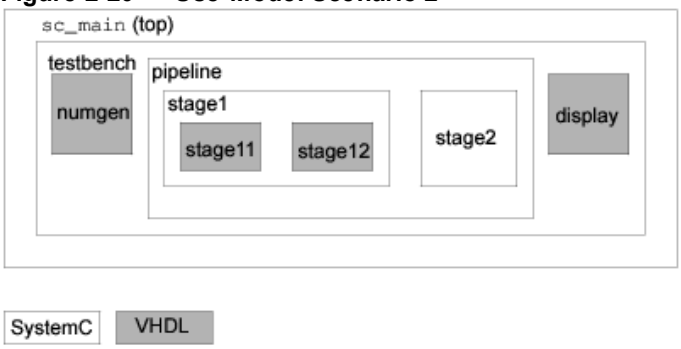
- At the HDL side, the hierarchy looks as follows:

```
HDLTop
     |__ testbench
         |__ numgen
         |__ display
         |__ pipeline [SystemC Proxy in HDL]
```

2.15.2.2 Scenario 2

The following figure shows scenario 2.

Figure 2-20 Use-Model Scenario 2



For this scenario, the elaborator can export the subhierarchy under `stage1` into HDL.

- Hierarchy before optimization:
  - At the SystemC side, the hierarchy looks as follows:

    ```
    sc_main/
         |__ testbench [sc_hdl_module]
             |__ pipeline [sc_module]
                  |__ stage1 [sc_module]
                       |__ stage11 [sc_hdl_module]
                       |__ stage12 [sc_hdl_module]
                  |__ stage2 [sc_module]
             |__ numgen [sc_hdl_module]
             |__ display [sc_hdl_module]
    ```

  - At the HDL side, the hierarchy looks as follows:

    ```
    HDLTop
         |__ numgen
         |__ display
         |__ stage11
         |__ stage12
    ```

- Hierarchy after optimization:
  - At the SystemC side, the hierarchy looks as follows:

    ```
    sc_main/
          |__ testbench [sc_hdl_module]
              |__ pipeline [sc_module]
                  |__ stage1 [sc_module]
                  |__ stage2 [sc_module]
              |__ numgen [sc_hdl_module]
              |__ display [sc_hdl_module]
    ```

  - At the HDL side, the hierarchy looks as follows:

    ```
    HDLTop
         |__ numgen
         |__ display
         |__ testbench_v_pipeline_v_stage1
              |__ stage11
              |__ stage12
    ```

2.15.2.3 Scenario 3

The following figure shows scenario 3.

Figure 2-21 Use-Model Scenario 3

For this scenario, the elaborator can export the subhierarchy under `stage1` into HDL, but it cannot include the module pipeline because `stage2` is Verilog instead of VHDL.

- Hierarchy before optimization:
  - At the SystemC side, the hierarchy looks as follows:

```
sc_main/
    |__ testbench [sc_hdl_module]
        |__ pipeline [sc_module]
            |__ stage1 [sc_module]
                 |__ stage11 [sc_hdl_module]
                 |__ stage12 [sc_hdl_module]
            |__ stage2 [sc_hdl_module]
        |__ numgen [sc_hdl_module]
        |__ display [sc_hdl_module]
```

  - At the HDL side, the hierarchy looks as follows:

```
HDLTop
     |__ numgen
     |__ display
     |__ stage11
     |__ stage12
     |__ stage2
```

- Hierarchy after optimization:
  - At the SystemC side, the hierarchy looks as follows:

```
sc_main/
    |__ testbench [sc_hdl_module]
        |__ pipeline [sc_module]
            |__ stage1 [sc_module]
            |__ stage2 [sc_hdl_module]
        |__ numgen [sc_hdl_module]
        |__ display [sc_hdl_module]
```

  - At the HDL side, the hierarchy looks as follows:

```
HDLTop
     |__ numgen
     |__ display
     |__ testbench_v_pipeline_v_stage1
           |__ stage11
           |__ stage12
     |__ testbench_v_pipeline_v_stage2
```

2.15.3 Debugging and Simulation Control

The simulation is started and run in the same manner as any normal SystemC simulation. Some variable settings specify the HDL simulator and the language used.

2.15.3.1 Structural Browsing

Structural browsing works in the same way as in SystemC Shell. The HDL modules in the SystemC hierarchy cannot be expanded further. So the HDL hierarchy cannot be traversed from the SystemC user interface. The available HDL browsing tool is used for browsing the HDL hierarchy. After the export, the exported structural HDL hierarchy does not show in the SystemC hierarchy browser; you have to use the HDL browser to traverse the hierarchy.

2.15.3.2 Structural Breakpoints

The structural breakpoints only work with the SystemC hierarchy. After the export, the exported structural HDL hierarchy is not part of the SystemC hierarchy. As a result, structural breakpoints do not work.

< > < >

3.16. 2.15 SystemC Shell Commands

This section describes the SystemC Shell commands related to SystemC-HDL co-simulation in detail.

- Overview

2.16.1 Overview

The following table summarizes the SystemC Shell commands related to SystemC-HDL co-simulation.

Table 2-3 SystemC Shell Commands for SystemC-HDL Co-Simulation

Command	Summary
cosim::map_hdl_library_name	Maps the HDL library name to a physical directory.
cosim::set_cosim_timeout	Sets the maximum time SystemC Shell waits for the HDL simulator to report back.
cosim::set_delta_cycle_sync	Sets the synchronization mode.
cosim::set_hdl_max_nesting	Sets the maximum number of levels allowed for nesting.
cosim::set_hdl_options	Sets options for the selected simulator.
cosim::set_hdl_sim_mode	Sets simulator mode to batch or interactive.
cosim::set_hdl_sim_type	Sets the HDL simulator.
cosim::set_verilog_timescale	Sets the time-scale unit for the generated Verilog code.
cosim::show_hdl_options	Shows simulation options specified for the selected HDL simulator.
get_hdl_lib_map	Returns the HDL library map.
hdl-options	Sets HDL co-simulation options.
map_hdl_library_name	Lists the HDL library names.
run-args	Specifies runtime arguments to the simulator.
set_cosim_timeout	Sets the maximum time SystemC Shell waits for the HDL simulator to report back.
set_delta_cycle_sync	Sets the synchronization mode.
set_hdl_max_nesting	Sets the maximum number of levels allowed for nesting.
set_hdl_options	Sets options for the selected simulator.
set_hdl_sim_mode	Sets simulator mode to batch or interactive.
set_hdl_sim_type	Sets the HDL simulator.

set_verilog_timescale	Sets the time-scale unit for the generated Verilog code.
show_hdl_options	Shows simulation options specified for the selected HDL simulator.
simargs	Provides runtime arguments to the simulator.

2.16.2 cosim::map_hdl_library_name

Maps an HDL logical library name to a directory path. For more information on mapping logical libraries, see "Mapping Logical Library Names to Physical Directories" on page 84.

Syntax

```
cosim::map_hdl_library_name name path
```

Arguments

name	Specifies the name of the HDL library.
path	Specifies the path name to the directory where the library resides.

Example

The following example maps an HDL library to the directory where the library resides:

```
cosim::map_hdl_library_name myLib /usr1/home/joe/work
```

2.16.3 cosim::set_cosim_timeout

Sets the maximum time SystemC Shell waits for the HDL simulator to report back. When the specified time is exceeded, SystemC Shell reports an error.

The time unit is seconds. For more information, see "Specifying Co-Simulation Time-Out" on page 80.

Syntax

```
cosim::set_cosim_timeout value
```

Arguments

value	Specifies the number of seconds SystemC Shell waits for the HDL simulator to report back.

Example

The following example illustrates that SystemC Shell expects to find the HDL simulator, and for the HDL simulator, to finish the simulation step in 20 minutes. If the task is not completed within this time, an error appears.

```
cosim::set_cosim_timeout 1200
```

2.16.4 cosim::set_delta_cycle_sync

Sets the synchronization mode between the SystemC Shell and the HDL simulators. For more information, see "Selecting the Synchronization Mode" on page 83.

Syntax

```
cosim::set_delta_cycle_sync value
```

Arguments

value	Can be one of the following: • `true` sets the delta-cycle-synchronization mode. • `false` the event-based synchronization mode.

2.16.5 cosim::set_hdl_max_nesting

Sets the number of language switches allowed for recursive mixed language nesting before loading the simulator. For example, the top level SystemC can contain HDL, which can contain SystemC, and so on. In this case, you must specify the maximum number of language nesting allowed. The top (always SystemC) is considered `1`; For more information, see "Rules for SystemC-HDL Co-Simulation" on page 17.

Syntax

```
cosim::set_hdl_max_nesting value
```

Arguments

value	Specifies the maximum number of language switches allowed for mixed-language nesting.

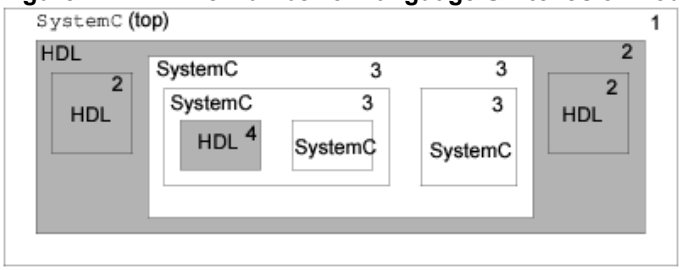
Example

The following command sets the maximum number of language switches allowed for mixed-language nesting to `4`.

```
cosim::set_hdl_max_nesting 4
```

In the following figure, the number indicates the number of language switches from the top to every module. The maximum number is 4.

Figure 2-22 The Number of Language Switches of Modules



2.16.6 cosim::set_hdl_options

Sets simulation options for the HDL simulator. You can set options for the different phases of the simulation process: compilation, elaboration, and runtime. For more information on setting options for the HDL simulators, see "Setting Simulation Options" on page 82.

Syntax

```
cosim::set_hdl_options simulator phase value [value]...
```

Arguments

simulator	Can be one of the following: • `ncsim` sets simulation options for the NC-Verilog, NC-VHDL, or NC-Sim simulator, depending on your installation. • `vcs` sets simulation options for the VCS simulator. • `vcsi` sets simulation options for the VCSi simulator. • `mti` sets simulation options for the ModelSim/Questa simulator.
phase	Can be one of the following: • `vlog_compile` sets simulation options for the Verilog compilation phase. • `vhdl_compile` sets simulation options for the VHDL compilation phase. • `elaborate` sets simulation options for the elaboration phase.

	• `runtime` sets simulation options for the runtime phase.
value	Is simulator specific. For more information on the options, see the documentation of your HDL simulator.

2.16.6.1 Examples

- The following command sets compilation options for a Verilog design simulated in VCS:

  ```
  cosim::set_hdl_options vcs vlog_compile -notice -l myVer.log
  ```

- The following command sets compilation options for a VHDL design simulated in NC-Sim:

  ```
  cosim::set_hdl_options ncsim vhdl_compile -mess -smartorder -status -list
                                                        -logfile myVhdl.log
  ```

- The following command sets elaboration options for a design simulated in NC-Sim:

  ```
  cosim::set_hdl_options vcs elaborate -notice
  ```

- The following command sets runtime options for a design simulated in VCS:

  ```
  cosim::set_hdl_options vcs runtime -notice -l sim.log
  ```

2.16.7 cosim::set_hdl_sim_mode

Sets the HDL simulation mode in GUI or command-line mode. If you do not specify a mode, the simulator starts in command-line mode. For more information, see "Selecting the Simulator Mode" on page 81.

Syntax

`cosim::set_hdl_sim_mode` *mode*

Arguments

mode	Can be one of the following: • `batch` runs the simulator from the command line. • `interactive` runs the simulator from the GUI.

2.16.8 cosim::set_hdl_sim_type

Sets the HDL simulator for SystemC-HDL co-simulation. For more information on selecting an HDL simulator, see "Selecting the HDL Simulator" on page 81.

Syntax

`cosim::set_hdl_sim_type` *simulator*

Arguments

simulator	Can be one of the following: • `ncsim` invokes the NC-Verilog, NC-VHDL, or NC-Sim simulator, depending on your installation. • `vcs` invokes the VCS simulator. • `vcsi` invokes the VCSI simulator. • `mti` invokes the ModelSim/Questa simulator.

Example

```
cosim::set_hdl_sim_type vcs
```

2.16.9 cosim::set_verilog_timescale

Sets the time-scale unit for the generated Verilog code.

Syntax

`cosim::set_verilog_timescale` *value*

Arguments

value	Can be one of the following: `fs`, `ps`, `ns`, `us`, `ms`, `sec`.

Example

```
cosim::set_verilog_timescale fs
```

2.16.10 cosim::show_hdl_options

Lists the options specified for the selected HDL simulator. If you do not specify a simulator, the options are shown for all HDL simulators. For more information, see "Setting Simulation Options" on page 82.

Syntax

```
cosim::show_hdl_options [simulator]
```

Arguments

simulator	Can be one of the following: • `ncsim` shows options specified for the NC-Verilog, NC-VHDL, or NC-Sim simulator, depending on your installation. • `vcs` shows options specified for the VCS simulator. • `vcsi` shows options specified for the VCSI simulator. • `mti` options specified for the ModelSim/Questa simulator.

2.16.11 get_hdl_lib_map

Returns the HDL library map.

Note This command is equivalent to `hdl-options -libmap` (without arguments).

Syntax

```
get_hdl_lib_map
```

Arguments

None.

Example

```
get_hdl_lib_map
```

2.16.12 hdl-options

Sets HDL co-simulation options.

Syntax

```
hdl-options -option optionArgs ...
```

Arguments

-option optionArgs

Can be one of the following. If you specify only one option and no value, the command returns the currently set value.

`-verilog-compile-options` *simulator optionList*	Sets Verilog compile options. This option is equivalent to `set_hdl_options`.

`-vhdl-compile-options` *`simulator optionList`*	Sets VHDL compile options. This option is equivalent to `set_hdl_options`.
`-run-options` *`simulator optionList`*	Sets runtime options. This option is equivalent to `set_hdl_options`.
`-elab-options` *`simulator optionList`*	Sets elaboration options. This option is equivalent to `set_hdl_options`.
`-timescale` *`value`*	Sets the time scale for Verilog. Possible values are: `fs`, `ps`, `ns`, `us`, `ms`, `sec`. The default is `fs`. This option is equivalent to `set_verilog_timescale`.
`-delta-cycle-sync` *`value`*	Sets the synchronization mode between SystemC Shell and the HDL simulators. For more information, see "Selecting the Synchronization Mode" on page 83. It can be `on` or `off`. The default is `on`. This option is equivalent to `set_delta_cycle_sync`.
`-max-nesting` *`level`*	Specifies the maximum nesting level. The default is `10`. This option is equivalent to `set_hdl_max_nesting`.
`-libmap` *`libName dirName`*	Specifies the library map. This option is equivalent to `map_hdl_library_name`. `hdl-options libmap` (without arguments) is equivalent to `get_hdl_lib_map`.
`-interactive` *`value`*	Sets the interactive mode. It can be `on` or `off`. The default is `off` (that is, batch mode). This option is equivalent to `set_hdl_sim_mode`.
`-simulator` *`value`*	Specifies the simulator. Possible values are: `ncsim`, `vcs`, `mti`. This option is equivalent to `set_hdl_sim_type`.
`-show`	Displays the settings. Issuing `hdl-options` without any argument is equivalent to the `-show` option. This option is equivalent to `show_hdl_options`.
`-help`	Provides command help.
`-uvmc_enabled` *`value`*	Enables the use of UVM-Connect. This option is equivalent to `set_uvmc_enabled`.

2.16.13 map_hdl_library_name

Maps an HDL logical library name to a directory path. For more information on mapping logical libraries, see "Mapping Logical Library Names to Physical Directories" on page 84.

Note This command is equivalent to `hdl-options -libmap`.

Syntax

```
map_hdl_library_name name path
```

Arguments

`name`	Specifies the name of the HDL library.
`path`	Specifies the path name to the directory where the library resides.

Example

The following example maps an HDL library to the directory where the library resides:

```
map_hdl_library_name myLib /usr1/home/joe/work
```

2.16.14 run-args

Specifies runtime arguments to the simulator.

Syntax

```
run-args time
```

Arguments

None.

2.16.15 set_cosim_timeout

Sets the maximum time SystemC Shell waits for the HDL simulator to report back. When the specified time is exceeded, SystemC Shell reports an error.

The time unit is seconds. For more information, see "Specifying Co-Simulation Time-Out" on page 80.

Syntax

```
set_cosim_timeout value
```

Arguments

value	Specifies the number of seconds SystemC Shell waits for the HDL simulator to report back.

Example

The following example illustrates that SystemC Shell expects to find the HDL simulator, and for the HDL simulator, to finish the simulation step in 20 minutes. If the task is not completed within this time, an error appears.

```
set_cosim_timeout 1200
```

2.16.16 set_delta_cycle_sync

Sets the synchronization mode between the SystemC Shell and the HDL simulators. For more information, see "Selecting the Synchronization Mode" on page 83.

Note This command is equivalent to `hdl-options -delta-cycle-sync`.

Syntax

```
set_delta_cycle_sync value
```

Arguments

value	Can be one of the following: • `true` sets the delta-cycle-synchronization mode. • `false` sets the event-based synchronization mode.

2.16.17 set_hdl_max_nesting

Sets the number of language switches allowed for recursive mixed language nesting before loading the simulator. For example, the top level SystemC can contain HDL, which can contain SystemC, and so on. In this case, you must specify the maximum number of language nesting allowed. The top (always SystemC) is considered 1; For more information, see "Rules for SystemC-HDL Co-Simulation" on page 17.

Note This command is equivalent to `hdl-options -max-nesting`.

Syntax

```
set_hdl_max_nesting value
```

Arguments

value	Specifies the maximum number of language switches allowed for mixed-language nesting.

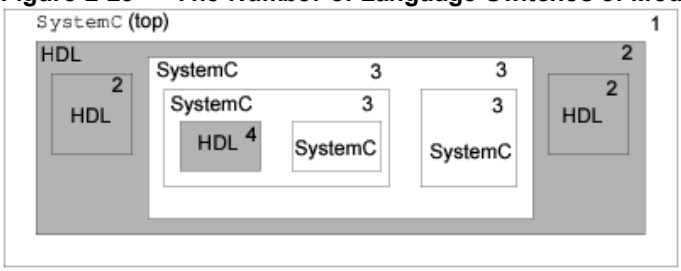
Example

The following command sets the maximum number of language switches allowed for mixed-language nesting to 4.

```
set_hdl_max_nesting 4
```

In the following figure, the number indicates the number of language switches from the top to every module. The maximum number is 4.

Figure 2-23 The Number of Language Switches of Modules



2.16.18 set_hdl_options

Sets simulation options for the HDL simulator. You can set options for the different phases of the simulation process: compilation, elaboration, and runtime. For more information on setting options for the HDL simulators, see "Setting Simulation Options" on page 82.

Note

This command is equivalent to the following commands:

```
set_hdl_options -verilog-compile-options
set_hdl_options -vhdl-compile-options
set_hdl_options -run-options
set_hdl_options -elab-options
```

Syntax

```
set_hdl_options simulator phase value [value]...
```

Arguments

simulator	Can be one of the following: • `ncsim` sets simulation options for the NC-Verilog, NC-VHDL, or NC-Sim simulator, depending on your installation. • `vcs` sets simulation options for the VCS simulator. • `vcsi` sets simulation options for the VCSI simulator. • `mti` sets simulation options for the ModelSim/Questa simulator.
phase	Can be one of the following: • `vlog_compile` sets simulation options for the Verilog compilation phase. • `vhdl_compile` sets simulation options for the VHDL compilation phase. • `elaborate` sets simulation options for the elaboration phase. • `runtime` sets simulation options for the runtime phase.
value	Is simulator specific. For more information on the options, see the documentation of your HDL simulator.

Examples

- The following command sets compilation options for a Verilog design simulated in VCS:

  ```
  set_hdl_options vcs vlog_compile -notice -l myVer.log
  ```

- The following command sets compilation options for a VHDL design simulated in NC-Sim:

  ```
  set_hdl_options ncsim vhdl_compile -mess -smartorder -status -list
                                                -logfile myVhdl.log
  ```

- The following command sets elaboration options for a design simulated in VCS:

  ```
  set_hdl_options vcs elaborate -notice
  ```

- The following command sets runtime options for a design simulated in VCS:

```
set_hdl_options vcs runtime -notice -l sim.log
```

2.16.19 set_hdl_sim_mode

Sets the HDL simulation mode in GUI or command-line mode. If you do not specify a mode, the simulator starts in command-line mode. For more information, see "Selecting the Simulator Mode" on page 81.

Note This command is similar to `hdl-options -interactive`.

Syntax

```
set_hdl_sim_mode mode
```

Arguments

mode	Can be one of the following: • `batch` runs the simulator from the command line. • `interactive` runs the simulator from the GUI.

2.16.20 set_hdl_sim_type

Sets the HDL simulator for SystemC-HDL co-simulation. For more information on selecting an HDL simulator, see "Selecting the HDL Simulator" on page 81.

Note This command is similar to `hdl-options -simulator`.

Syntax

```
set_hdl_sim_type simulator
```

Arguments

simulator	Can be one of the following: • `ncsim` invokes the NC-Verilog, NC-VHDL, or NC-Sim simulator, depending on your installation. • `vcs` invokes the VCS simulator. • `vcsi` invokes the VCSI simulator. • `mti` invokes the ModelSim/Questa simulator.

Example

```
set_hdl_sim_type n2sim
```

2.16.21 set_verilog_timescale

Sets the time-scale unit for the generated Verilog code.

Note This command is similar to `hdl-options -timescale`.

Syntax

```
set_verilog_timescale value
```

Arguments

value	Can be one of the following: `fs`, `ps`, `ns`, `us`, `ms`, `sec`.

Example

```
set_verilog_timescale fs
```

2.16.22 show_hdl_options

Lists the options specified for the selected HDL simulator. If you do not specify a simulator, the options are shown for all HDL simulators. For more information, see "Setting Simulation Options" on page 82.

Note This command is similar to `hdl-options -show`.

Syntax

```
show_hdl_options [simulator]
```

Arguments

simulator	Can be one of the following: • `ncsim` shows options specified for the NC-Verilog, NC-VHDL, or NC-Sim simulator, depending on your installation. • `vcs` shows options specified for the VCS simulator. • `vcsi` shows options specified for the VCSI simulator. • `mti` shows options specified for the ModelSim/Questa simulator.

2.16.23 simargs

Provides runtime arguments to the simulator.

Syntax

```
simargs argument [argument]...
```

Arguments

argument	Specifies the argument you want to pass to the SystemC simulator.

Example

```
simargs x y
```

2.16.24 set_uvmc_enabled

Enables your design to make use of UVM-Connect. Once set, SystemC and System Verilog modules can use the UVM-Connect library.

Syntax

```
set_uvmc_enabled value
```

Arguments

value	Can be `true` or `false`.

< > < >

3.17. 2.17 Profiling a SystemC-HDL Co-Simulation

Unlike a standard simulation, a co-simulation cannot be profiled with `gprof`. For more information, see "Using gcov for Coverage Analysis" in the *SystemC Shell Manual*. The reason is that a co-simulation is linked into the HDL simulator as a shared object. Instead of `gprof`, you can use `sprof`, which is installed on most Linux systems.

Profiling with `sprof` is controlled by the following environment variables:

- `LD_PROFILE`: Must contain the name of the shared object that you want to profile.
- `LD_PROFILE_OUTPUT`: Should point to a directory where the output should go.

To successfully profile a co-simulation, the following rules apply for using `sprof`:

- `LD_PROFILE` must contain the name of the shared object in exactly the way that is used to open it. For example, if you have a co-simulation named `sim` that is built with the `Debug` configuration, then the shared object will be `Debug/sim.so`. Setting `LD_PROFILE` to `Debug/sim.so` before executing the simulation will tell `sprof` to collect the profiling data.
- `sprof` writes the output to `Debug/sim.so.profile`, located in the directory specified by `LD_PROFILE_OUTPUT`. However, `sprof` does not create any intermediate directories. In the above example, the profiling output will be in `$LD_PROFILE_OUTPUT/Debug/sim.so.profile`.

  If the `$LD_PROFILE_OUTPUT/Debug` directory does not exist, `sprof` does not write any data. Therefore, set `LD_PROFILE_OUTPUT` to the simulation directory since it already contains the necessary subdirectories.

After a simulation run, use the `sprof` command to translate the collected profile data into a human-readable form. The call in the above example will look like:

```
sprof Debug/sim.so $LD_PROFILE_OUTPUT/Debug/sim.so.profile
```

This results in mangled C++ symbols in the profiling output. You can unmangle them by using the program `c++filt`, which is installed on any Linux host:

```
sprof Debug/sim.so $LD_PROFILE_OUTPUT/Debug/sim.so.profile | c++filt
```

< > < >

3.18. 2.18 Limitations and Known Problems

This section describes:

- Limitations
- Known Problems

2.18.1 Limitations

- SystemC and HDL debugging environments are separate. There is no support for unified breakpoint/debugging capabilities. When a SystemC breakpoint is reached, the control will be in the SystemC simulator, and when the HDL breakpoint is reached, the control will be in the HDL simulator.
- The simulation environment always starts from the Synopsys SystemC user interface (`scsh`). If the design top is HDL, a top-level HDL proxy module (SystemC module) is needed to connect to the HDL. This can be hand written, or generated using `hdl_proxy_gen`, or for simple cases, it can be generated using `top_sc_gen`.
- You cannot instrument signals passing the SystemC-HDL co-simulation interface. Hence, analysis is only possible for SystemC signals.
- The SystemC environment supports a CLI mode and a script batch mode. The HDL co-simulation runs in either of these modes. A GUI is not available to invoke SystemC-HDL co-simulation.
- Graphical insertion of the HDL model instances in the SystemC hierarchy is not available.
- In one simulation session, only one vendor HDL simulator is used to co-simulate with Synopsys's SystemC simulator. This HDL simulator may support mixed language simulation, in which case the HDL models instantiated in the SystemC hierarchy could be either Verilog or VHDL models. But if the simulator supports only VHDL or Verilog, then all the HDL models instantiated should be of the language supported by the simulator.
- NC-Sim internally works with a time resolution of 1 fs. If you are using VHDL, then that time resolution is being used. There is no way to make it coarser. In case of Verilog, the models can use the time-scale facility to use a multiple of the basic kernel time resolution. In all the supported HDL simulators, the SystemC HDL co-simulation system will internally query the HDL time resolution and try to verify the time resolution relation constraints specified in "Selecting the Synchronization Mode" on page 83. A mixed language simulation on the HDL side with NC-Sim will thus force the HDL simulator time resolution to 1 fs because of the presence of the VHDL part of the design.
- For ModelSim/Questa, the following limitations must be taken into account:
  - For DO file support, `-do` has to be the first option in the list of runtime options.
  - Delta-cycle synchronization does not work in the mixed-language mode.

2.18.2 Known Problems

- When using Verilog source files specified with and without time scale together in one design, `ncelab` reports the following error:

  ```
  ncelab: *F,CUMSTS: Timescale directive missing on one or more modules.

  nchelp: 05.30-s011: (c) Copyright 1995-2005 Cadence Design Systems, Inc.
  ncelab/CUMSTS =
  ```

If any module has been compiled with a time-scale directive, the elaborator requires that all modules be compiled with a time-scale directive. Use the `-messages` flag to have the time scales for each module printed. Use the `-TIMESCALE` option to provide a time-scale directive for all modules that do not have one.

The solution is to provide a default time scale in the command-line option for the elaborator:

- The following `ncelab` command-line option is available:

  `-timescale` *`args`*

  For example:

  `-timescale 1ps/1ps`

- The following SystemC Shell command-line options are available:

  `options -elab-options ncsim "-timescale` *`args`*`"`

  or

  `cosim::set_hdl_options ncsim elaborate "-timescale` *`args`*`"`

  For example:

  `hdl-options -elab-options ncsim "-timescale 1ps/1ps"`

  or:

  `cosim::set_hdl_options ncsim elaborate "-timescale 1ps/1ps"`

< > < >

3.19. 2.19 Performance Optimization with VCS Co-Simulation

SystemC Shell allows you to pass arbitrary options to VCS and to the `simv` executable. These options can be specified with the following commands

- `hdl-options -verilog-compile-options`
- `hdl-options -elab-options`
- `hdl-options -run-options`

For large co-simulations, it usually pays off to specify some optimization options when compiling, elaborating and running a co-simulation. The following options are recommended when you want to improve performance of your VCS co-simulation:

- `hdl-options -verilog-compile-options vcsi +rad`
- `hdl-options -run-options vcsi +vcs+learn+pli+pli_learn.tab`
- `hdl-options -elab-options vcsi +applylearn+pli_learn.tab`

With these options,

1. Enable radiant technologies (`+rad`).
2. Analyze the PLI interface to find those parts of the design are accessed using PLI interface. Other parts of the design can be optimized by VCS (`+vcs+learn+pli+pli_learn.tab`).
3. Pass the PLI data learned from the last run to VCS during elaboration (`+applylearn+pli_learn.tab`).

For a detailed description of these options, see the *VCS Documentation*.

< > < >

3.20. 2.20 Inout Network Resolution Mechanism

The *default resolution* mechanism does not work when there are connections originating from HDL `inout` port, passing through SystemC (using `sc_signal_resolved` or `sc_signal_rv`) and again ending at another HDL `inout` port. This is the *HDL-SystemC-HDL* (HSH) topology.

This HSH topology, or a topology that contains HSH would show wrong and inconsistent results along different parts of the topology (on either side of the language boundary) when used with the default resolution mechanism.

The *Inout Network Resolution* mechanism supports HSH topology and the topologies that include HSH in them.

This section includes two examples of topologies using HSH in them.

- A Topology Consisting Only HSH
- A Topology Containing HSH

Figure 2-24 A Topology Consisting Only HSH

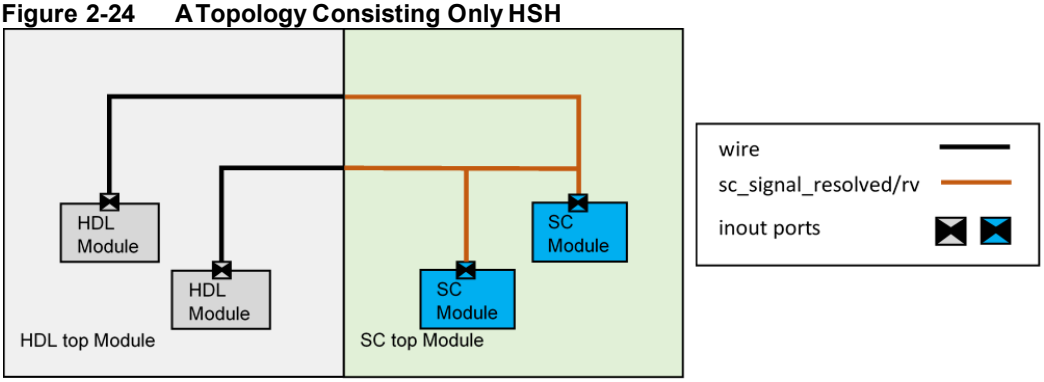
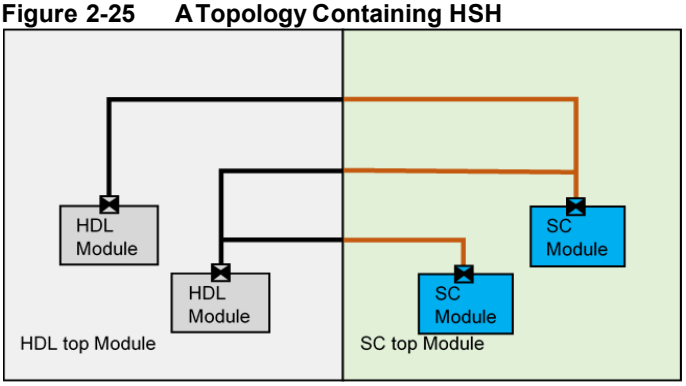


Figure 2-25 A Topology Containing HSH



2.20.1 Usage

2.20.1.1 Virtualizer-Centric Flow

- Virtualizer Studio

  Use the `CoSimulationInterface` block to configure co-simulation. In the Configuration tab of the block, under *CoSimulation Type*, setting the option *Inout Network Resolution* to `true` activates the mechanism.

- Platform Creator/Platform Architect

  Under the *Simulation* menu, select *Project Settings* (or press *F8*), go to *HDL/Co-Sim Settings* and select the *Inout Network Resolution* check box.

2.20.1.2 VCS-Centric Flow:

Do the following steps to activate *Inout Network Resolution*:

1. During the VCS elaboration step, enable the global callback access capability by replacing the PLI table file `cwrvcspli.tab` (as mentioned in "Building the HDL Design" on page 116) with `cwrvcspli-global-cbk.tab`, this should be used as a command-line argument to the `vcs` executable as:
   `-P ${SNPS_VP_HOME}/common/etc/hdlcosim/vcs/cwrvcspli-global-cbk.tab`
2. In order to activate the *Inout Network Resolution* mechanism, an additional argument `--inout_net_resolution` needs to be passed to the `cosim_helper`. For more information about `cosim_helper`, see "Running the Simulation" on page 116).

Note

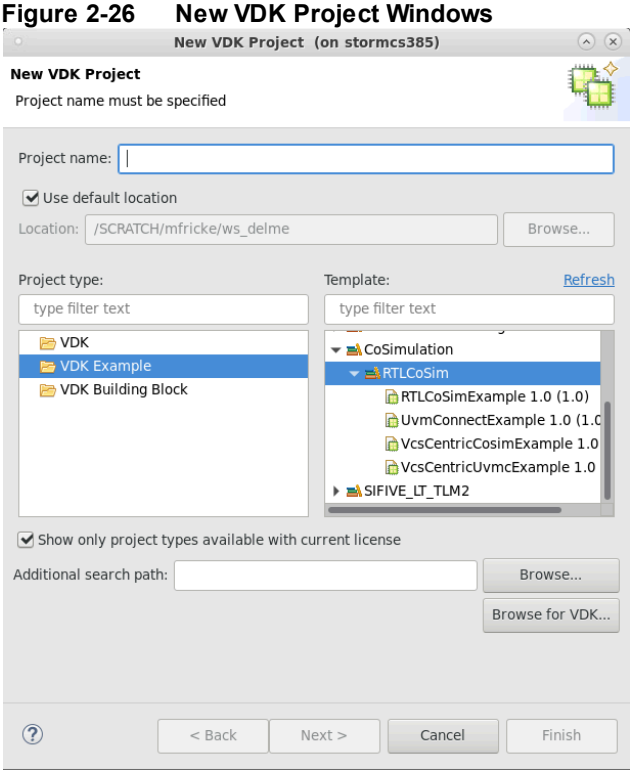
- The *Inout Network Resolution* mechanism is not enabled by default because, it may have an impact on the simulation speed.
- When using *Inout Network Resolution*, the below conditions must be met for the inout ports of SystemC wrapper/connector module:
  - Datatype of inout ports must be `sc_logic` or `sc_lv`.
  - The inout ports must only be bound to `sc_signal_resolved` or `sc_signal_rv`.

< > < >

3.21. 2.26 Example VDK in Virtualizer Studio

HDL CoSimulation is supported in Virtualizer Studio using the *CoSimulationInterface* configurable building block. An example VDK is available from the GUI, using the *File* > *New* > *VDK Project* dialog, as shown below.

Figure 2-26 New VDK Project Windows



Once the example VDK has been instantiated, its documentation can be accessed from the GUI, using the *Documentation* tab of the VDK.

4. Chapter 3
VCS-Centric HDL Co-Simulation

The VCS-centric HDL Co-Simulation presents an alternative to the approach described in "HDL CoSimulation" on page 15. Specifically, it is tailored towards RTL verification engineers, who are aiming to integrate a Virtualizer VDK subsystem authored with Virtualizer Studio into an existing, typically complex, HDL platform or testbench. Typical use cases are acceleration of a verification scenarios by integration of an LT processor model into an HDL platform, or extension of an UVM testbench by real-software scenarios running on an LT processor model.

Note	The VCS-centric HDL Co-Simulation is only supported for use with Virtualizer VDKs authored within Virtualizer Studio. This flow is currently not supported for use with Platform Creator or Platform Architect.

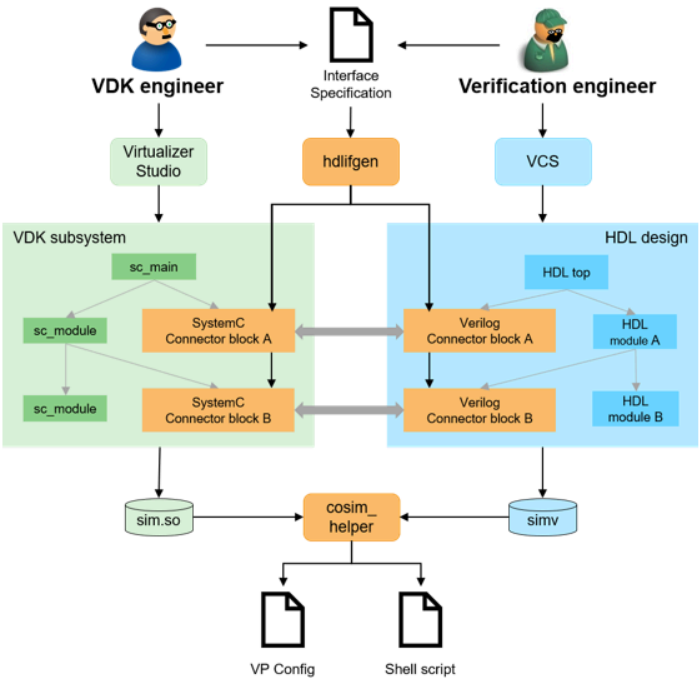
The chapter describes:

- Overview of the VCS-Centric HDL Co-Simulation flow
- Defining the Cross-Language Interface
- Building the Simulation
- Running the Simulation
- Example VDK in Virtualizer Studio
- Limitations

4.1. 3.1 Overview of the VCS-Centric HDL Co-Simulation flow

The general concept of the VCS-centric HDL Co-Simulation flow is to allow the VDK engineer and RTL verification engineer to author independent design hierarchies, each with their own set of tools, and to facilitate Co-Simulation through an interface specification they agree upon. The interface specification is used to generate a SystemC connector block and its Verilog counterpart, which are instantiated anywhere in the two design hierarchies to facilitate cross-language communication.

Figure 3-1 VCS-Centric Co-Simulation Flow Overview



The flow of authoring, building and running the simulation is displayed in Figure 3-1 and generally consists of the following steps, which are discussed in detail throughout the rest of this chapter:

1. The VDK and/or verification engineers define the signal-level interface between the SystemC and Verilog subsystems. The SystemC and Verilog connector block modules are created during this step.
2. Authoring of the two subsystems happens independently.
   a. The VDK subsystem is created in Virtualizer Studio, using the `CoSimulationInterface` configurable building block to configure the Co-Simulation.
   b. The HDL subsystem is created with VCS, typically incorporated into build systems such as `Make`, where a few additional command line flags are required for Co-Simulation.
3. Once both subsystems have been built, the simulation can be launched either through a VP Config or directly from the shell.
4. The simulation can be controlled, debugged or analyzed with the Virtualizer and/or VCS tool set.

Virtualizer Studio supports authoring of the VDK subsystem for the VCS-centric HDL Co-Simulation through the `CoSimulationInterface` configurable building block. For more details see, see the *CoSimulationInterface Configurable Building Block* manual, that is available at `IP/CoSimulation/docs/CoSimulationInterface/IP_CoSimulation.pdf`.

< > < >

4.2. 3.2 Defining the Cross-Language Interface

The cross-language interface can either be defined directly in Virtualizer Studio, or by manually writing an interface specification file.

3.2.1 Definition in Virtualizer Studio

To define the cross-language interface directly from Virtualizer Studio, the `CoSimulationInterface` is used. For more details, see the *CoSimulationInterface Configurable Building Block* manual, that is available at `IP/CoSimulation/docs/CoSimulationInterface/IP_CoSimulation.pdf`.

3.2.2 Manual Definition

Interfaces can also be defined without the use of Virtualizer, by writing one or more interface specification files, and using a stand-alone utility (`hdlifgen`) to generate the connector block source code as well as a library file that can be used within Virtualizer Studio as well as Platform Creator.

3.2.2.1 Writing Interface Specification Files

Interface specification files describe the set of connector blocks and signals crossing the language boundary. Each interface specification file can contain one or more connector block definitions, and each connector block definition can either specify a custom set of signals or a pre-configured protocol.

Example 1: Connector block definition with a custom set of signals

```
connector_block my_connector_block
#      <name>      <direction>      <width>      <data_type>      <category>
port    IRQ1          input           1             bool           interrupt
port    IRQ2          input           1             bool           interrupt
port    CLK           input           1             bool           clock
port    RESET         output          1             bool           reset
port    USR1          output          *             int            control
port    USR2          inout           28             *             control
port    DATA          inout           32            sc_lv           *
```

The definition begins with the `connector_block` keyword and a name, followed by one or more lines specifying the ports of the connector block. Each port definition begins with the `port` keyword, followed by its name, direction, width, data type and category, in that order.

The direction is specified from the perspective of SystemC, that is, input specifies a signal driven by Verilog and propagated to SystemC. The width field specifies the bit width of the signal. The data type defines SystemC/C++ data type that the generated SystemC connector block will use. Supported data types are listed in Table 3-1. Finally, the category defines the port category on the SystemC side. Possible values are control, reset, clock and interrupt, and the value defines the connectivity pane in Platform Creator/Virtualizer Studio. The category has no effect on the Verilog connector block.

It is possible to omit certain fields by specifying *, and the tool will use the default data type (`sc_int`) or category (`control`) or try to guess the port width based on the data type.

If invalid combinations of fields are used, the tool will print an error message.

Table 3-1 Supported data types

Keyword	SystemC/C++ Data Type	Verilog Data Type	Width *W*
bool	bool	scalar	1
char	char	wire [W-1:0]	8
uchar	unsigned char	wire [W-1:0]	8
schar	signed char	wire [W-1:0]	8
short	short	wire [W-1:0]	16
ushort	unsigned short	wire [W-1:0]	16
int	int	wire [W-1:0]	32
uint	unsigned int	wire [W-1:0]	32
long	long	wire [W-1:0]	64
ulong	unsigned long	wire [W-1:0]	64
longlong	long long	wire [W-1:0]	64
ulonglong	unsigned long long	wire [W-1:0]	64

float	float	wire [W-1:0]	32
double	double	wire [W-1:0]	64
sc_int	sc_int	wire [W-1:0]	0 < W <= 64
sc_uint	sc_uint	wire [W-1:0]	0 < W <= 64
sc_bigint	sc_bigint	wire [W-1:0]	0 < W
sc_biguint	sc_biguint	wire [W-1:0]	0 < W
sc_bit	sc_bit	scalar	1
sc_bv	sc_bv	wire [W-1:0]	0 < W
sc_logic	sc_logic	scalar	1
sc_lv	sc_lv	wire [W-1:0]	0 < W

Example 2: Connector block definition for pre-configured protocol

```
connector_block my_axi_initiator_block
protocol      axi_initiator
addr_width     64
data_width     32
id_width       8
user_width     4
```

The definition begins with the `connector_block` keyword and a name, immediately followed by a line specifying the protocol using the `protocol` keyword, as well as optional width specifiers.

Supported width parameters are `addr_width`, `data_width`, `id_width` and `user_width`, and they define the widths of the AXI address signals, AXI data signals, AXI transaction identifier signals and AXI user signals respectively. If a protocol does not support AXI transaction identifier or AXI user signals, specifying a width for them will result in an error.

Note

- Optional AXI transaction identifier signals are always included in pre-defined connector blocks. If these are not desired, a custom interface can be created.
- Assigning widths to optional AXI user signals on a per port basis is not supported. Only one width can be configured that is used for all user signals. If individual widths are required, a custom interface can be created.

Table 3-2 Available Protocols for Pre-Configured Connector Blocks

Keyword	Platform Creator protocol	Orientation
axi4_initiator	AXI4Initiator	Verilog initiator -> SystemC target
axi4_target	AXI4Target	SystemC initiator -> Verilog target
axi_initiator	AXIInitiator	Verilog initiator -> SystemC target
axi_target	AXITarget	SystemC initiator -> Verilog target
ahblite3_initiator	AHBLite3Initiator	Verilog initiator -> SystemC target
ahblite3_target	AHBLite3Target	SystemC initiator -> Verilog target
ahblite_initiator	AHBLiteInitiator	Verilog initiator -> SystemC target
ahblite_target	AHBLiteTarget	SystemC initiator -> Verilog target

apb4_target	APB4Target	SystemC initiator -> Verilog target
apb3_target	APB3Target	SystemC initiator -> Verilog target
apb_target	APBTarget	SystemC initiator -> Verilog target

3.2.2.2 Generating and Instantiating the Connector Blocks

Once the interface specification file has been created, it can be used to generate the SystemC and Verilog connector blocks as well as a library file to be used in Virtualizer Studio or Platform Creator. The `hdlifgen` utility is used for that purpose.

Usage:

```
hdlifgen [-h | --help]
   [--sysc [--import]]
   [-o <output_dir]
   <if_spec>
```

-h, --help	Print this help and exit.
--sysc	Generate SystemC module and Platform Creator import script.
--import	Import SystemC module using Platform Creator. Requires `--sysc` to also be specified.
-o <output_dir>	Specify output directory. Default is `hdlifgen`.
<if_spec>	Path to interface specification.

4.3. 3.3 Building the Simulation

The SystemC and HDL subsystems are compiled separately and can be modified independently, not requiring recompilation of the other side if the interface between the two sides has not changed.

3.3.1 Building the VDK Subsystem

The VDK subsystem is built with Virtualizer Studio. After instantiating, configuring and generating the `CoSimulationInterface`, simply hit the *Build* button.

Note	Building the VDK will result in a shared object containing the SystemC side of the simulation. It is usually located in bin/export/Release/sim.so in your Virtualizer Studio project, and is an essential component to launch the Co-Simulation.

3.3.2 Building the HDL Design

Generally, the HDL side is built just as any other HDL design. The Verilog connector blocks, which are generated during interface specification, must be instantiated and connected. This can happen anywhere in the design hierarchy. During the VCS elaboration step, a few additional command line arguments are required:

- `+vpi`

  Enables the Verilog Procedural Interface (VPI).

- `-P ${SNPS_VP_HOME}/common/etc/hdlcosim/vcs/cwrvcspli.tab`

  Passes the Co-Simulation-specific PLI table file.

- ${`SNPS_VP_HOME}/common/libso-${COWARE_CXX_COMPILER}/libCwrVcs.so`

  Shared object containing Co-Simulation infrastructure.

4.4. 3.4 Running the Simulation

There are two options to launch the simulation:

- Using a VP Config which is launched from the Virtualizer Studio GUI or shell (`vssh`).
- Standalone launch from the command line.

Note	The standalone launch does not support VP scripting. If the SystemC subsystem depends on scripting, a VP Config must be used.

Irrespective of how the simulation should be launched, an additional step is required, preparing the launch depending on several aspects, including:

- Location of the simulation executable generated by VCS (`simv`).
- HDL command line arguments.
- Location of the simulation shared object generated by Virtualizer (`sim.so`).
- SystemC command line arguments.
- The working directory that the simulation should run in.

To automate this step, a utility tool `cosim_helper` is provided. It is placed in the build directory of your Virtualizer Studio workspace (typically under `<project>/bin/export/Release/cosim_helper`) when the SystemC subsystem is built.

Note	It must be run from that directory, as it depends on certain files being present relative to that location.

3.4.1 Launching Using VP Config

To prepare a VP Config launch, a new, Co-Simulation-specific VP Config is created based on an existing one. The new VP Config takes on the same name as the existing one, adding the postfix `_cosim`. It is identical to the existing one except that the executable is changed to a generated script, which acts as a wrapper around `simv`.

The VP Config can be generated using the `cosim_helper` utility as follows:

```
<build_directory>/cosim_helper -V <vpconfig_name> gen_vpconfig <simv_path> <simv_arguments>
```

The SystemC command line arguments as well as the working directory of the simulation are configured in the VP Config.

Note	When the HDL subsystem depends on files using paths relative to the current working directory, those files must be made accessible from the working directory configured in the VP Config, for example by creating symbolic links to those files in that directory.

Once the VP Config has been created, it can be used to launch the Co-Simulation. This can be done either using the Virtualizer Studio GUI, or the shell variant `vssh` as follows:

```
vssh -r <path_to_vpconfig>
```

3.4.2 Standalone launch

Unless the simulation depends on VP scripting infrastructure to function correctly, it can be launched from the shell directly without Virtualizer Studio being involved. The `cosim_helper` utility can either generate a shell script wrapper around `simv` that can be used to launch it, or launch it directly.

In the simplest case, the launch script can be generated as follows:

```
<build_directory>/cosim_helper -o <output_file> gen_run_script <simv_path> <simv_arguments>
```

Alternatively, the simulation can be launched directly using:

```
<build_directory>/cosim_helper run <simv_path> <simv_arguments>
```

3.4.2.1 Controlling the working directory of the simulation

The working directory defaults to the current working directory when `cosim_helper` is being run. To specify a different directory, pass the `-d <path>` argument.

When the SystemC and/or HDL subsystems depend on files using relative paths, all files must be accessible from the working directory. When the two sides depend on paths relative to their respective, different build directories, it is advised to create symbolic links to the required files in a common location that will be used as working directory.

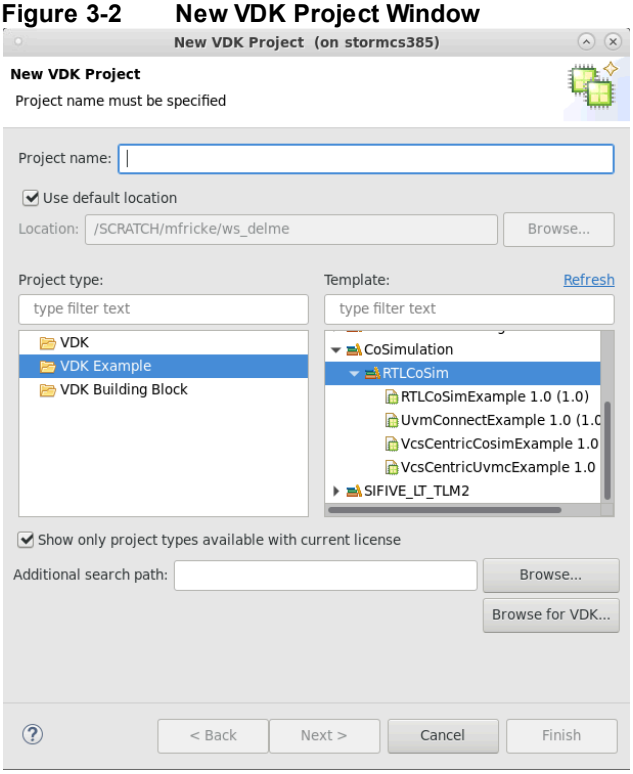
3.4.2.2 Specifying SystemC command line arguments

Command line arguments for the SystemC side must be written to a file that is being passed to `cosim_helper` using the `--sysc_args <path_to_file>` argument. The file must contain exactly one argument per line.

4.5. 3.5 Example VDK in Virtualizer Studio

VCS-centric HDL Co-Simulation is supported in Virtualizer Studio using the *CoSimulationInterface* configurable building block. An example VDK is available from the GUI, using the *File* > *New* > *VDK Project* dialog, as shown below.

Figure 3-2 New VDK Project Window



Once the example VDK has been instantiated, its documentation can be accessed from the GUI, using the *Documentation* tab of the VDK.

4.6. 3.6 Limitations

The section lists the limitations that have been taken into account for VCS-centric HDL Co-Simulation.

- The flow only supports Co-Simulation with the VCS simulator.
- Direct Co-Simulation with the VHDL language is not supported, though it is possible to involve VHDL indirectly using two hops, SystemC (Virtualizer) - Verilog (VCS) - VHDL (VCS).

5. Chapter 4 UVM-Connect

UVM-Connect (UVMC) is an open source library developed by Mentor. It enables transaction-level communication between SystemC models and System Verilog testbenches and provides a UVM Command API that allows controlling a UVM simulation from SystemC.

Virtualizer supports UVMC for Co-Simulation designs with the VCS and Mentor QuestaSim simulators, enabling convenient TLM connections between a VDK subsystem and UVM testbenches. Its usage is supported for both the Platform Architect/Virtualizer-

centric as described in "HDL CoSimulation" on page 15 and the VCS-centric Co-Simulation flows is described in "VCS-Centric HDL Co-Simulation" on page 111.

Typical use cases are an extension of an UVM testbench by real-software scenarios running on an LT processor model (VCS-centric flow), or re-use of a UVM testbench for verification of a TLM2 model (Virtualizer-centric flow).

This chapter describes:

- Creating a UVMC Connection in Virtualizer Studio
- Creating a UVMC Connection in SystemVerilog
- Enabling UVMC for a VCS-Centric Co-Simulation
- Example VDK in Virtualizer Studio
- Limitations

< > < >

5.1. 4.1 Creating a UVMC Connection in Virtualizer Studio

UVMC is supported in Virtualizer Studio using the `CoSimulationInterface` configurable building block. For more details, see the *CoSimulationInterface Configurable Building Block* manual, that is available at `IP/CoSimulation/docs/CoSimulationInterface/IP_CoSimulation.pdf`.

< > < >

5.2. 4.2 Creating a UVMC Connection in SystemVerilog

To connect a UVM TLM socket to its SystemC counterpart, the UVMC SystemVerilog API is used, using the same UVMC identifier as on the SystemC side, for example:

```
uvmc_tlm #()::connect(myInitiatorSocket, "timerCtrl");
```

For more information on the UVMC API and its usage, refer to the documentation provided by Mentor.

< > < >

5.3. 4.3 Enabling UVMC for a VCS-Centric Co-Simulation

To enable UVMC for the VCS-centric approach to Co-Simulation (see "VCS-Centric HDL Co-Simulation" on page 111), the UVMC System Verilog package must be compiled into the design. It is essential to use the UVMC source code that is shipped with Virtualizer. Add the following command line options to your VCS build:

- `+incdir+${SNPS_VP_HOME}/common/etc/hdlcosim/uvmc-2.3.1/src/connect/sv`

  UVMC include directory.

- `${SNPS_VP_HOME}/common/etc/hdlcosim/uvmc-2.3.1/src/connect/sv/uvmc_pkg.sv`

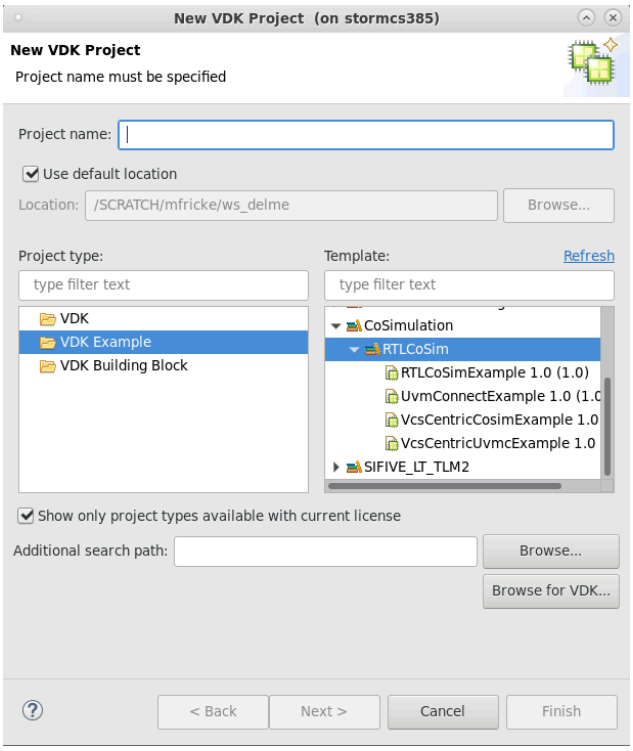
  Path of the UVMC SystemVerilog package.

< > < >

5.4. 4.4 Example VDK in Virtualizer Studio

UVM-Connect is supported in Virtualizer Studio using the *CoSimulationInterface* configurable building block. Two example VDKs (one for the Virtualizer-centric/Platform Architect-centric flow, and one for the VCS-centric flow) are available from the GUI, using the *File > New > VDK Project* dialog, as shown below.

Figure 4-1 New VDK Project Window



Once the example VDK has been instantiated, its documentation can be accessed from the GUI, using the *Documentation* tab of the VDK.

5.5. 4.5 Limitations

The section lists the limitations that have been taken into account for UVM connect.

- UVMC is only supported for the VCS and Mentor QuestaSim simulators. Cadence NCSim is not supported.
- Only TLM2 connections using the `tlm_generic_payload` transaction type are supported.

6. Chapter 5 Turbo Mode

In Co-Simulation designs, the RTL portion of the design typically runs much slower than the TLM parts, presenting the limiting factor for the overall simulation performance. This can have particularly drastic effects in cases where the SystemC subsystem must reach a certain point in simulated time before meaningful progress on the RTL side can be made. For example, a verification engineer may wish to utilize a fast, instruction accurate processor model to generate stimuli for their DUT based on real software (*SW-driven verification*). Often, the software must reach a certain point, such as a Linux driver being loaded, before stimuli are being generated. This can take a significant amount of simulated time, during which the RTL is mostly idle. Still, simulating the RTL for that amount of time can take hours or even days.

The purpose of the turbo mode is to improve performance in these situations by decoupling the SystemC and HDL portions, letting SystemC run ahead in simulated time and skipping the costly simulation of the idling RTL. The turbo mode dynamically detects when there is activity at the language boundary (either signals changing values or TLM transactions taking place) and forces the SystemC and HDL simulators to run synchronously during these phases.

This chapter describes:

- Configuration Parameters
- Support for Clocked Designs
- Enabling and Configuring the Turbo Mode for Co-Simulation

- Limitations

6.1. 5.1 Configuration Parameters

Operation of the turbo mode is controlled using several parameters that control when the TLM can run ahead of the RTL, and when the simulation falls back to a synchronous mode.

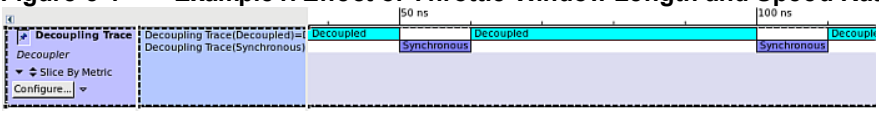
5.1.1 SystemC/HDL Speed Ratio and Throttle Window Length

These two parameters together configure the speed limiter for the turbo mode. Its purpose is to ensure that SystemC does not run ahead in simulated time indefinitely, which would lead to starvation on the HDL side.

The `throttle window length` defines the length of the window in simulated time during which the speed limiter enforces SystemC and HDL to run synchronously when it kicks in.

The SystemC/HDL `speed ratio` defines how far SystemC may run ahead at once, as a factor of the `throttle window length`.

Figure 5-1 Example1: Effect of Throttle Window Length and Speed Ratio



Assume the two parameters being configured as follows:

- `Throttle window length`: 10 nanoseconds
- `Speed ratio`: 5

When there is no boundary activity, SystemC can simulate five times as much time as HDL during each 50 nanoseconds (= `speed ratio` * `throttle window length`) window. After the two sides run synchronously for 10 nanoseconds (= `throttle window length`), SystemC is allowed to run ahead for 40 nanoseconds before the speed limiter kicks in and enforces another 10 nanoseconds throttle window of synchronous simulation. This behavior is also displayed in Figure 5-1.

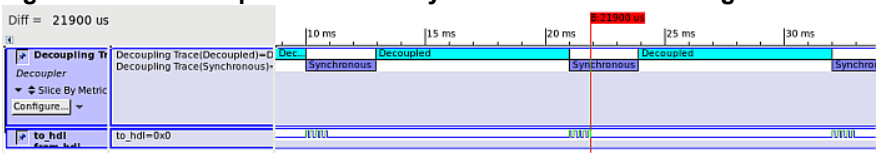
Note	The displayed time is the SystemC simulation, which is not equal to the HDL simulation time. In phases marked *Synchronous* both SystemC and HDL are simulating, while in phases marked *Decoupled*, only the SystemC side advances in time.

5.1.2 Synchronous Window Length

This parameter controls the behavior of the turbo mode when boundary activity is detected. Whenever signal-level or TLM-level activity is detected, the turbo mode forces the two simulators to run synchronously for at least the `synchronous window length`.

Note	The `synchronous window length` should be chosen to at least cover a significant number of clock cycles, so that the decoupling is not re-enabled in the middle of ongoing bus transactions.

Figure 5-2 Example2: Effect of Synchronous Window Length



Assume the `synchronous window length` is set to `2` milliseconds. That means whenever boundary activity is detected, the two simulators are forced to run synchronously for two milliseconds.

The behavior is also shown in figure 5-2. Periodically, some pin wiggles occur on a signal that connects the SystemC and HDL side. After the last pin wiggle has been observed, the turbo mode continues to enforce synchronous simulation for another two milliseconds. Only then is the SystemC side allowed to decouple and run ahead in simulated time.

6.2. 5.2 Support for Clocked Designs

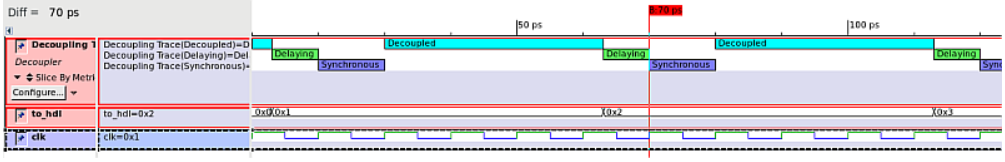
The turbo mode provides automatic support for clocked interfaces, typically buses, at the language boundary. This may seem counter intuitive, as a ticking clock at the boundary would prevent the turbo mode to decouple SystemC and HDL altogether. However, by limiting the supported clock sources to SCML clocks, it can support clocked interfaces with minimal impact on simulation performance.

To use this feature, the following requirements must be met:

- The clock generator must reside on the SystemC side.
- It must use an SCML clock object that is wired from SystemC to HDL.
- There must only be one SCML clock object at the language boundary.
- The clock parameters of the SCML clock must not change at runtime.

If these are `true`, the Co-Simulation will detect the clock generator at the language boundary and adjust its operation accordingly. No additional configuration is required. The main impact is that the turbo mode will only switch between the *Decoupled* and *Synchronous* operation at clock edges. This ensures that no clock skew will occur between SystemC and HDL. Additional clock ticks that occur while the turbo mode is in *Decoupled* mode will not be propagated to HDL.

Figure 5-3 Example3: Effect of SCML Clock at the Language Boundary



The figure 5-3 shows the turbo mode operation for a clocked design with a period of 10 picoseconds. When a pin wiggle of a non-clock signal occurs while *Decoupled*, the turbo mode does not fall back to *Synchronous* mode right away. Instead, it is delayed until the next clock edge. Similarly, it only re-enables *Decoupled* operation at the next clock edge, even if the other parameters would allow to do so earlier.

5.2.1 Using Separate SystemC and HDL Clock Sources

It is also possible to use two identical clock sources on the SystemC and HDL sides, though the user must assure that these two clocks have the exact same parameters in terms of clock period, duty cycle, start time, and so on. In addition, using clock sources that change their parameters during runtime is not supported. Since special care must be taken that the two clocks do not behave any different, this approach is rather error prone, and its use is hence discouraged.

However, it can help in certain designs where having a single clock source on the SystemC side is not feasible. For example, the HDL part of the design could include a large, complex clock tree that cannot be moved to SystemC.

When separate clock sources are used, there are two additional parameter for the turbo mode that the user must set:

- `Clock period`
- `Clock start time`

To pick up on the example from the previous section, instead of having a single SCML clock that is wired across the language boundary, it would also be possible to instead instantiate a clock source on each side, not being wired across the language boundary. Both clock sources must be configured using the same period (10 picoseconds in figure 5-3). Finally, the `clock period` parameter for the turbo mode must also be set to `10` picoseconds, and the `clock start time` parameter must be set to match the two clock sources. Once that is done, the behavior would be identical to the one from the previous example, that was displayed in figure 5-3.

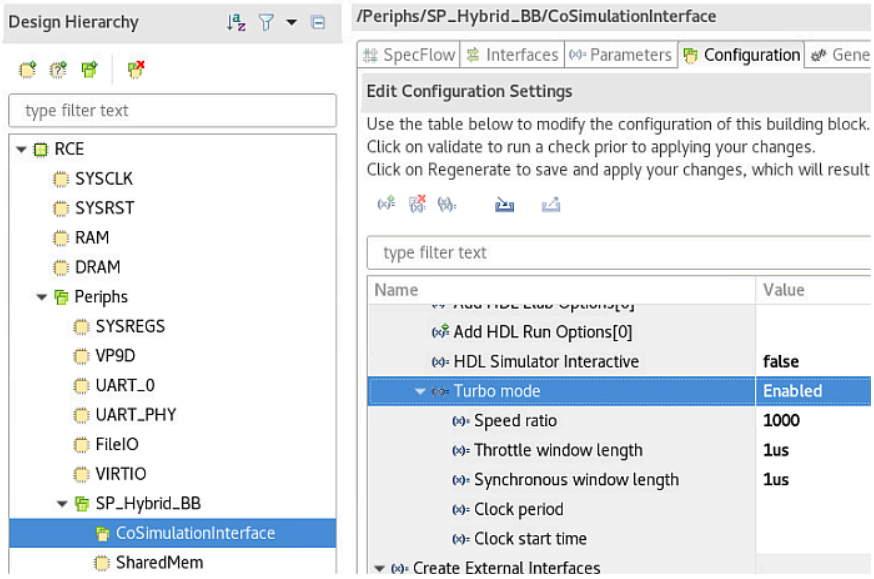
< > < >

6.3. 5.3 Enabling and Configuring the Turbo Mode for Co-Simulation

The turbo mode is supported for both the Virtualizer-centric ("HDL CoSimulation" on page 15) and the VCS-centric (for details see "VCS-Centric Co-Simulation Flow Overview" on page 112) Co-Simulation flows. Its configuration varies depending on the used flow and product.

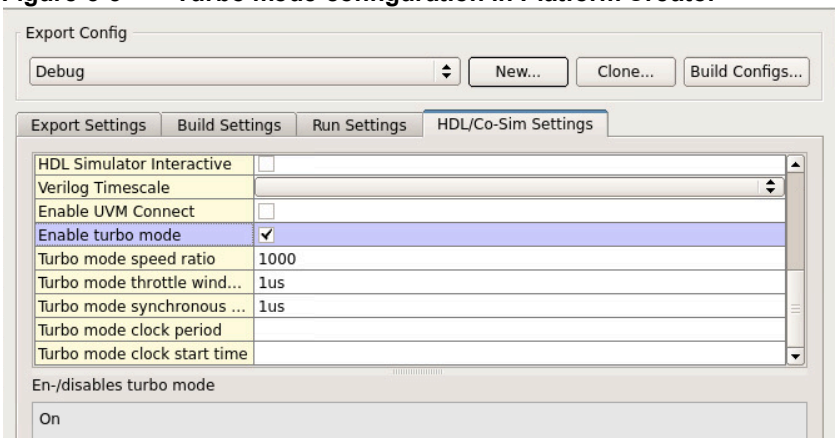
When configuring a Virtualizer-centric Co-Simulation using the `CoSimulationInterface` in Virtualizer Studio, the turbo mode can be enabled and configured through its configuration, shown in figure 5-4.

Figure 5-4 Turbo mode configuration in `CoSimulationInterface`



The same set of settings can be found in the Co-Simulation settings in Platform Architect, displayed in figure 5-5.

Figure 5-5 Turbo mode configuration in Platform Creator



For Co-Simulations that use the VCS-centric approach, the turbo mode is configured through the `cosim_helper` utility using the following command-line arguments:

- `--turbo_speed_ratio`
- `--turbo_throttle_len`
- `--turbo_sync_len`
- `--turbo_clk_period`
- `--turbo_clk_start_time`

Note	Setting the clock period and start time parameters are not required when an SCML clock source is used and should not be specified in that case.

6.4. 5.4 Limitations

There are some limitations for using the turbo mode with clocked designs:

- Only SCML clock sources on the SystemC side that are wired over to HDL are supported. Using other clock implementations will result in a regularly ticking clock at the language boundary, and will have a major impact on simulation performance.
- Multiple SCML clocks at the language boundary are not supported and will result in an error.
- Changing the SCML clock parameters at runtime is not supported and will result in an error.

7. Index

A

Accellera Systems Initiative. *See* ASI

API

definition 10

SystemC-HDL Co-Simulation 62

Application Programmer's Interface. *See* API

ASI, definition 10

B

breakpoints 86

C

C macros

SN_LIST 10

SN_TYPE 10

CLI, definition 10

clocked designs 122

command-line arguments 83

Command-Line Interface. *See* CLI

compile-time options 44

configure turbo mode 123

constraints 105

cosim::map_hdl_library_name command 84, 93

cosim::set_cosim_timeout command 81, 93

cosim::set_delta_cycle_sync command 84, 93

cosim::set_hdl_max_nesting command 80, 94

cosim::set_hdl_options command 82, 94

cosim::set_hdl_sim_mode command 81, 95

cosim::set_hdl_sim_type command 81, 96

cosim::set_verilog_timescale command 96

cosim::show_hdl_options command 96

co-simulations. *See* SystemC-HDL co-simulation

CWR_SYSC_COMPILE_OPTIONS 44

CWR_SYSC_FILES_LIST 40

CWR_SYSC_MODULE_HDR_FILE_NAME 42

CWR_SYSC_MODULE_NAME 41

CWR_SYSC_NAMED_MAPPING_REQD 42

D

debugging 85

Design Under Test. *See* DUT

DO files 85

DUT, definition 10

E

e code 10

e design 10

e file 10
e language 10
e ports 10
e units 10
error messages 44
ESI
  definition 10
exporting structural HDL 87
  command 87
  debugging and simulation control 91
  use-model scenarios 88
External Simulation Interface. *See* ESI

G

GCC, definition 10
get_hdl_lib_map command 97
GNU Compiler Collection. *See* GCC
gprof 104
Graphical User Interface. *See* GUI
GUI, definition 10

H

Hardware Description Language. *See* HDL
HDL library name 25
HDL module name 25
HDL proxy module
  checking its source 29
  creating 23
  definition 10
  in SystemC designs, examples 63
  specifying architecture 28
HDL simulator 81
HDL source files 26
hdl_proxy_gen command 30
  options 30
HDL, definition 10
hdl-options command 97
header file name 42
hierarchy
  navigating 86
  SystemC-HDL co-simulation 18

I

InterProcess Communicationl. *See* IPC
IPC, definition 10

K

known problems 106

L

language switches, setting number 80
library names, mapping 84
limitations 105

loading
SystemC source files 40
SystemC-HDL co-simulation 80

M

map_hdl_library_name command 84, 98
mapping
library names 84
ports 19, 26, 42
module name, specifying 41

P

PLI, definition 10
ports
connecting 19
mapping 19, 26, 42
profiling a co-simulation, with sprof 104
Programming Language Interface. *See* PLI

R

run-args command 99

S

sc_proxy_gen command 48
options 48
set_cosim_timeout command 81, 99
set_delta_cycle_sync command 84, 100
set_hdl_max_nesting command 80, 100
set_hdl_options command 82, 101
set_hdl_sim_mode command 81, 102
set_hdl_sim_type command 81, 102
set_hdl_structural_optimization command 87
set_verilog_timescale command 103
show_hdl_options command 103
signal types, mapping 20
simargs command 83, 104
simulation options, setting 82
simulator mode 81
Simulink Plant Model. *See* SPM
SN_LIST C macro 10
SN_TYPE C macro 10
source code, debugging 85
source file names, specifying 40
SPM, definition 10
sprof 104
structural breakpoints 91
structural browsing 91
structural HDL, exporting 87
synchronization mode, selecting 83
synchronous window length 122
SystemC adapter 10
SystemC modules, referencing 42

SystemC Proxy Generator
- C++ compile-time options 53
- command line 48
- co-simulation boundary port data types 57
- examples 57
- HDL foreign module 50
- HDL language 49
- HDL simulator 49
- introduction 48
- limitations 61
- log file 55
- nontemplatized example 57
- nontemplatized example with inout 58
- output file 55
- port mapping type 52
- SystemC header file 56
- SystemC module 50
- SystemC port types 56
- SystemC source files for compilation 54
- template support 59
- templatized example 59

SystemC proxy model
- creating 40
- definition 10
- in HDL designs, examples 71

SystemC source file names, specifying 40

SystemC wrapper 46

SystemC-HDL co-simulation
- API reference 62
- breakpoints 86
- command-line arguments 83
- debugging source code 85
- design flow 15
- DO files 85
- errors and warnings 85
- HDL simulator 81
- loading design 80
- mapping library names 84
- navigating SystemC hierarchy 86
- rules 17
- running 85
- selecting simulator mode 81
- setting number of language switches 80
- setting simulation options 82
- simulation options 82
- simulator mode 81
- synchronization mode 83
- time-out 80

T

Target Language Compiler. *See* TLC
throttle window length 121
time scale, known problem 106
timing out 80
TLC, definition 10
TLM, definition 10
top_sc_gen command 46
Transaction-Level Modeling. *See* TLM

U

UVM-Connect 119

V

VDK, definition 10
Verilog Procedural Interface. *See* VPI
VHDL package name 27
Virtualizer Development Kit. *See* VDK
Virtualizer Systems Interface.*See* VSI
VPI, definition 10
VSI, definition 11

W

warnings, suppressing 44
wrapper, for SystemC-HDL co-simulation 46